

ORION NEA

By Hamza Maniar

CONTENTS

Chapter 1 Analysis	1
Problem definition	1
Research.....	2
Client Brief	8
Features & Requirements.....	11
Project Scope	15
Chapter 2 Design	19
High-Level Design	19
Application Protocol	20
Database	29
Topic Handler Service	38
User Portal & Admin Portal Internal Services	39
User Portal	43
Admin Portal	49
Server and Router Console Justification.....	55
Server	55
Router	58
Iterative Development & Testing Plan.....	64
Chapter 3 Development & Testing	65
Sprint 1	65
Sprint 2	103
Sprint 3	125
Sprint 4	136
Sprint 5	140
Sprint 6	142
Sprint 7	150
Chapter 4 Evaluation	152

Solution Overview & Post-Development Testing	152
Final Solution Evaluation.....	153

CHAPTER 1 ANALYSIS

Problem definition

PROBLEM OVERVIEW

Organisations rely on secure and effective communication to boost productivity and collaboration. Due to the efficacy of digital solutions, many businesses utilise on proprietary software that function over the internet. This reliance on the internet can prove to be insecure since sensitive communication data ends up outside of the company's network, where it can be manipulated for malicious purposes. Malicious persons can access this data through several means. However, if the data never left the internal network, it would be much more difficult to exploit. Even within an organisation, there may be malicious actors, in which case an "intranet" chat app would have to provide security features to prevent malicious acts.

SOLUTION

The solution will be a secure chat application that runs entirely within an organisation's internal network.

The problem involves the following features that computers can easily do:

- Handling large volumes of data – Computers can handle large volumes of data very quickly and efficiently.
- Real-time communication – Computers can instantly send messages to each other, enabling real-time communication between users.
- Automatic Logging – Computers can automatically keep logs of all communications made between users.

In addition to all of this, most organisations store all their data and records electronically so, to share these with other people in a physical format, a lot of time, resources and effort would be required. Hence, it is imperative that resources and ideas are shared electronically which is only possible through a computational solution.

Computational Methods Applicable to Solution

Decomposition

The problem can be decomposed much further into many smaller problems. For example, it can be split up into the following smaller tasks (although not small enough as this is just an example):

- Sending messages
- Encrypting messages
- Storing messages
- Displaying messages

Abstraction

If the application is split up into layers, each layer would not need to know how the other layers work

Pattern Recognition

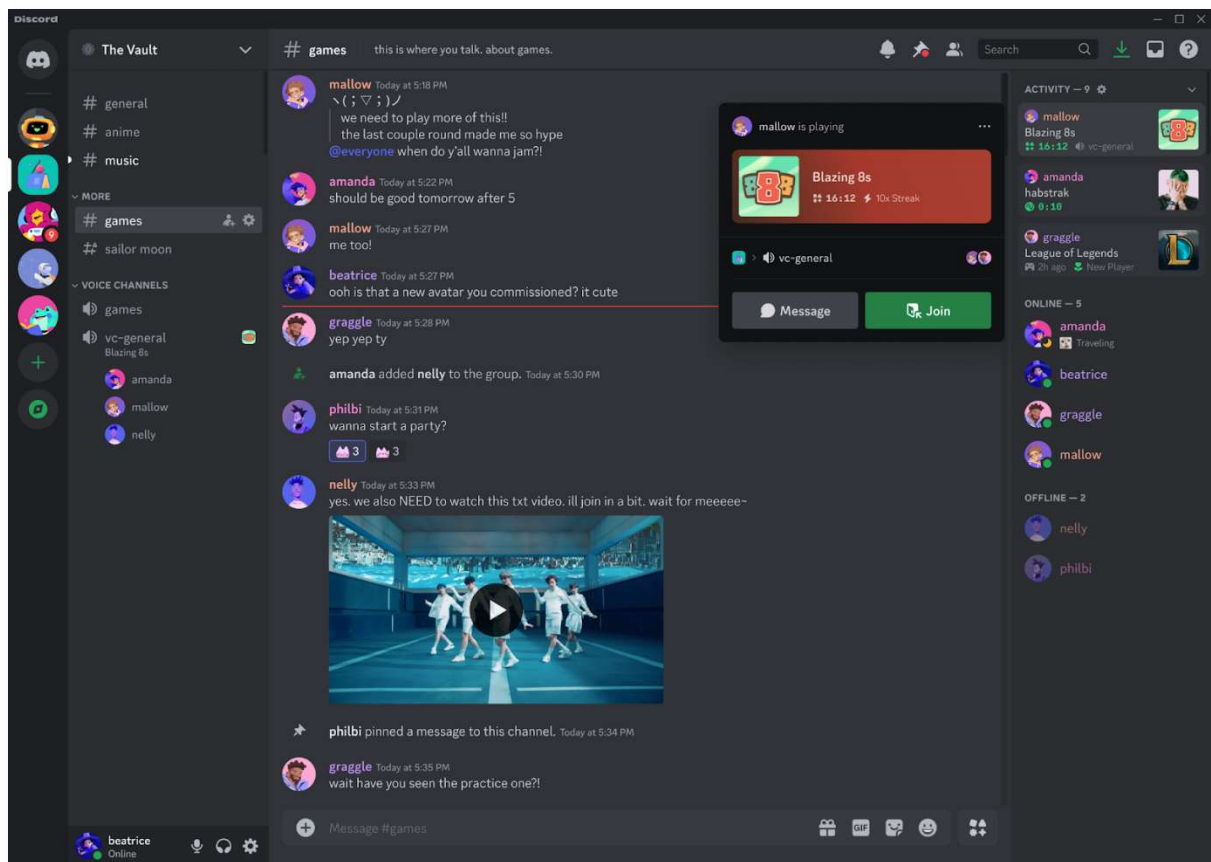
Pattern recognition can be used to detect and filter spam and provide autocompletions when writing messages among other things.

Research

SIMILAR SOLUTIONS ANALYSIS

Discord

Picture



Overview

Discord is a messaging app originally created for gamers, although it has become popular across all demographics. The user interface follows a consistent layout, with a sidebar and main content section. This consistency helps users navigate the application with ease. However, some users find Discord's interface to be "cluttered" and "overwhelming" since it can be disorganised (e.g. there is a shop in the direct messages screen) and presents a lot of options at first sight.

Feature Overview

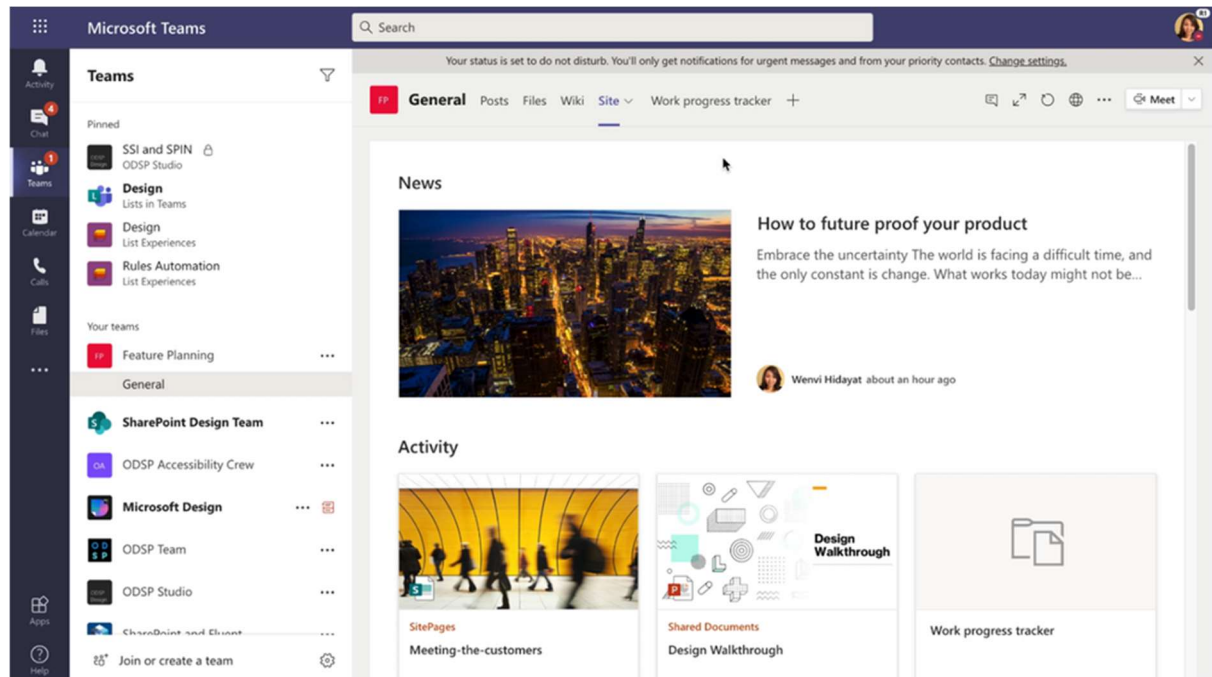
- Users can join or create "servers," which can function as either communities or chat rooms. The server's owner can give members of each server rights by using "roles". Selected users can then be given administrative rights to manage the server (i.e. maintain peace and order).
- The sidebar shows all the servers a user is part of, with a button at the top for direct messages. DMs are in a separate page to servers which creates a clear distinction between personal and grouped communication.
- Discord uses a small range of colours (different shades of the same colour for the most part) and blue accents highlighting interactive elements like buttons. One of the main aspects of Discord that makes it easy to use is its visual feedback, with animations throughout the app.
- A user status system indicates a person's availability: "online", "offline", or in "do not disturb" mode (which mutes all their notifications). This feature allows other users to gauge how quickly they will receive a response and when they can 'freely' send messages to a certain person.

Parts applicable to the solution

- Consistent layout – Helps the application stay easy to navigate and use.
- Separation of grouped and personal communication
- Minimalist colour scheme and UI Design – Helps the application be easy to use and engaging for users.
- User status – Showing when a user is "available", "offline" or on "do not disturb" is useful to indicate how available a user is at any point in time.

Teams

Picture



Overview

Microsoft Teams is an application designed for collaboration through the use of "teams." Members of an organization can create teams to group people for specific purposes. For example, a school may have one team per class. Teams provides various features such as role-based administration, channels for organization, and integration with other Microsoft Office apps.

Feature Overview

- Teams uses a 'level-based' system to manage access rights. Users are assigned a 'level' (e.g. Admin or User) and this determines what a user can do.
- Each team can have multiple channels, files are organised within these channels.
- Teams provides direct messaging which is separate to communication that occurs over channels in teams.
- Built-in support for audio and video meetings.
- Features such as Outlook calendar integration enhance productivity.
- A sidebar allows quick access to calls, chats, and notifications.
- Users can choose between a list view or a card-based layout for displaying teams.
- Attachment Handling: Attachments and their captions appear as a single message for better clarity.

Parts Applicable to the solution

- Role-based access rights are simple to manage and create
- Organising files based on the channels they were sent in may be useful

POTENTIALLY USEFUL TECHNOLOGIES & CONCEPTS

Encryption

Transmissions between the client and the server need to be encrypted to ensure that sensitive information is not exposed if a malicious user was to intercept messages. Different transmissions demand different levels of protection, for example, an important document needs to be more secure than a text message. Additionally, different transmissions demand different response times, for example a text message needs to be sent quickly while an image can take more time.

Symmetric Encryption vs Asymmetric Encryption

There are two ways of encrypting data, symmetric and asymmetric. Symmetric encryption utilises the same key for encryption and decryption while asymmetric encryption uses 2 keys, a public key used for encryption and a private key used for decryption. This means that symmetric encryption is faster and less computationally extensive to execute than asymmetric encryption however, asymmetric encryption is more secure.

Key exchanges

To increase security, every time a client connects to the server, they should receive a new encryption key. If transmissions between the server and the client are prone to being intercepted, sending a new key by itself risks the whole communication. Upon first thought, encrypting the key before sending it seems like a good idea however, it would not only lengthen loading times but would also require hard coding another encryption key in both the client and the server which could be obtained by reverse engineering the applications.

One way to send a new key to the client is by using a Diffie Hellman key exchange which uses mathematical formulas to share parts of keys that cannot be used to recreate the key itself.

Takeaways

All transmissions should be symmetrically encrypted with a new key for each connection

Attachments, login credentials and changes to user information (name changes or password changes) should be asymmetrically encrypted

A Diffie Hellman key exchange is one way to share and create symmetric keys for each connection

Compression

Compression can speed up loading times and maximise the storage space available to the server. However, some pieces of data must use lossless compression, an example would be text messages, which can increase the size of the data instead of decreasing it. In this section, the use of the different types of compression is discussed.

Lossless compression

Most pieces of data that the server will need to store, such as text messages, user credentials and group names, cannot be stored with lossy compression as any information loss, even a single

character, will mitigate the data's usability for example, a single character loss in a message could change its meaning. Thus, the only way to compress these types of data would be to use lossless compression. However, lossless compression is only effective on large pieces of data which have more patterns to abstract like PDFs, on smaller pieces of data like text messages which are usually only one sentence long there are not as many, if any, patterns to exploit. One way to counter this is to look for patterns across groups of messages such as a whole channel or a whole direct communication however, this might prove time consuming and may not save much space anyway so compressing small pieces of data, even as a group, proves to be superfluous.

Thus, small pieces of data, i.e. text messages and user credentials, will not be compressed while larger pieces of data, namely text documents and PDFs, will be stored with lossless compression.

Lossy compression

Images can take up a lot of space but can afford to lose information without changing the overall look of the image thus, they can be compressed via lossy compression. As a lot of space can be saved, the best course of action is to store images with lossy compression.

Takeaways

The solution should store files such as text files and PDFs with lossless compression, since any character loss could change the file's information. Images should be stored with lossy compression since a lot of storage space can be saved without sacrificing the information it presents.

Sockets

Overview

One way of performing inter-process communications is via sockets. A socket is an endpoint for a two-way communication between two processes (even across a network). Each socket has a unique address that is comprised of an IP address and a port number. Sockets typically follow a client-server system. The client socket connects to the server socket, after which data can be exchanged without any prompting between the client and the server in any order. This is what makes sockets so powerful, they provide an easy way to establish real time communication.

There are two main types of sockets:

- Stream Sockets – These ensure that data is reliably sent and received, there is very little risk of losing data. Here is a quote from the link below: “A stream socket is like a phone call -- one side places the call, the other answers, you say hello to each other (SYN/ACK in TCP), and then you exchange information. Once you are done, you say goodbye (FIN/ACK in TCP). If one side doesn't hear a goodbye, they will usually call the other back since this is an unexpected event; usually the client will reconnect to the server. There is a guarantee that data will not arrive in a different order than you sent it, and there is a reasonable guarantee that data will not be damaged.”

- **Datagram Sockets** – These are less reliable than stream sockets but are faster at sending data. Here is a quote from the same link: “A datagram socket is like passing a note in class. Consider the case where you are not directly next to the person you are passing the note to; the note will travel from person to person. It may not reach its destination, and it may be modified by the time it gets there. If you pass two notes to the same person, they may arrive in an order you didn't intend, since the route the notes take through the classroom may not be the same, one person might not pass a note as fast as another, etc.”

The link: <https://stackoverflow.com/questions/4688855/whats-the-difference-between-streams-and-datagrams-in-network-programming>.

Takeaways

To allow real-time communication, the solution will have to utilise sockets. For messages, files and all data that cannot be lost at all, stream sockets should be used. For phone calls and video calls, datagram sockets should be used.

Client-Server vs Peer to Peer

Client Server

Another implementation involves 2 applications, one which routes all the messages and handles all the data in one central server and another in which users send messages to each other. The application that handles all the data should present logs of every interaction that happens on the network within the application this means it should only be accessed by administrators and should have a login system.

Having a centralised location for storing all the data and routing messages simplifies the applications that will run on users' computers to just sending messages to the server and retrieving them while all the processing can happen on the server. Having centralised location for all the data also mitigates the need to sync data between client computers as when data is updated, removed or created, it is all done on the server and accessed through that server.

However, setting up the application will be complicated and expensive as the client will need a server and be able to manage and maintain the network between the server and the clients.

Advantages	Disadvantages
More secure	Complicated and expensive to set up
Simpler user applications	May be expensive to maintain as network needs to be maintained
No need to sync data between user applications	
Can be faster as the server can be dedicated better hardware	

Peer to Peer

An easy to set up implementation would be to have a single application which is accessed by all users of all permissions. The application would present different options for users with different access levels e.g. admins would be presented with the ability to add user accounts while ordinary users would only see the bare minimum.

While this would be simple to set up and use, it presents issues such as having to sync information across devices. For users to be able to sign in on different devices a copy of all the data of all users will have to be stored and updated on each application which will use up too much memory and place too much responsibility on the client applications.

Advantages	Disadvantages
Easy to set up	Too much responsibility on client device
	Added complexity in syncing data

Takeaways

It is evident that for maximising security and speed, it is essential to use the client server approach over the peer-to-peer approach as shown by the summary tables depicting advantages and disadvantages of each stratagem.

Client Brief

TARGET MARKET & DEMOGRAPHICS

Chat apps of this nature (run and maintained fully within a single network) will typically be used within organisations. In these organisations, there are three types of people that will interact with the app:

- **Users** – These will be the normal users of the chat app. They will be the actual people that communicate using the chat app.
- **Administrators** – These will manage all the information in the chat app. They will need functionalities to view all information in the chat app and update user information or create new groups for projects.

Bobby Contract owns a small business and will be representing the **administrators**. He needs secure and reliable communication as his projects can contain sensitive information. He would use the solution to manage and facilitate communication between the members of his business to boost their productivity and provide more quality to his clients. For brevity, he will be referred to as the Admin or Administrator.

Bob Jefferson is a member of the business and will be representing the **users**. He will use the application to communicate with other members of the business and share important documents. For brevity, he will be referred to as the User.

CLIENT DISCUSSIONS

Initial Briefs

The Admin stated that “I want to be able to manage all users, groups and channels on the chat app.”

The User stated that “I want to be able to send messages, files and attachments to other people using the chat app. It would be nice to have chats where I can chat one on one to other users and other chats where I can chat to a group of people, they may be members of my team, for a specific project.”

Questions

To the User I will ask the following questions:

1. How often will you be using the chat app?
2. You mentioned that you work on projects, how big are those projects, do you think you need some kind of separation within the grouped chat room?
3. How concerned are you about privacy?
4. Do you want anyone to be able to message you one on one?
5. How important is speed and message delivery reliability?

To the Admin I will ask the following questions:

1. What level of control do you want over users?
2. Do you want to be able to give other people rights to manage users and groups?
3. Do you prefer role-based or rights-based authorisation?
4. Should users be able to change their information themselves?
5. What logging and auditing features would be useful for tracking user actions?

Answers

The user answered their questions below.

1. How often will you be using the chat app?

I will be using the chat app daily, primarily during working hours, to collaborate with my team and stay updated on project discussions.

2. You mentioned that you work on projects, how big are those projects, do you think you need some kind of separation within the grouped chat room?

Our projects vary in size, but most of them are handled by a single team. It would be helpful to have separate sub-groups or channels in groups for managing different deadlines or features for projects.

3. How concerned are you about privacy?

Often the projects I work on, and the company works on, requires sharing sensitive information. It is vital that none of this information can be easily leaked.

4. Do you want anyone to be able to message you one-on-one?

I'm open to direct messages from colleagues as it is a small company.

5. How important is speed and message delivery reliability?

Speed and reliability are crucial. Delays in message delivery could impact decision-making and project progress, so I need messages to be delivered in real-time with minimal downtime.

The Admin answered their questions below:

1. What level of control do you want over users?

I need the ability to manage user access, create and remove accounts, assign roles, and enforce security policies. It would also be useful to control permissions for messaging, group creation, and data access based on organizational needs.

2. Do you want to be able to give other people rights to manage users and groups?

Yes.

3. Do you prefer role-based or rights-based authorization?

A role-based authorization system would be preferable, where users are assigned predefined roles (e.g., Admin, Manager, Employee) with specific permissions. However, some rights-based customization should be available for special cases.

4. Should users be able to change their information themselves?

Users should be able to update basic profile details like their display name, profile picture and password themselves. However, I would also need the ability to update this information.

Insights

User Insights:

- Frequent Usage - Users will rely on the chat app daily and often so it is vital that it can run for a long time.
- Privacy Concerns – Since projects information can be sensitive, the app needs to be very secure

Admin Insights:

- User Management Control: Admins require full control over user access, roles, and security policies, ensuring structured and secure usage.
- Delegation of Responsibilities: Admins should be able to assign management rights to others.
- Role-Based Authorization: A role-based system is preferred, with potential for rights-based adjustments for specific cases.

Features & Requirements

SECURITY FEATURES & REQUIREMENTS

Feature/Requirement	Justification
All transmissions must be symmetrically encrypted at least	Encrypting all transmissions is necessary to prevent malicious users from intercepting data, even if they intercept messages being sent within the app, they should not be able to find out what the messages contain. Symmetric encryption is faster than asymmetric encryption so its favoured over asymmetric encryption to reduce response times.
Attachments, login credentials, changes to user information (name changes or password changes) and all transmissions containing sensitive information should be asymmetrically encrypted	This prevents a single point of failure if the symmetric encryption key for a given connection was found.
A new key should be generated for every connection	This also prevents a single point of failure: if only a single key was used for the entire application, it may get leaked rendering the entire encryption useless. Even if the key was changed periodically, it is still exploitable in the same way. Thus it makes sense for every connection to have their own key.
When a new key is generated, it should use a Diffie Hellmann like approach	This prevents the key being intercepted when its transported.

BACKEND FEATURES & REQUIREMENTS

The backend will need to be set up and maintained by a specialised technician, this is so they can maintain it and keep it up and running.

Feature/Requirement	Justification
Should be simple and quick to set up, run and close.	Even though this will be maintained by a professional, it should be easy for them to open and close for some arbitrary server maintenance reason.
Should be able to see logs of every request made to the server	This is essential to deduce suspicious requests for example, if a malicious user set up a script to send 1000 requests to the server in a second in the hopes to crash it. It may also be useful for maintenance and allocation of server resources so the server can be optimised to best match how users are using the app
Should be able to effectively handle multiple requests at once	A lot of requests will be sent at the same time, especially when users are communicating with each other which will happen a lot

FRONTEND FEATURES & REQUIREMENTS

Normal User Portal Features & Requirements

Features/Requirement	Justification
Simple and intuitive interface	Users should be able to use the app without being familiar with it, this will allow them to quickly and easily migrate to using the app
Should be clear what group/user the user is talking to	The user should not be easily confused and potentially leak insider project information to a group meant for a different project or something of the likes
Add Direct Messages Button	Users should be able to choose who they can one to one chat to, this way they do not have to navigate between a massive number of direct messages before finding the one they are interested in
Messages should be ordered by last updated	The first thing users should see is the latest conversation, and the deeper they search the older the conversation should be, this conforms to norms maintained by virtually all chat apps
It should be clear who sent a message	Users need to be able to see clearly who sent them a message so they can contextualise and respond accordingly

It should be clear when a message was sent/last updated	Users need to be able to keep track of their conversation history and it allows them to recognise how long it took them to respond among other things
Users should be able to see and edit their information	Users should have the ability to change their own password and username, in the unlikely event a user sets an inappropriate username, the administrators will be able to change their username back and then potentially take further action
Users should be able to edit their messages	Editing messages allows a user to make corrections, whether its adding clarity to a message or correcting spelling errors
Users should be able to delete messages	Users may want to delete a message to replace it with a clearer one
Each user should have to sign in with their credentials	This ensures users cannot see messages meant for other users and allows users to sign into their account on any device on the network

Admin Portal Requirements

Feature/Requirement	Justification
The app should only be accessible after authenticating the user (i.e. have a login screen)	The admin portal provides the ability to view and change all information in the network of everyone that uses the app, it would be incredibly unsafe to leave that ability accessible to anyone with physical access to the admin portal (anyone with a download of the admin portal)
Only admins should be authorised to access the app	Only entrusted people should be able to access the app, not just anyone. EXPAND
If any other user tries to login to the admin portal, they should NOT be told that they do not have the authority to access the app, instead they should simply be told the same error message an unrecognised set of credentials would produce	If a malicious user tries to sign into the admin app and they use the credentials of a normal user (not an administrator), although they may not gain access to the admin portal, they will have found out the credentials of another user which could be easily misused.
Should be able to delete users	If a person leaves the organisation, keeping their information may be a waste of space and keeping users even after they are no longer a part of the organisation provides a gateway for unauthorised users to access the app and network and potentially cause harm
Should be able to edit all user information	If someone wants to change their name, the app should be able to reflect that change in addition changing passwords is a necessity for security,

	whether someone discovers their password has been leaked or for routine password changes.
Should be able to add users	People will join the organisation; they will need new accounts.
Should be able to delete groups	If a group is created for a specific project, and that project has been closed/finished for a long time, keeping the group will be a waste of space.
Should be able to edit all group information, including member lists	People can be reassigned to different projects, so users may need to be added and removed from groups pertaining to projects. In addition, project names and related information may change and keeping old project names can foster confusion.
Members should be added to groups by their username	Even if there are only 20 users, having a dropdown for all of them will be hard to navigate and it will be easy to mis-click the wrong person, it would be better for admins to add users by their name.
Should be able to add groups	New projects can come in which would require new groups.

LIMITATIONS

Limitation	Justification
There will not be a notification system	A notification system is very useful but would require too many computational resources to maintain for the current project.
No load balancer will be built	The app is going to be used by a small company, so there is not going to be a large enough number of users and connections at any one time to credit a load balancer
There will not be systems that automatically back up data	Since the company is small, they will not have sufficient resources to permit consistent backups of all the messages on the chat app
There will not be a file editor (e.g. no support for cropping, rotating and annotating images)	This is handled by a lot of other tools and apps that are free and easy to use, and it does not have a major impact on the usability of the app, it would be better
Not all data in the database will be encrypted	If all data in the database was encrypted, it would increase the response time for every single request (i.e. sending a message, updating a group etc.). In addition to this, the changing of encryption keys used for the data would be very expensive computationally as it would require decrypting and encrypting every piece of data in the database. Some pieces of data might be very sensitive like user passwords or file attachments, so they may be encrypted and it is acceptable to

	have slow response times for operations involving attachments, so they may still be encrypted and stored, however it's worth noting that if a lot of attachments are stored, maintaining their encryption keys will become computationally expensive.
--	---

Project Scope

LIMITATIONS SUMMARY

- The app may not be able to handle a large number of users (100+)
- The app will not have an automatic backup system
- The app will not contain a file editor
- Not all data stored in the database will not be encrypted

SUCCESS CRITERIA

The following success criteria have been listed and organised based on the requirements above:

1. Real-time Messaging Criteria:
 - a. Messages update in real-time.
 - b. Messages are reliably delivered to only the recipients requested (A message meant for only Alice should not be sent to Bob).
2. Direct Communications Criteria:
 - a. Users can add their own direct communications with other users
 - b. Only users within a direct communication can see it and send messages within it
3. Message History Criteria:
 - a. All messages sent within a communication (direct messaging or grouped), that have not been deleted, are listed chronologically.
 - b. Last updated time or sending time is displayed with messages.
 - c. If messages are updated, their previous versions are not visible except in logs.
 - d. If messages are deleted, they cannot be viewed except in logs.
4. Security Criteria:
 - a. All transmissions should be symmetrically encrypted.
 - i. Each connection should generate and use a different symmetric encryption key.
 - ii. Symmetric keys should not be explicitly shared, the sharing of a key should happen through a key exchange algorithm such as Diffie Hellmann.
 - b. Sensitive information such as file attachments and user credentials should be asymmetrically encrypted when sending and receiving data.
5. Access levels are correct
 - a. Normal users can send and receive messages.
 - b. Administrators can create and edit groups and users.
 - c. Only super administrators can create other administrators,
6. Attachments Criteria:
 - a. Support for sending images, and files (.txt, .pdf, .png, .jpeg – most common file types).
7. Group Criteria:
 - a. Messages sent to groups are received by every member of the group.
 - b. Everyone can see the members of any group they are part of.
 - c. Groups are displayed with the group's photo and name.
 - d. Messages are sent within channels in the group
8. Logging Criteria:
 - a. Admins can track app usage and activity via logs.
 - b. Every interaction is logged
 - i. Interactions include direct messages, grouped messages, message deletions, message changes, new users, user account changes and user deletions.

How To Evidence Success Criteria

Criteria to Evidence Table

Criteria	How to evidence
1a: Messages update in real-time without needing a refresh.	Screen recording of 2 instances of the application sending messages to each other in real-time.
1b: Messages are reliably delivered to only the recipients requested (A message meant for only Alice should not be sent to Bob).	Screen recording of 3 instances of the app showing 2 of them sending messages to each other and the third one receiving no evidence of the other 2 communicating.
2a: Users can add their own direct communications with other users.	Screen recording of a user adding a direct communication.
2b: Only users within a direct communication can see it and send messages within it.	Screen recording of 3 instances all with direct communications open showing how all three cannot and do not share any direct communication.
3a: All messages sent within a communication (direct messaging or grouped), that have not been deleted, are listed chronologically.	Screenshot.
3b: Last updated time or sending time is displayed with messages.	Screenshot.
3c: If messages are updated, their previous versions are not visible except in logs.	Screen recording of editing messages and showing how no matter what happens, even by restarting the application, the old version of the message cannot be displayed.
3d: If messages are deleted, they cannot be viewed except in logs.	Screen recording, similar to the one above except with deleting messages instead of editing them.
4a: All transmissions should be symmetrically encrypted.	Screenshot or screen recording of WireShark capture with transmissions between the app showing how none of the transmissions are decipherable.
4b: Sensitive information such as file attachments and user credentials should be asymmetrically encrypted when sending and receiving data.	Symmetrically decrypt the data with the key using a bunch of debugging and tracking to get the key (malicious users would not be able to do this) and then see how the resulting plain text is still encrypted.
5a: Normal users can send and receive messages.	Screen recording of a normal user sending messages and receiving messages.
5b: Administrators can create and edit groups and users.	Screen recording of how a normal user cannot create or edit groups while administrators and super administrators can.
5c: Only super administrators can create other administrators.	Screen recording of how administrators and normal users cannot create other administrators while super administrators can.

Criteria to Evidence Table - Continued

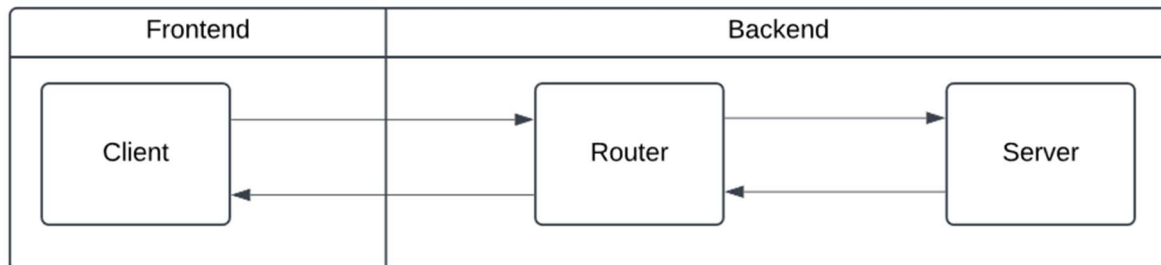
Criteria	How to evidence
6a: Support for sending images and files (.txt, .pdf, .png, .jpeg – most common file types).	Screen recording of sending files.
7a: Messages sent to groups are received by every member of the group.	Screen recording of 3 instances of the app sending messages on the same group.
7b: Everyone can see the members of any group they are part of.	Screenshot of a user viewing the members of a group.
7c: Groups are displayed with the group's photo and name.	Screenshot of group photo and name in groups list.
8a: Admins can track app usage and activity via logs.	Screen recording of logs logging when users are using app.
8b: Every interaction is logged	Screenshots for each interaction.

CHAPTER 2 DESIGN

High-Level Design

DIAGRAM

The High-Level Design is shown in the diagram below.



BREAKDOWN

Router – middleware responsible for routing messages to and from the server, all communications from the client to the server will go through the router.

Server – responsible for adding, deleting, and updating the data on the database based on client requests. The server will also manage authentication and authorisation to ensure that data cannot be accessed or affected by those that do not have permission to do so i.e. a user cannot request to add a message to a group if they are not in that group.

Client – this represents the person making changes to data in the chat app. The client includes both the admin portal and the normal user chat app, both will make requests to the server to change data.

JUSTIFICATION

Having an intermediary between the server and the clients reduces the load on the server. The server will not have to worry about maintaining concurrent connections or handling sudden client disconnections due to power cuts and the like. This will help reduce response times. The server will also not have to keep track of which users are signed in so transmissions informing changes to shared resources (e.g. a new group message) will just involve the server informing the router which users to send the notification to while the router forwards it to the users out of that whitelist that are signed in.

Application Protocol

LAYERED APPROACH

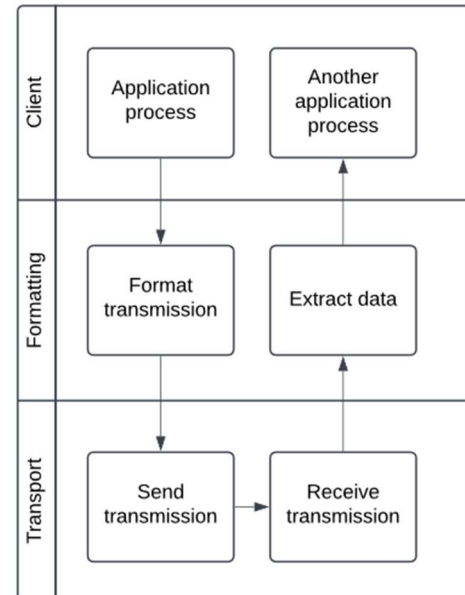
Although different clients may accept different formats of data, they all need to be able to communicate with the router, so transmissions between the client need to be standardised. The best way to do this is with a layered approach.

As shown in the diagram to the right, there are three layers:

Client – An abstraction for the router, server, user portal, and admin portal. Each application may receive data in different forms i.e. the server expects a different data format than the router, but each application needs to communicate with each other. This layer will fulfil the remaining success criteria.

Formatting – Standardises transmissions between applications and handles encrypting and decrypting the data. This layer will satisfy success criteria 4a. and 4b.

Transport – Handles transporting the data between applications. This layer should ensure that data is sent and received and that no data is lost or corrupted. This layer will satisfy success criteria 1a. and part of 1b. by sending transmissions reliably and in real-time.



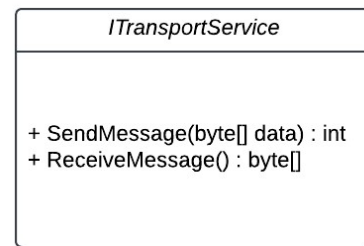
A layered approach like this separates the logic of the program which ensures that if a section of code were to change, it would not affect other sections of the code. For example, if the mode of transportation were to change, only the transport layer would need changes and, the rest of the code would function as before with no changes to them. This makes the code more maintainable and, the separation also allows us to test individual parts of the code effectively.

Note: this is **NOT** replacing the TCP/IP protocol stack – all of this belongs in the application layer of the TCP/IP stack.

TRANSPORT LAYER

Abstraction

Overall, the transport layer will provide 2 subroutines corresponding to sending and retrieving data. For clarity, they will be named **SendMessage** and **ReceiveMessage** to signify the fact that they send and receive whole messages and not just parts of data.



SendMessage takes in a byte array representing the data to send and outputs an integer that specifies how many bytes of data were sent, this allows the other layers to verify that all their data was sent.

ReceiveMessage outputs a byte array representing the data it received.

The transport layer will be implemented in a single class, with its abstraction represented in the diagram as **ITransportService**. Using an abstraction allows for multiple implementations of the transport layer. For now, there is only need for a single implementation for now which will use TCP to ensure reliable data transmission. However, if in the future, video calls need to be added, a different implementation of the transport layer could be used under the same abstraction, utilising UDP instead of TCP. One caveat to using these abstractions is that a transmission sent by one implementation of the transport layer can only be received by that same implementation.

Implementation

As demonstrated in the analysis, data will be sent over sockets. Although useful for real time applications sockets are reliant on the buffer space allocated by both the receiver and the sender for the corresponding actions. This means that in a single receive operation, all the data may not be retrieved and likewise, in a single send operation, all the data may not be sent. In addition, exceptions may be raised when trying to send data that cannot fit in the send buffer allocated by the operating system.

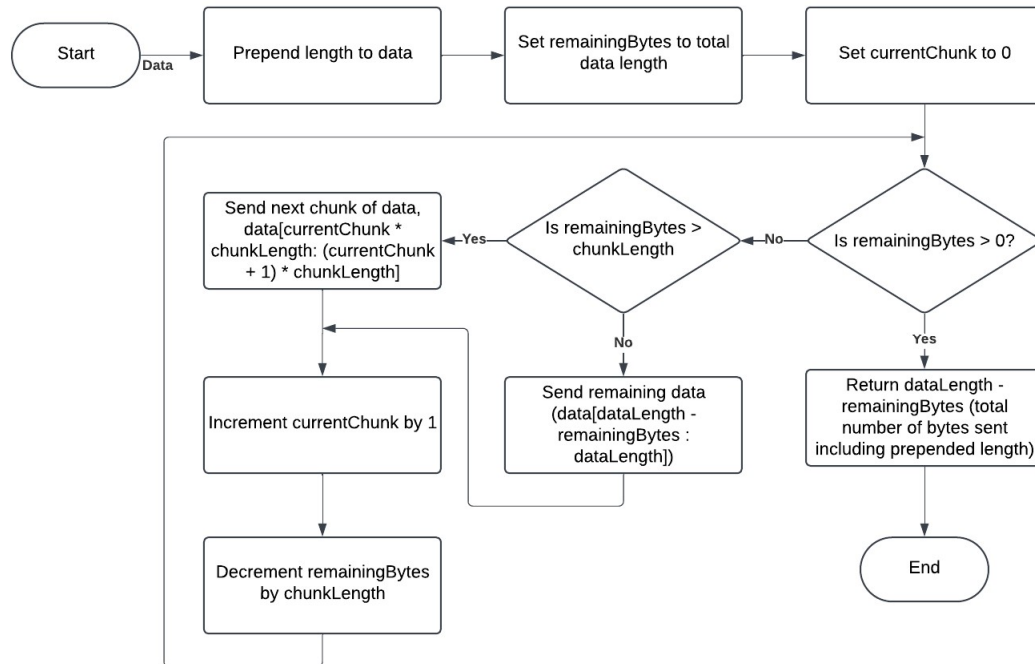
To overcome this, transmissions should be prepended with their length in bytes and, they should be sent in chunks of an appropriate length. The length allows the receiving side to continue receiving data until the full transmission is received and chunking the data prevents overflowing the send buffer.

Storing length in transmissions – Integers are represented by 4 bytes and the length of a byte array in C# is stored as an integer, so the first 4 bytes of transmissions will be the length of the transmission. The length does not need to be encrypted as it does not carry any information that could be misused by malicious people (they can also simply count the bytes being sent themselves).

Chunking - An appropriate chunking size should be big enough to send average sized messages in one go and should be small enough to not overflow the send buffer. A good value is 16384 (2^{14}) which is small enough to not overflow the send buffer and is big enough to send 2044 characters of UTF-8 text (roughly 400 words) which is plenty for most text messages to be sent in one operation.

SendingMessage

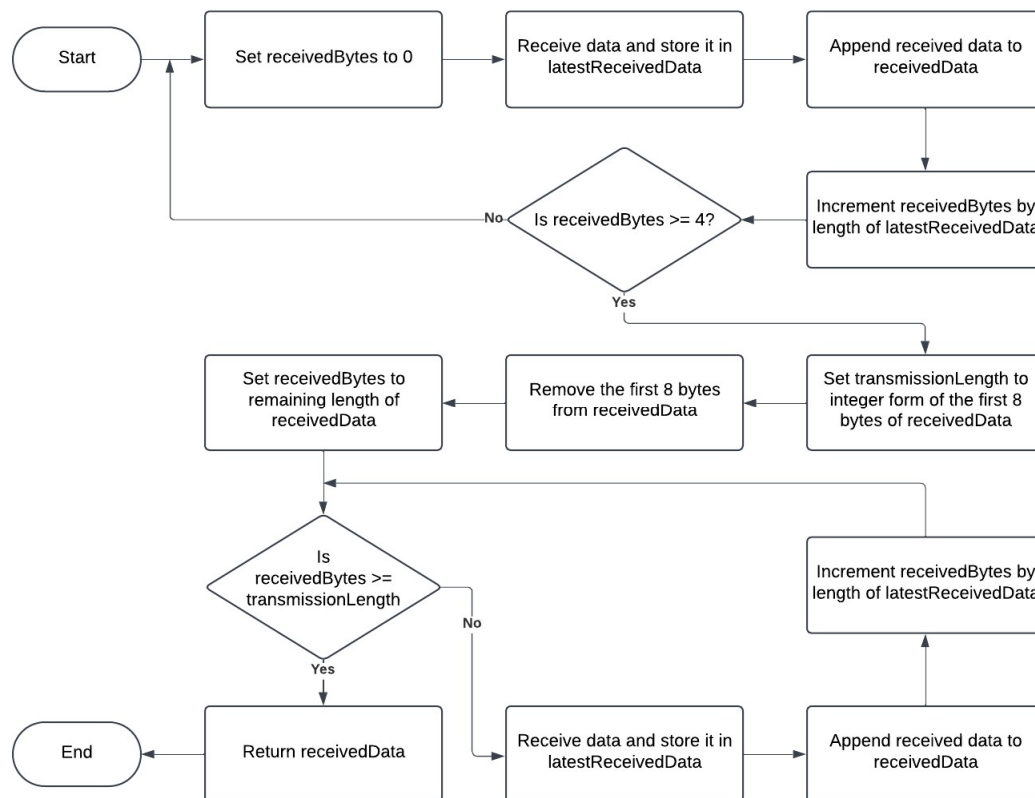
Based on the information above, the flowchart below was created for sending data:



The `chunkLength` referred to in the flowchart is 16384, as discussed previously.

ReceiveMessage

The logic for receiving messages is depicted in the flowchart below:



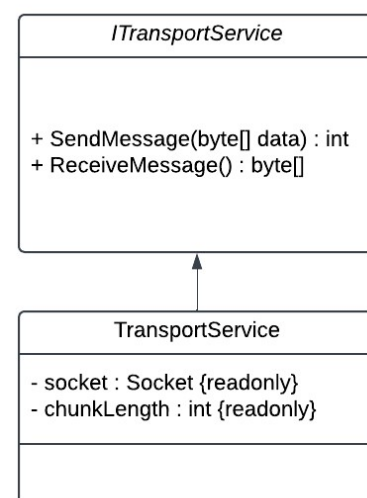
Since every transmission will have at least 4 bytes that depict the length of the transmission, the algorithm first keeps receiving until 4 bytes are received. It then converts the first 4 bytes to an integer representing the length of the ‘payload’. It then keeps receiving until it receives the number of bytes specified by the length.

Summary

The implementation of the transport layer is described in the diagram to the right and it will be called **TransportService**.

As shown, it will inherit from the abstraction to enable polymorphism, where the other layers are independent of the individual implementation.

The implementation will need two more attributes than the abstraction, one that stores the socket used to send and receive data named **socket** and another that stores the chunking length named **chunkLength**. As shown, both will be private and read-only to prevent them from being changed unexpectedly within the class and outside the class.



FORMATTING LAYER

Abstraction

Overall, the formatting layer will provide the functionality to initialise the connection, send data and receive data.

<i>ITransmissionService</i>
+ Initialised : bool = false
+ InitialiseConnection() + TransmitData(object data, bool isHighRisk) + ReceiveData<T>() : T

This will be via subroutines named **InitialiseConnection**, **TransmitData** and **ReceiveData** respectively. The property that states whether the connection has been initialised or not is called **Initialised**, it will be read only to prevent accidental tampering.

Like the transport layer, the formatting layer will be implemented in a class and its abstraction is depicted in the class diagram, and it is called **ITransmissionService**.

InitialiseConnection will handle all operations that should take place before any data is transmitted or received. These operations will mainly be for securing connections, one such operation could be a key exchange. When this subroutine is completed successfully, it should set **Initialised** to true. If this subroutine is called when **Initialised** is already true, it will not run as both sides will need to initialise at the same time to prevent errors – the other client may be using old encryption keys while this client has replaced theirs with new ones.

TransmitData will send data securely and ‘standardise’ data so it can be processed by any programming language (e.g. by converting data to JSON). Data can only be transmitted when the connection has been initialised. This takes in an object representing what data to transmit. Some implementations of **TransmitData** may send data differently if they are ‘high-risk’ (e.g. by asymmetrically encrypting high risk data such as user credentials) which is why **TransmitData** will also take a boolean parameter **isHighRisk**.

ReceiveData will receive each transmission as the required data type, it will undo all operations done to the data by **TransmitData** and extract the actual payload. This takes in a type parameter that specifies what datatype the payload should be and outputs data of that type.

Implementation

To ‘standardise’ transmissions, **TransmitData** should convert data to JSON and convert the JSON to binary with UTF-8 encoding. Likewise, **ReceiveData** should convert the received binary to a string via UTF-8 encoding and should convert the received JSON back into an object.

For transmissions to be secure, they will all be symmetrically encrypted before being sent in accordance with success criteria 4a. Additionally, to conform to success criteria 4b, sensitive information will be asymmetrically encrypted. This means that during initialisation, asymmetric key pairs should be generated and their public keys should be exchanged between clients. To fulfil

success criteria 4a_{ii}, the initialise subroutine will perform a Diffie Hellman key exchange along with generating a new asymmetric key pair.

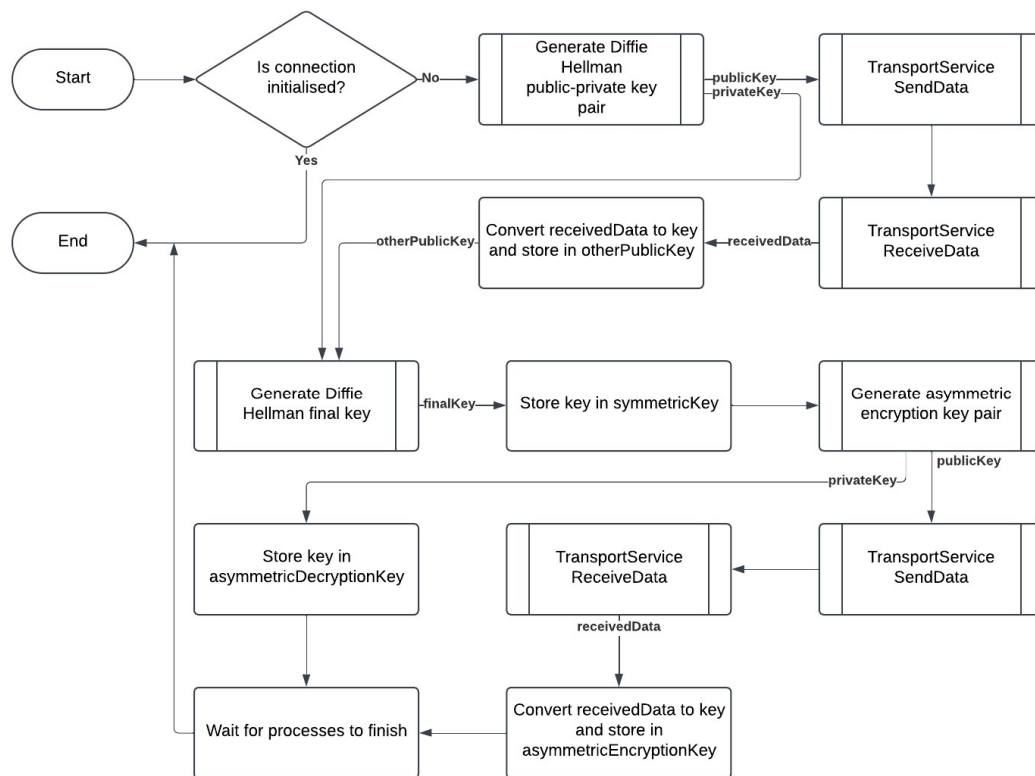
InitialiseConnection

As described above, the following initialisation tasks need to be carried out for every connection:

Diffie Hellman key exchange -A new symmetric encryption key will be generated and shared between the clients using a Diffie Hellman key exchange.

Asymmetric encryption key exchange – A new public-private key pair will be generated for asymmetric encryptions. The public key will be sent to the other client and the other client's public key will be stored. The other public key will be used for encrypting 'high-risk' transmissions and the private key will be used to decrypt asymmetrically encrypted transmissions.

The flowchart for this implementation is shown below:



The attributes symmetricKey, asymmetricEncryptionKey and asymmetricDecryptionKey are assigned values in this subroutine as shown in the flowchart. These attributes will be needed to encrypt and decrypt transmissions and, they will be kept private as they should not be accessible outside of the class.

TransmitData

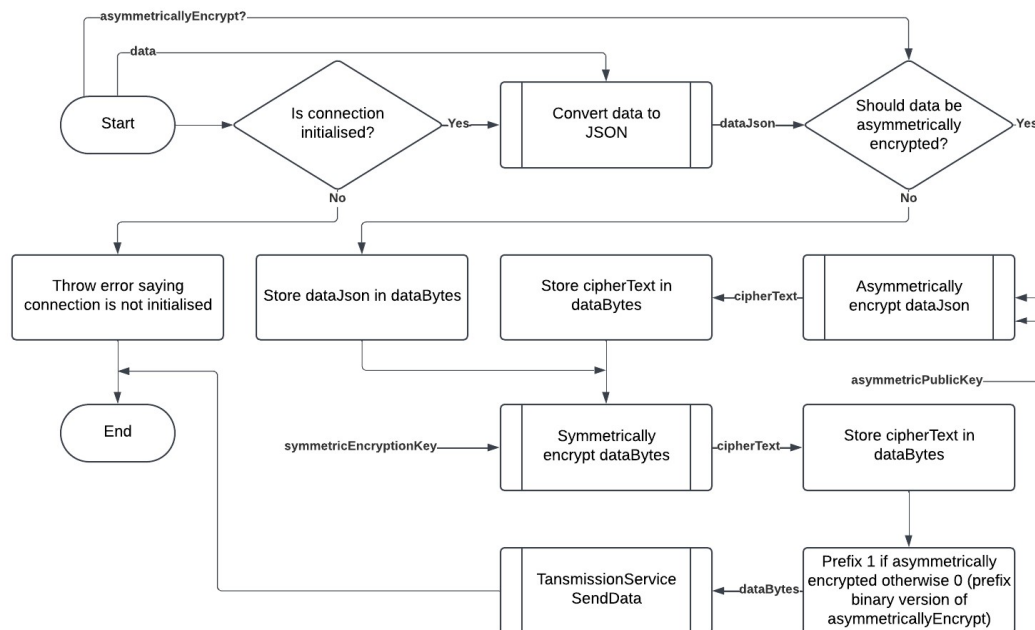
As described above, the following steps have to be taken before sending data:

1. The data should be converted to JSON
2. The JSON should be converted to binary using UTF-8 encoding
3. The binary representation of the data should then be symmetrically encrypted
4. If the data is 'high risk', it should be asymmetrically encrypted

The **ReceiveData** subroutine should be able to deduce whether the data has been asymmetrically encrypted or not. This is because the receiving end should not be concerned with whether the data was high-risk. Hence, a flag will be prepended to all transmissions: if it is 1, the rest of the transmission was asymmetrically encrypted otherwise, if it is 0, the transmission was just symmetrically encrypted.

In C#, booleans are stored in a single byte, the lowest order bit of that byte is 1 if the boolean is true otherwise it is 0. This means that the asymmetrically encrypted flag bit can simply be derived from the parameter `isHighRisk`, by converting it to binary and taking the lowest order bit.

The algorithm for sending data is shown in the flowchart below:



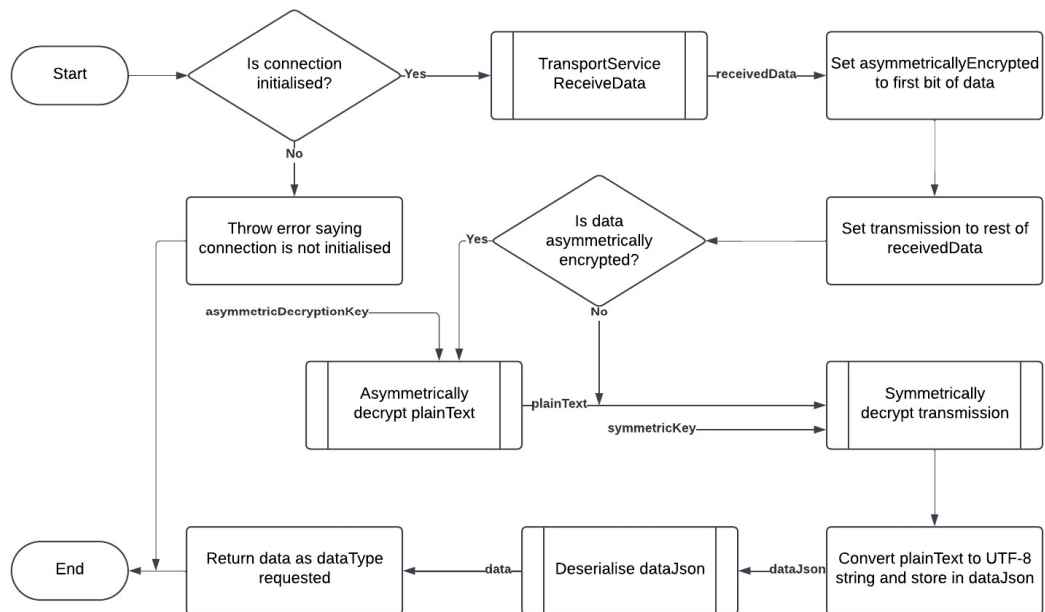
The `isHighRisk` parameter is named `asymmetricallyEncrypt` in the flowchart for clarity. In addition, the subroutine uses the properties `symmetricKey` and `asymmetricEncryptionKey` for encrypting data, so it cannot be called when the properties have not been assigned to. This is why the subroutine throws an error when the class is not initialised.

ReceiveData

To undo the formatting done by SendData, the following steps will have to take place:

1. If the data has been asymmetrically encrypted, asymmetrically decrypt it
2. Symmetrically decrypt the data
3. Convert the data to a string using UTF-8 encoding
4. Convert the JSON string back to the datatype requested

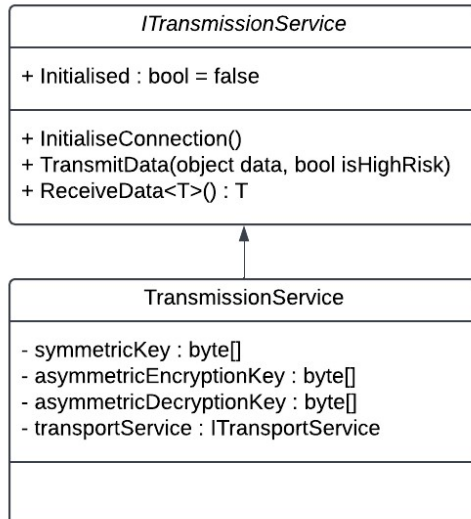
The algorithm for this is shown in the flowchart below:



The subroutine uses the properties `symmetricKey` and `asymmetricDecryptionKey` for decrypting data, so it cannot be called when the properties have not been assigned to. This is why the subroutine throws an error when the class is not initialised.

Summary

The implementation of the formatting layer is shown below and is called **TransmissionService**. Like the **TransportService**, the **TransmissionService** will inherit from the abstraction to enable polymorphism.



The extra attributes shown in the diagram are described below.

symmetricKey – The symmetric key used for encryption and decryption

asymmetricEncryptionKey – The public asymmetric key from the recipient used for encrypting high risk transmissions

asymmetricDecryptionKey – The private asymmetric key used for decrypting high-risk transmissions

transportService – The instance of the transport layer used for the actual sending and retrieving of data.

All of these are private to prevent them from being changed unexpectedly outside the class. The **transportService** is read-only so that it cannot be changed at all, if a new connection needs to be created, another instance of the **TransmissionService** should be created.

CLIENT LAYER

Transmissions

Clients need to be able to quickly discern the purpose of transmissions. A simple way to do this is to attach a ‘topic’ to every transmission. The topic can describe what the transmission contains e.g. “AuthenticationRequest” for an authentication request.

Every transmission can be an object to make it easy to debug and maintain. An abstract class for this object is described in the table below, this will not be written in code and is more of just a structure to follow for all transmissions.

Transmission		
Attribute Name	Datatype	Description
Topic	String	The topic of the transmission that specifies what kind of transmission it is.
Data	Object?	The actual “payload” of the transmission.

Database

ENTITIES

Entity Explanations

Entity	Description	Explanation
User	Any user of the application, includes admins and normal users.	Each person of the organisation (or anyone that will use the application in any way) will need a profile to sign into. The only other way to keep everyone's messages and groups separate would be by IP address which would restrict people to using a single node for their account.
Group	A group.	Success criteria 7a to 7c require groups.
Channel	A channel in a group.	Success criteria 7d requires channels within groups.
Message	A message, represents both group messages and direct communication messages	The entire chat app revolves around users sending messages to each other.
Direct Communication	Conversations between only 2 users.	Success criteria 2a and 2b.
Log	A log.	Success criteria 8a and 8b.

Data Models for Entities

Entities will be represented in the code as objects. Each entity may also have many DTOs for different types of requests. This will just make it easier to work with and transport entities between the applications.

User

User		
Attribute Name	Datatype	Description
UserID	Guid	Unique identifier.
Username	String (Max 100)	The username, maximum 100 characters to save storage space and require less time to transport.
Role	String	The user's role, either "ADMIN", "USER" or "SUPERADMIN".
PasswordHash	Byte[]	The password hash of the user. The actual password of

		the user will not be stored to protect against data breaches.
ProfilePhoto	Byte[]	The profile photo of the user.
Messages	Message[]	All the messages sent by the user.
DirectCommunications	DirectCommunication[]	All the direct communications the user is a member of.
Groups	Group[]	All the groups the user is a member of.

The user portal and admin portal will use the following DTO to represent credentials when signing in.

Credentials		
Attribute Name	Datatype	Description
Username	String	The username.
Password	String	The password, hashing it before sending can be a security risk so it will be sent as is (i.e. as plain text).

Often, the only information of a user needed will be its username, id and profile photo (e.g. when represented as a member of a group). This calls for its own class.

UserProfile		
Attribute Name	Datatype	Description
UserID	Guid	The userID.
Username	String	The username.
ProfilePhoto	Byte[]	The profile photo.

User information will need to be transported e.g. when a user signs in with valid credentials, the user portal will need information about the user. Getting all the messages of the user would take up too much network stuff. The user role would also be unnecessary.

UserDTO		
Attribute Name	Datatype	Description
UserID	Guid	The userID.
Username	String	The username.

ProfilePhoto	Byte[]	The profile photo.
DirectCommunicationProfiles	DirectCommunicationProfile[]	The directCommunicationProfiles of the user. These are just the profiles as transmitting all the messages in every the direct communications would be too expensive.
GroupProfiles	GroupProfile[]	The groupProfiles of the user. These are just the profiles as transmitting all the information (i.e. messages and all members) about all the groups would be too expensive.

Another datamodel is needed when adding or updating a user. This model is just the information required for a user i.e. their username password, role and profile photo. When retrieving the information about a user to update i.e. their username and userID, the password will be blank since all passwords are hashed before they are stored into the database. Similarly, when adding a user, the userID attribute should remain null since it will be generated by the server.

UserInformation		
Attribute Name	Datatype	Description
UserID	Guid?	The userID. Should be null when adding a user as it will be generated by the server.
Username	String (Max 100)	The username.
ProfilePhoto	Byte[]	The profile photo.
Password	String?	The password. Will be null when retrieving user information to update, as stated above.
Role	String	The role. Can only be "USER", "ADMIN" or "SUPERADMIN".

Group

Group		
Attribute Name	Datatype	Description
GroupID	Guid	Unique identifier.

GroupName	String	The group's name.
GroupIcon	Byte[]	The group's icon (profile photo for the group).
Members	User[]	A list of all the users that are members of the group.
Channels	Channel[]	A list of all the channels in the group.

GroupDTO		
Attribute Name	Datatype	Description
GroupID	Guid	
GroupName	String	
GroupIcon	Byte[]	
MemberProfiles	UserProfile[]	
ChannelProfiles	ChannelProfile[]	

GroupInformation		
Attribute Name	Datatype	Description
GroupID	Guid	
GroupName	String	
GroupIcon	Byte[]	
MemberProfiles	UserProfile[]	
ChannelProfiles	ChannelProfile[]	

GroupProfile		
Attribute Name	Datatype	Description
GroupID	Guid	
GroupName	String	
GroupIcon	Byte[]	
MemberProfiles	UserProfile[]	The userprofiles of the members of the group.
ChannelProfiles	ChannelProfile[]	

Channel

Channel		
Attribute Name	Datatype	Description
ChannelID	Guid	Unique identifier.
ChannelName	String	The name of the channel.
Group	Group	The group that contains the channel.
Messages	Message[]	The messages sent within the channel.

ChannelProfile		
Attribute Name	Datatype	Description
ChannelID	Guid	
ChannelName	String	
GroupID	Guid	The id of the group that contains the channel.

ChannelInformation		
Attribute Name	Datatype	Description
ChannelID	Guid	
ChannelName	String	
GroupID	Guid	Should not be changed when updating channel.

Message

This data model is not related to direct communications or channels, rather it is used by them. It is better to have a single class to represent messages than one for direct communications and another for channels. This is because it is easier to understand and maintain – if in the future the structure of messages were to change, the change would only have to be made in one place.

Message		
Attribute Name	Datatype	Description
MessageID	Guid	Unique identifier.
Content	String	The message content.
Sender	User	The user that sent the message.

LastUpdated	DateTime	The last time the message was updated. Is initially the time the message was created.
Attachment	Byte[]?	Attachment of the message, if any.

MessageDTO		
Attribute Name	Datatype	Description
MessageID	Guid	
Content	String	
SenderProfile	UserProfile	
LastUpdated	DateTime	
Attachment	Byte[]?	

ChannelMessageDTO		
Attribute Name	Datatype	Description
ChannelID	Guid	The id of the channel the message was sent in.
Message	MessageDTO	The messageDTO of the message.

DirectCommunicationMessageDTO		
Attribute Name	Datatype	Description
DirectCommunicationID	Guid	The id of the direct communication the message was sent in.
Message	MessageDTO	The messageDTO of the message.

NewDirectCommunicationMessage		
Attribute Name	Datatype	Description
DirectCommunicationID	Guid	The id of the direct communication the message is being sent in.
SenderID	Guid	The id of the user that is sending the message.

Content	String	
LastUpdated	DateTime	
Attachment	Byte[]?	

NewChannelMessage		
Attribute Name	Datatype	Description
ChannelID	Guid	The id of the channel the message is being sent in.
SenderID	Guid	The id of the user that is sending the message.
Content	String	
LastUpdated	DateTime	
Attachment	Byte[]?	

DirectCommunication

There can only be exactly 2 members of a direct communication. Despite this, they will be stored in a list instead of in 2 separate attributes member1 and member2. This is because it is easier to work with a list than 2 attributes.

DirectCommunication		
Attribute Name	Datatype	Description
DirectCommunicationID	Guid	Unique identifier.
Members	User[]	The users that are members of the direct communication. There can only be 2 members, no more no less.
Messages	Message[]	The messages sent within the direct communication.

Direct communications can be represented in a useful format by just their id and member profiles. The messages sent in a direct communication provide no information about the direct communication.

DirectCommunicationProfile		
Attribute Name	Datatype	Description
DirectCommunicationID	Guid	
MemberProfiles	UserProfile[]	

DirectCommunicationDTO		
Attribute Name	Datatype	Description
DirectCommunicationID	Guid	
MemberProfiles	UserProfile[]	The user profiles of the members in the direct communication.
Messages	MessageDTO[]	The messages in the direct communication as MessageDTOs

The only time it is easier to work with separate attributes over a list is when adding direct communications.

NewDirectCommunication		
Attribute Name	Datatype	Description
SenderID	Guid	Id of user creating the direct communication.
ReceiverID	Guid	Id of the other user in the direct communication.

Log

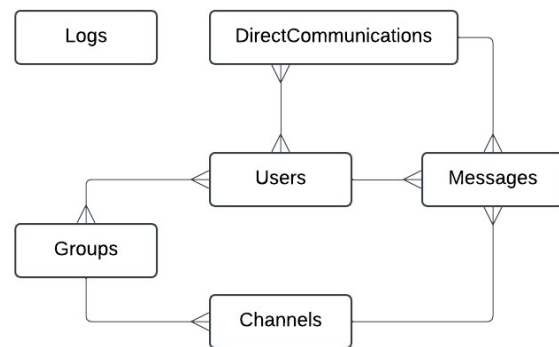
Log		
Attribute Name	Datatype	Description
LogID	Guid	Unique identifier.
LogMessage	String	The log message/content.
DateLogged	DateTime	The date the log was created/logged.

Entity Relationship Diagram (ERD)

Unresolved ERD

On the right is a diagram showing an overview of the relationships between the entities. The many-to-many relationships have not been resolved in this diagram.

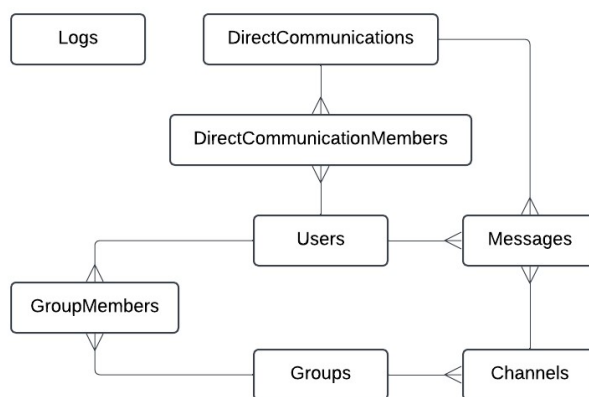
As shown, Logs has no relationship with any of the other entities and is standalone. It may also be designated its own database in the future since there could be a lot of them.



Resolved ERD

On the left is the resolved version of the abstraction above.

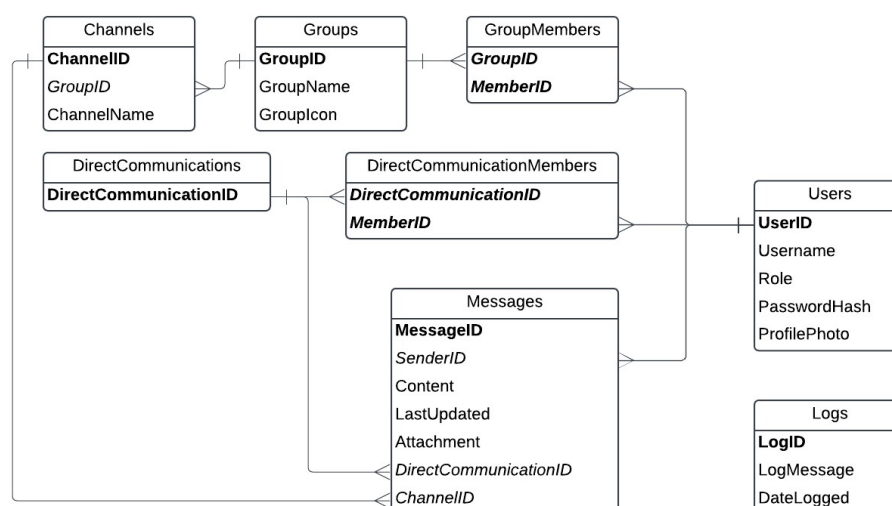
The tables DirectCommunicationsMembers and GroupMembers resolve the many to many relationships between groups, direct communications and their users.



Detailed ERD

Diagram

Below is shown the entities and their relationships and which fields are foreign keys etc.: Primary keys are shown in **BOLD** and foreign keys are shown in *ITALICS*.



Foreign Key Constraints

FK Constraint	Explanation
A message must have either a DirectCommunicationID or a ChannelID and cannot have both	A message cannot belong to both a group and a direct communication and a message can only be sent within a group or a direct communication.
A message must have SenderID, if the sender is deleted, the message should also be deleted	A message can only be sent by a user, if there is no user the message cannot exist.
Every record of DirectCommunicationMembers must have both MemberID and DirectCommunicationID	Both foreign keys make up a composite key, so both are needed.
Every record of GroupMembers must have both MemberID and GroupID	Both foreign keys make up a composite key, so both are needed.
Every channel must have a GroupID	A channel can only exist within a group.

Topic Handler Service

IMPLEMENTATION

The server, user portal and admin portal all need to execute different subroutines for different transmissions they receive. For example, the server may have one subroutine for authentication and another subroutine for adding a new user.

To do this, a dictionary will map topics to subroutines that “handle” them. This dictionary will contain different mappings for each application. The dictionary will be maintained and stored by a class called TopicHandlerService. It will also be populated with its data prior to runtime and no topic-handler pairs will be removed, modified or added during runtime – the dictionary will be fixed.

As such, the thing the class needs to implement is a way to get handlers by their topic.

GetTopicHandler

This will simply take a topic as a parameter, check if a mapping for it exists in the dictionary, if it does, it will return the handler otherwise it will return null.

TopicHandlerService
- topicToHandler : Dictionary<string, Func> {readonly}
+ GetTopicHandler(string topic) : Func

Class Diagram

The class TopicHandlerService is described in the diagram to the left. The dictionary is called topicToHandler and is private and read-only so it cannot be changed at all.

User Portal & Admin Portal Internal Services

PUBLISHER-SUBSCRIBER MODEL

When a request is sent to a server with a topic of “X”, the server will always respond with a transmission of topic “XResult”. This is how the user portal and admin portals can deduce when a transmission is a server response.

The user portal and admin portal both need a way to “subscribe” to responses from the server. This allows for a piece of code to wait to be executed until a response from the server has been received – like Angular’s publisher subscriber system.

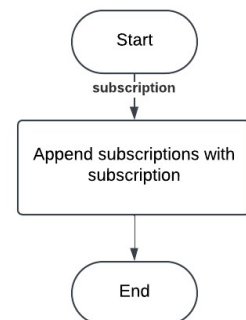
Subscribable objects will store the lambda function that need to be executed when receiving a response. As such, these Subscribable objects will be returned to requesters (the place where the request was made e.g. the login screen) when sending requests.

Subscribable

Subscribe

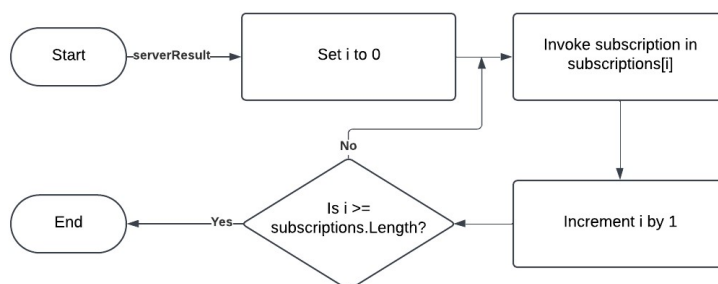
The Subscribable will allow the sender of the request to “subscribe” to the response. When subscribing, a lambda function will be given to the Subscribable which accepts a ServerResult as an argument (the server will always return a ServerResult object for every operation as discussed in the server section). These lambda functions will simply be stored in a list.

The flow chart on the right shows how this works.



Publish

When the app receives the server’s response, the object stored in the Dictionary against the topic of that response will be “published” which is when all the lambda functions, “subscriptions”, of the Subscribable will be executed.



The flow chart on the left shows how the publish subroutine works.

Summary

The diagram on the right describes the Subscriptable class.

Subscriptable
- subscriptions : List<Func<ServerResult>>>
+ Publish(ServerResult serverResult) + Subscribe(Func subscription)

Subscriptable Service

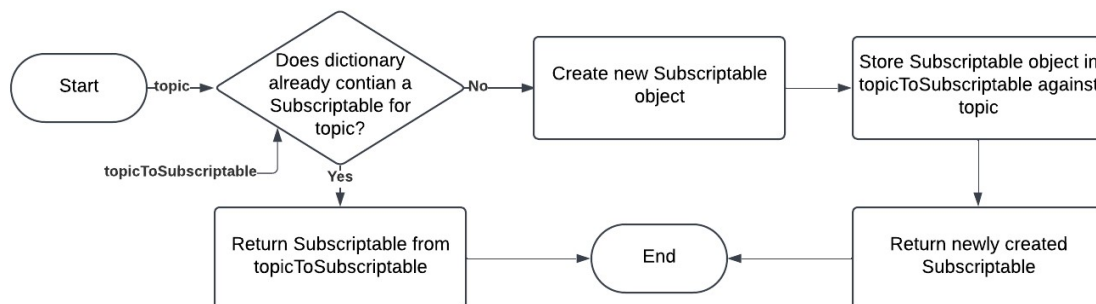
The subscriptable service will manage all the Subscriptables. It will do this by providing subroutines to “publish” Subscriptables and create or get Subscriptables for topics.

Get or Create Subscriptable By Topic

Every time a request is sent to the server, the subscriptable service will create or return a Subscriptable object. If the client is already waiting for a similar request (e.g. if a user sends a direct communication message and then another one very quickly) it should return the same Subscriptable created for that request.

All the Subscriptables will be stored in a dictionary mapping the topic of the server’s response (when the server sends the response) to the Subscriptable object. The dictionary will be called topicToSubscriptable. This is how a single Subscriptable can be reused for “similar” requests.

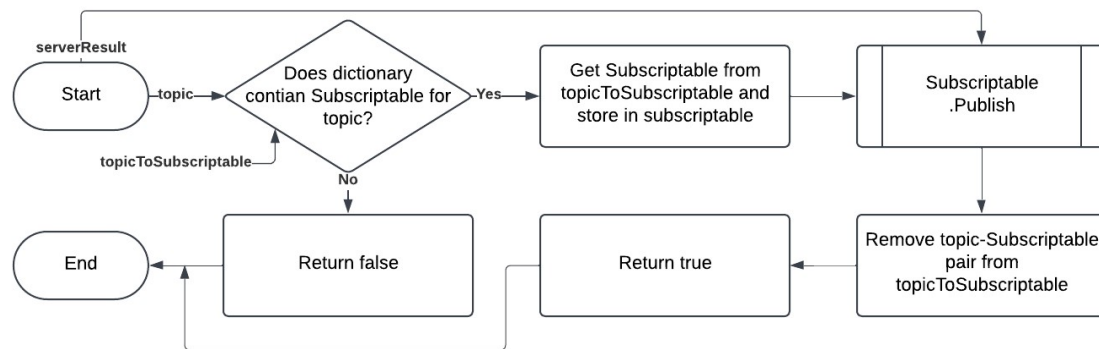
The flowchart below shows how this works:



Publish Topic

When publishing a topic, the subscriptable service simply needs to find the subscriptable of that topic, if there is one, then call its Publish method. After that, it removes it from the dictionary since, once a Subscriptable is published, it should be disposed (removed from the dictionary). The Subscriptable will need the ServerResult of the server’s response to pass into all its subscriptions so, the subscriptable service will also need the ServerResult of the server’s response.

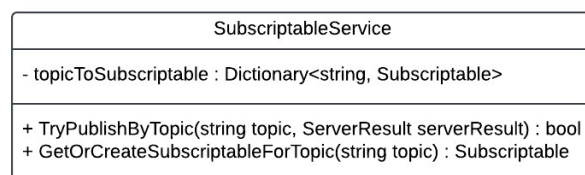
The flowchart below shows how this works.



As shown, if the topic given is recognised and is published, the subroutine will return true otherwise it will return false.

Summary

The class diagram on the right describes the SubscriptionService.



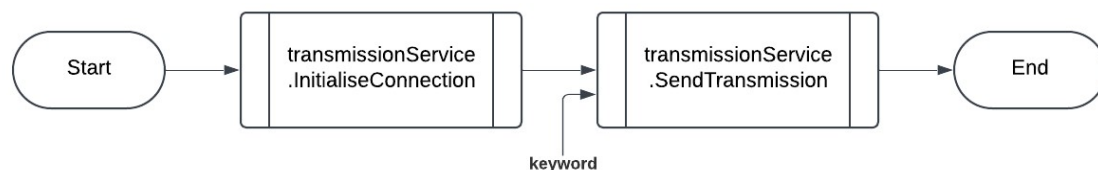
CLIENT SERVICE

Both the user portal and admin portal need a way to access their transmissionService object that handles their connection between the router and themselves. A class called ClientService will be used for this.

RunClient

This will initialise the connection between the router and the client and start listening for transmissions from the router.

The initialisation is shown in the flowchart below (starting to listen for transmissions is not shown):

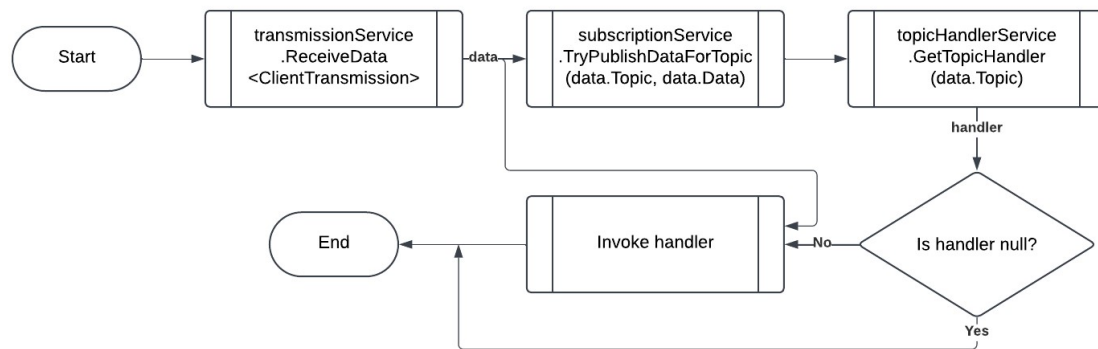


The keyword attribute mentioned in the flowchart will either be “CLIENT” or “ADMIN”. The relevance of this step is covered in the router section.

ReceiveData

This is how transmissions from the router will be handled. It will look at the transmission’s topics and see if there are any subscribers for that topic or if there are any topic handlers for it and will execute any that it finds.

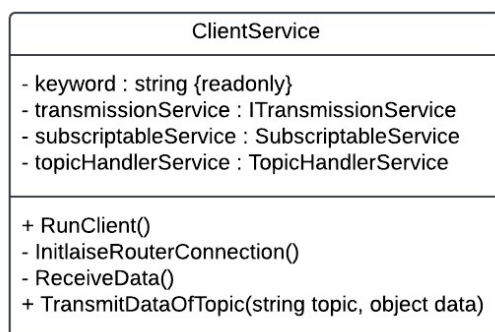
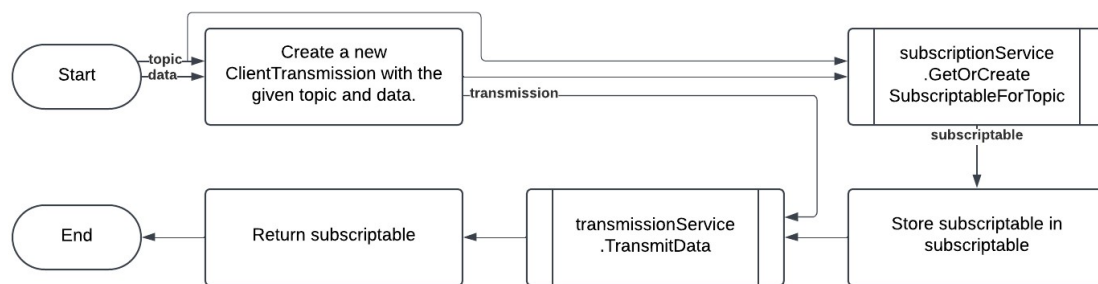
The flow chart for this is shown below:



TransmitDataOfTopic

This is how transmissions will be sent to the router. When sending transmissions to the router, the topic and data of the transmission need to be stored in a ClientTransmission object and a new Subscriptable has to be made/found for the request, then transmitted with the transmissionService. It is vital that the Subscriptable is made/found before the request is sent since (although highly unlikely) the response could be received before the Subscriptable has been made.

This is shown in the flowchart below.



Summary

The diagram on the left shows the client service.

The InitialiseRouterConnection and ReceiveData subroutines are both private.

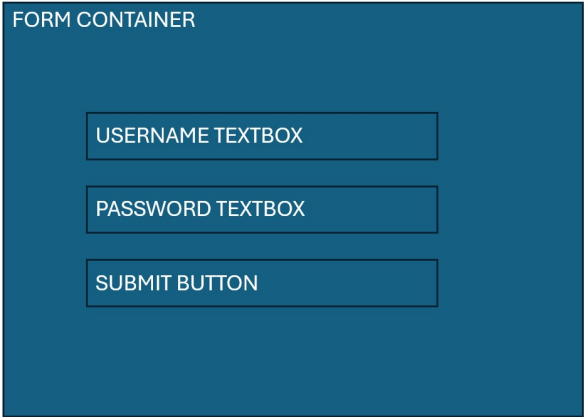
The InitialiseRouterConnection subroutine is what was shown in the flowchart up in the RunClient section.

User Portal

SCREENS, COMPONENTS AND POPUPS

Login Form

SOME IMAGE MAYBE



Above is shown the layout of the login form.

The submit button should be coloured in a special colour to make it clear where to press. The username and password textboxes should either have labels or placeholders to show where to enter what information. The password textbox will mask the user's input, internally the input will be stored but the user will only see a series of "*"s, this is to prevent shouldering attacks.

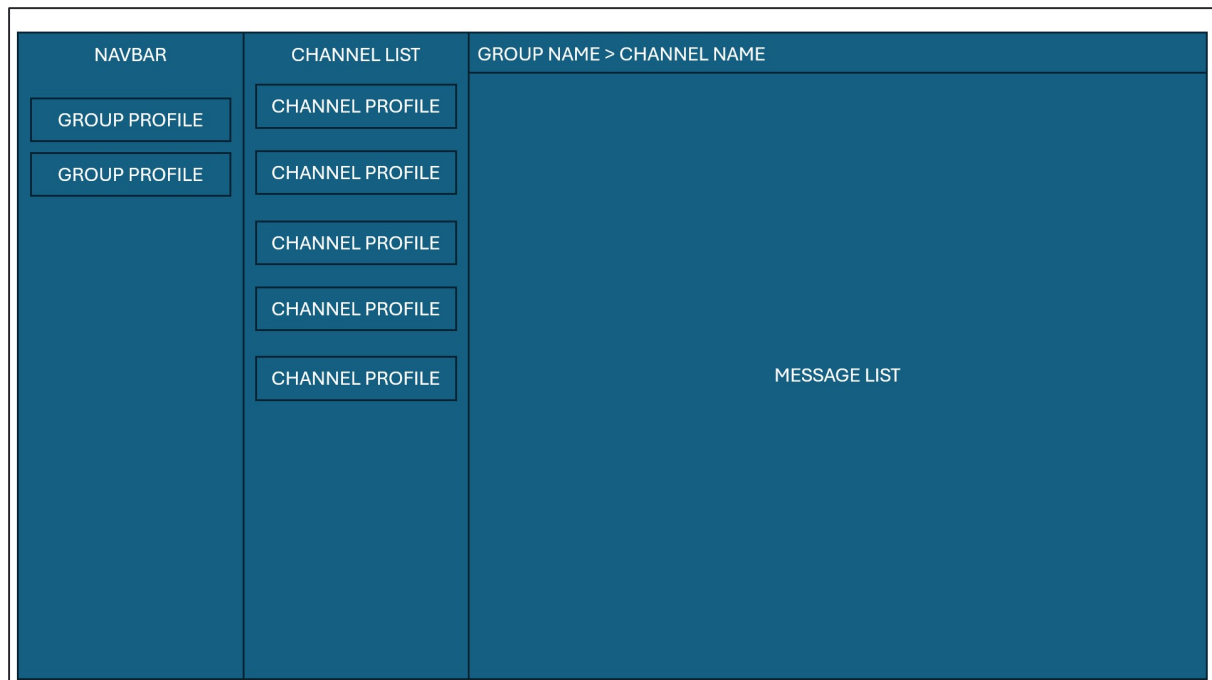
If the credentials the user enters are correct, they should be taken into the app otherwise an error message should be displayed at the top of the form saying, "Invalid credentials". Having a separate error message "Invalid password" when the username is valid and the password is not will be a security risk.

When clicking the Login Button, the username and password will populate a Credentials object. This will then be sent in a transmission to the server with the topic "AuthenticateUser".

In summary, the usability features of this screen are listed below:

- Login button coloured in a way that stands out to make clear where to press to login.
- Buttons and textboxes clearly labelled and big to make it clear where to type what information.
- Clear and big title to clarify what the user is doing.

Groups



Like Discord, all the groups the user is in will be displayed in the navbar. When a group is selected, all its channels will be displayed in the ChannelList section (just as a list of channel names) and when a channel is selected, the groupname and channel will be displayed on the top. This makes it easy for the user to determine which group and channel they are currently viewing. The messages of the channel will be displayed in the message list section.

When the user logs in successfully, the topmost group will be selected. Every time a group is selected, the topmost channel in the group will be selected by default.

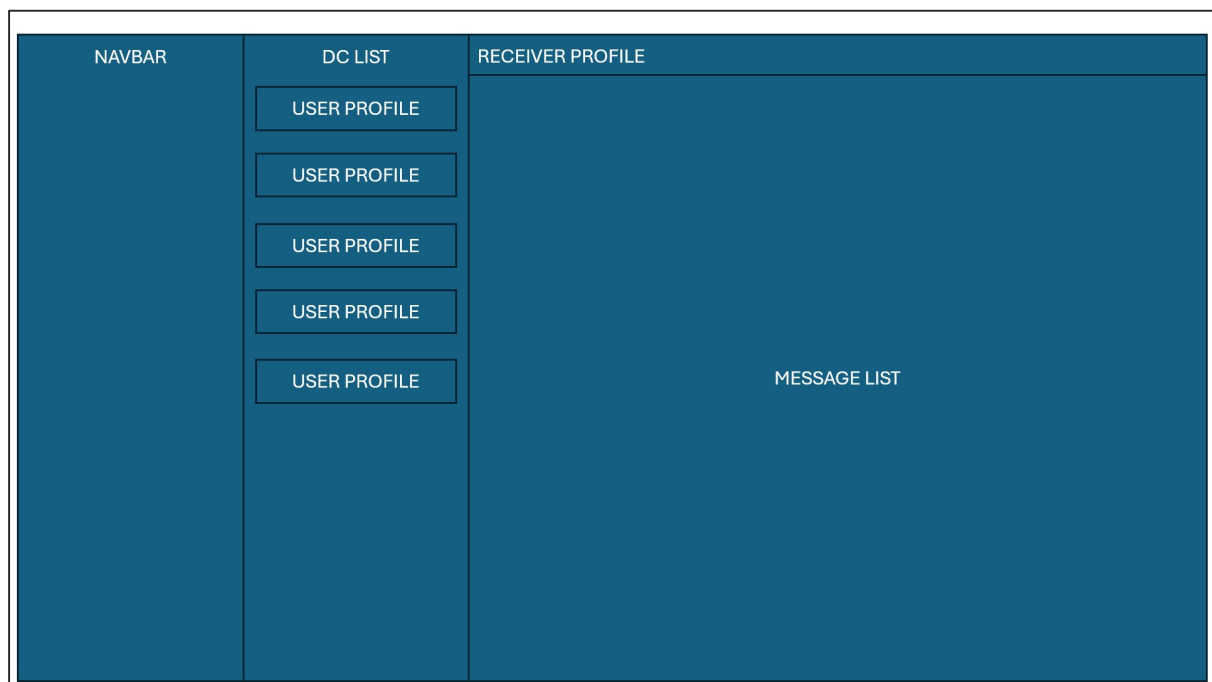
To populate the messages, a request of topic “GetMessagesByGroupId” containing the selected group’s id will be made to the server.

When a message is sent while in a channel, a request will be made to the server of topic “AddChannelMessage” and it will contain an object of NewChannelMessage. The ChannelID will be the currently selected channel’s id, the senderID will be the currently logged in user’s userID, the content will be what the user had entered in the textbox, LastUpdated will be the current date and time, and the Attachment will be set to whatever the user had set it to.

In summary, the usability features of this screen are listed below:

- Selected group will have a highlight in the groups list to clearly show which group the user is in
- Selected channel will have a highlight to clearly show which channel the user has selected
- The names of the selected channel and group will be displayed in the top bar to further emphasise the channel and group the user has selected

Direct Communications



Like Discord, the user will click on a button in the navbar to navigate to the direct communications page.

The DCList section will hold a list of all the user profiles of the other users in the direct communications (i.e. if Alice had direct communications with Bob and Charlie), she would see Bob's profile and Charlie's profile listed in the DCList section. When a direct communication is selected, the profile of the other user in the direct communication will be shown in the ReceiverProfile section to make it easy to identify which user the user is currently talking to. A user profile is simply the user profile photo paired with the username. The messages of the selected direct communication will be presented in the message section.

When first navigating to the direct communications page, the topmost direct communication will be selected by default.

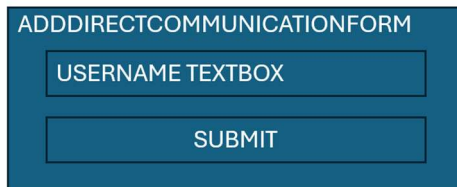
To populate the messages, a request of topic "GetMessagesByDirectCommunicationId" containing the selected direct communication's id will be made to the server.

When the send message button is clicked from within the direct communications list page, a NewDirectCommunicationMessage object will be created. The SenderID will be set to the currently logged in user's id. Content will be set to the content of the message text box. The DirectCommunicationID will be set to the currently selected direct communication's id. LastUpdated will be set to the current date and time. If the user added an attachment, it will be saved in Attachment. This will then be sent in a transmission to the server with the topic "AddDirectMessage".

The usability features of the above screen are summarised below:

- Clear title in the direct communications list to show the user they are in the direct communications list page.
- Selected direct communication will have a highlight to clarify which direct communication the user has selected.
- The profile of the other user in the selected direct communication will be displayed at the top bar to clearly show which user the user is talking to.
- The direct communications list will show the user profiles of the users on the receiving-end so the user does not have to memorise which direct communication is to which user.
- The add direct communication button will be clear and easy to see but will not have a special colour so as to not distract the user.

Add Direct Communication Form



The form on the left will be a popup shown when adding direct communications. This will only popup when pressing add direct communication from the direct communications page. To make it clear, there will be a label or placeholder text in the textbox “username”.

It would not be practical to list all the users in the app in a combobox, so they will be found by username. If the entered username is not recognised, an error message will be displayed “User does not exist”, if the entered username has already been added as a direct message another error message will be displayed “Direct communication with user already exists”.

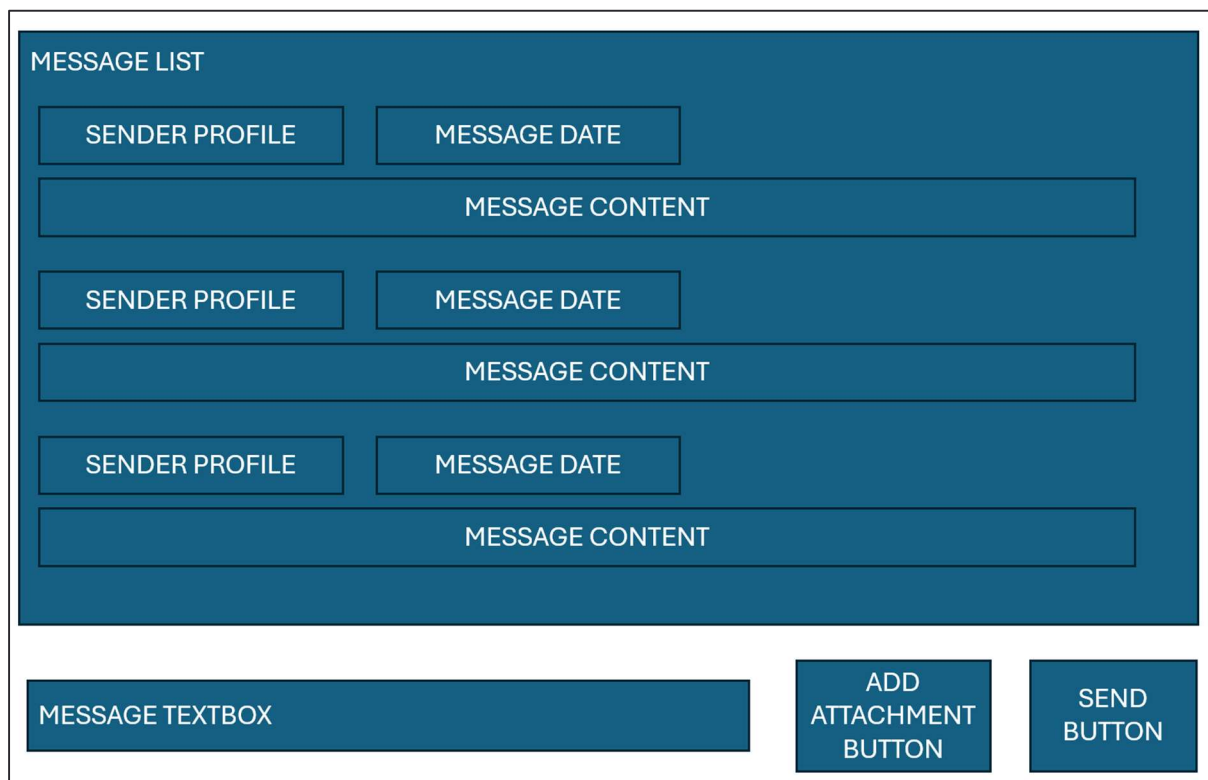
If the popup loses focus or escape is pressed, it will be closed. The popup will also close once a direct communication has been added.

When the submit button is pressed, the user portal will first send a request to the server to check if a user exists with the given username and it will have a topic of “GetUserByUsername”. The request will simply contain the entered username. If the user does exist the server should return its profile, another request will be sent to the server of topic “AddDirectCommunication” with the data being an object of NewDirectCommunication. The SenderID will be the currently logged in user’s ID and the ReceiverID will be the userID of the user whose username was entered in the textbox.

The usability features of the above popup are summarised below:

- Easy to close so the user can go back if they accidentally (or curiously) open it.
- Clear labels for add button and username textbox to clarify where the user needs to input.
- Clear title to clarify what the form is for.
- Coloured button to show the user where to press.
- Informative validation messages to help the user.

Messages



The message list component will be populated with a list of messages. The messages will be presented as follows: the SenderProfile section will display the profile of the user that sent the message, the MessageDate section will display the date the message was last updated and whether it was edited or not and the message content section will display the content of the message including any attachments.

When sending messages, the message textbox will be used to type the content of the message. If the user wants to add an attachment, they can click the add attachment button. To send the message the user can simply click the send button. The user will only be able to send a maximum of 1000 characters at a time to save storage, this will be handled by the message textbox. When the user types more than 1000 characters, an error message will be displayed but the user's text will not be removed automatically so that the user does not lose anything. A counter will start to show once the user types 200 characters. All of this should make it clear that the user can only type 1000 characters and should allow them to recover when they go over the limit. The send button will be disabled when they try to send a message over 1000 characters. Similarly, a maximum attachment of 3MB can be sent, once again for storage, so an error message will be displayed when trying to send an attachment larger than that.

Since the message list component will be reused by channels and direct communications, the action taken when the send button is pressed will be decided by what is using the component.

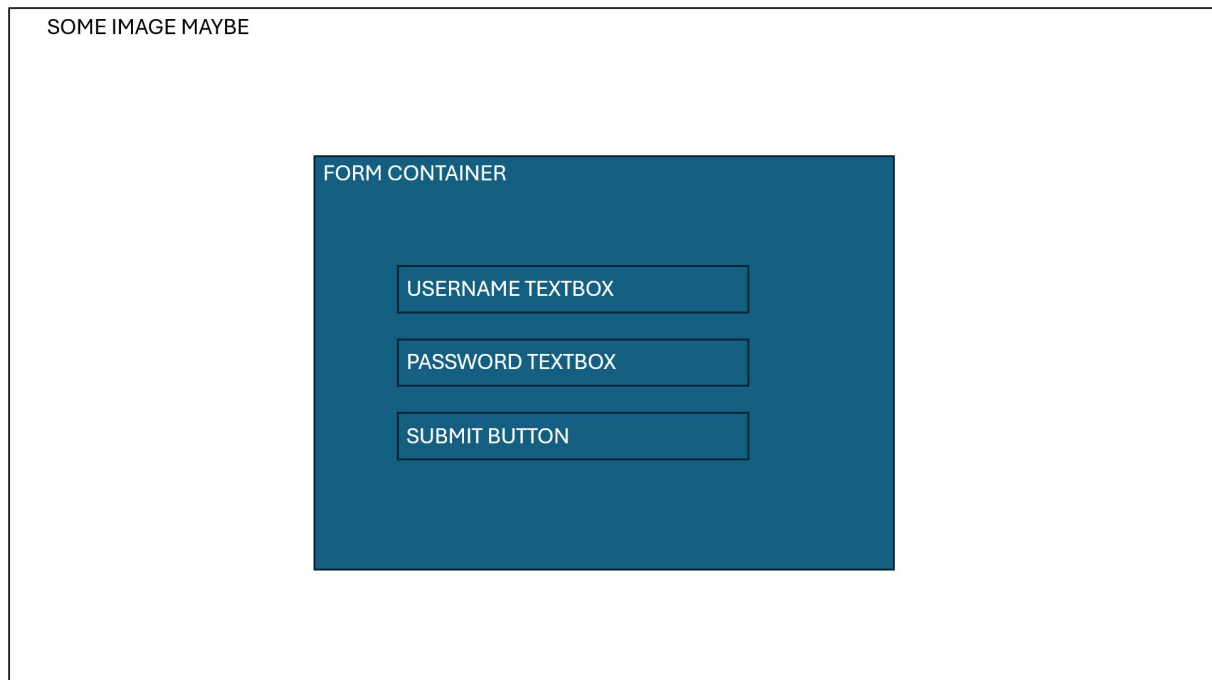
The usability features of the component above are summarised below:

- Sender profile on every message clearly depicts who sent which message.
- Message data shows when each message was sent.
- Chronological order makes it easy to navigate messages.
- Message textbox will have a placeholder to show the user where to type their message.
- Message textbox increases in size up to a point when sending multiline messages so the user can more easily write long messages.
- Clear add attachment and send message button to clarify where the user can press to do what.
- Character counter, validation message and disabled button to help the user not exceed the character limit
- Keeping the user's extra characters to ensure they do not lose anything they write by accident
- Same message component reused between channels and direct communication so the user can become familiar with the structure of the messages.

Admin Portal

USER INTERFACE DESIGN

Login Form



The login form for the admin portal will essentially be a copy of the login form of the user portal since their functionalities are so similar. To make it easier to distinguish between them, there will be a different background image and title.

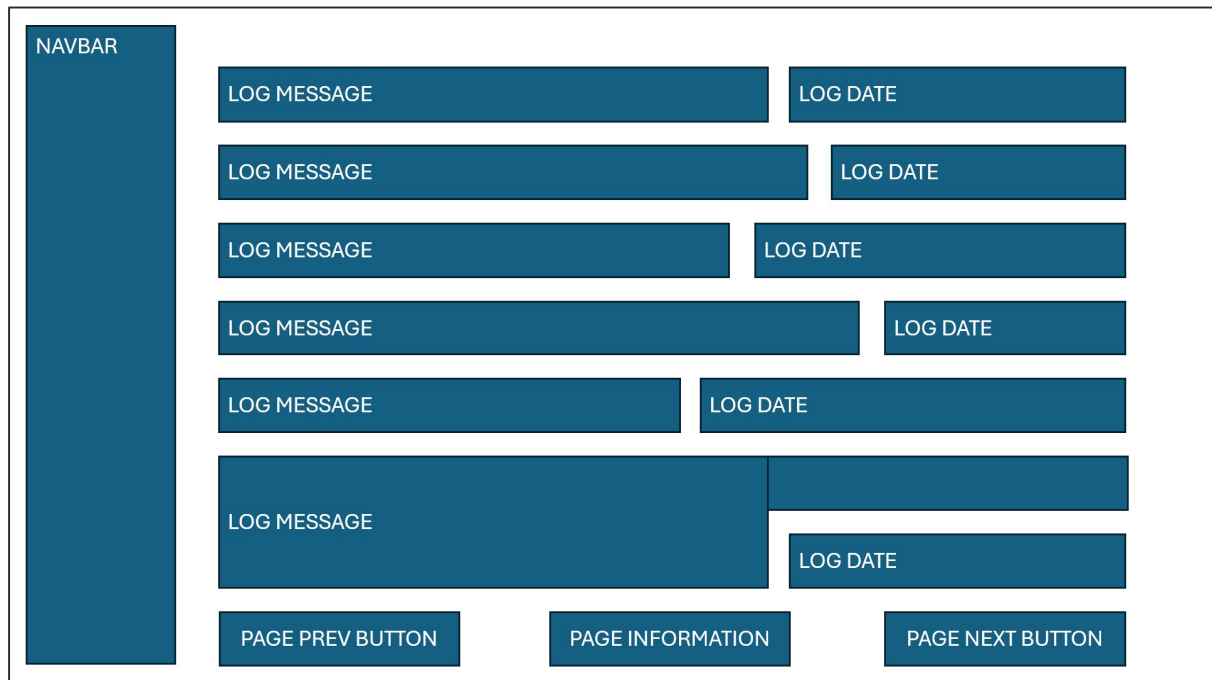
The validations will also be the same as that of the user portal. However, if the user credentials match a user but the user is not an admin, the error message “Invalid credentials” will be displayed. This is so a brute force attack does not yield any user’s credentials whatsoever.

The usability features for this screen (not including previously stated ones in User Portal Login Form) are summarised below:

- Different title to help differentiate between admin portal and user portal

Logs

Log List



The logs will be displayed in a list, the log message and log date will simply be appended next to each other as a single string. This is because not much structuring is required for logs.

Paging

It is more beneficial to have paging in the logs rather than a scrolling-only system since there will be so many. The user can also choose which page to go to by changing the page information text box. The user portal will internally store the page number, the page next button will increment the page by 1 and the page prev button will decrement the page by 1. If the page is 0 then the page prev button will be disabled and the page next button will be disabled when on the last page. Whenever the page number is changed, a request will be made to the server of topic "GetLogPage". When initially the log page is loaded, the page will be set to 0 and the request "GetLogPage" will be made to the server.

Users

User List

NAVBAR	TITLE		ADD USER BUTTON
	USERNAME	EDIT USER BUTTON	DELETE USER BUTTON
	USERNAME	EDIT USER BUTTON	DELETE USER BUTTON
	USERNAME	EDIT USER BUTTON	DELETE USER BUTTON
	USERNAME	EDIT USER BUTTON	DELETE USER BUTTON
	USERNAME	EDIT USER BUTTON	DELETE USER BUTTON
	USERNAME	EDIT USER BUTTON	DELETE USER BUTTON
	USERNAME	EDIT USER BUTTON	DELETE USER BUTTON

The title section will simply display the title Users. The delete user button will delete a user by sending a request to the server of topic “DeleteUserById”. The request will contain the user’s id. The users will be structured in a table, the username should be enough to discern which user is which since all usernames will be unique. The table headers will be User, Edit and Delete going from left to right. If there are a lot of users that take up the whole screen, the admin will be able to scroll down and up within the table.

To populate the table, a request will be made to the server of topic “GetAllUsers”.

The diagram shows a blue rectangular container labeled "USER FORM CONTAINER". Inside the container, there are four stacked rectangular boxes: "USERNAME TEXTBOX", "PASSWORD TEXTBOX", "ROLE DROPDOWN", and "PROFILE PHOTO IMAGE UPLOAD". At the bottom of the container is a wide rectangular box labeled "SUBMIT BUTTON".

User Form

The user form will popup when one of the Edit User or Add User buttons are pressed.

The form will take up the whole page and can only be closed by pressing the close button (an x on the top right of the popup) or submit button.

As shown, roles will be a dropdown since there are a fixed set of roles a user can take, so this is the most usable way of assigning a user's role.

When the edit user button is pressed, the form will be populated with all the user's information except for the user's password, which can be left blank to remain unchanged. To make this obvious, it will be stated above the password textbox, the admins should be trusted enough to not misuse or misunderstand this functionality. If the edit user button opened the form, the submit button will send a request of topic "UpdateUser" to the server. If the add user button opened the form, the submit button will send a request of topic "AddUser" to the server.

User passwords will have to be at least 8 characters long, contain a symbol and a capital letter and a lowercase letter. If any of these requirements are not met, an error message will be displayed detailing the unmet requirement. All validations will be displayed at the top of the form.

Groups

Group List

NAVBAR	TITLE		ADD GROUP BUTTON
	GROUPNAME	EDIT GROUP BUTTON	DELETE GROUP BUTTON
	GROUPNAME	EDIT GROUP BUTTON	DELETE GROUP BUTTON
	GROUPNAME	EDIT GROUP BUTTON	DELETE GROUP BUTTON
	GROUPNAME	EDIT GROUP BUTTON	DELETE GROUP BUTTON
	GROUPNAME	EDIT GROUP BUTTON	DELETE GROUP BUTTON
	GROUPNAME	EDIT GROUP BUTTON	DELETE GROUP BUTTON
	GROUPNAME	EDIT GROUP BUTTON	DELETE GROUP BUTTON

This page will be similar to the user list page due to the nature of its content.

To populate the table, a request will be made to the server of topic "GetAllGroups". When the delete group button is pressed, the selected group will be deleted via a request to the server of topic "DeleteGroupByGroupId" which will contain the id of the selected group.

FORM CONTAINER

GROUPNAME TEXTBOX

GROUP PHOTO IMAGE UPLOAD

CHANNEL NAME TEXTBOX ADD BUTTON

CHANNEL LIST

CHANNEL NAME REMOVE BUTTON

MEMBER USERNAME TEXTBOX ADD BUTTON

MEMBERS LIST

MEMBER USERNAME REMOVE BUTTON

SUBMIT BUTTON

Group Form

This form will open as a full screen popup when the add group button is pressed.

Like the user form, it can only be closed by pressing the submit form button or the close form button on the top right of the popup.

The channel list and members list will be scrollable. To add a channel, its name will be typed in the channel name textbox and the add button will be pressed. Similarly, to add a member, the user's username will be typed into the member username textbox and the add button will be pressed.

Every time a username is added (as with the add direct communication form) a request will be made to the server of topic "GetUserProfileByUsername" to ensure a user exists with the given username.

When the submit button is pressed a request to the server will be made of topic "AddGroup".

Edit Group & Channels

GROUPNAME TEXTBOX	
GROUP PHOTO IMAGE UPLOAD	
CHANNEL NAME TEXTBOX	ADD BUTTON
CHANNEL LIST	
CHANNEL NAME	DELETE BUTTON
CHANNEL NAME	DELETE BUTTON
MEMBER USERNAME TEXTBOX	ADD BUTTON
MEMBERS LIST	
MEMBER USERNAME	REMOVE BUTTON
SUBMIT BUTTON	

This form is bigger than the other form but might still be in a popup. After all the changes are made to the group, a request will be made to the server of topic "UpdateGroupAndChannels". The server will deduce which channels it needs to add, delete and update. The channel names can be changed as each element in the channel list has a textbox populated with the channel name, which can be updated.

Server and Router Console Justification

The server and router will simply be console programs that can be run with single commands. This makes it easy to set them up and, as they do not waste resources rendering GUIs, they will be faster. This was also requested by the IT Technician guy.

Server

RESPONSIBILITIES

The server will carry out the following tasks:

- **Perform operations on the database** – All data (i.e. messages, users, groups etc.) will be stored in a database which will only be accessible through the server.

- **Authorisation** – Whenever a user makes a request, the server will check if the user has the rights to do that. For example, users should not be able to send messages in a group they are not a part of.

SERVER TRANSMISSIONS

Transmission Topics

Requests & Responses

The table below shows all the possible topics of requests that the user portal or admin portal will make. The table includes what type of data will be sent in the request and what type of data will be sent in the response along with a description of the request.

Topic	Request Datatype	Response Datatype	Description

Publishes

The table below shows all the possible topics of publishes the server will produce including what data the publishes will contain along with a description of the publish.

Topic	Datatype	Description

Transmission Types

ServerResult

When the server is responding to a client request, it will return a ServerResult object. More information about this object is given in the table below.

ServerResult		
Attribute	Datatype	Description
OperationInformation	OpertionInformation	Object that contains information about the operation performed – whether it was successful and any messages the server needs to relay.
Data	Object?	Contains the data of the response i.e. a request for a list of groups would be responded with a list of groups

OperationInformation		
Attribute	Datatype	Description
Status	String	If the operation was successfully carried out "SUCCESS" otherwise "FAILURE".
MessageCode	String	A brief message that can only be a certain set of values. This will allow for receivers to quickly analyse the result of the operation without having to know the exact message e.g. "AUTHENTICATION SUCCESSFUL" when the entered credentials were valid.
Message	String	A detailed message which could be displayed to the user by the requester.

ServerResponse

When the server is responding to a request, it needs to specify the requestID from the router (discussed in the router section) along with the response's topic and the data of the response. This is represented as a class described below.

ServerResponse		
Attribute	Datatype	Description
ServerResult	ServerResult	The result of the operation requested.
Topic	String	The topic.
RequestID	Guid?	The id of the request the server is responding to.

ServerPublish

When the server needs to send updates to clients e.g. a new direct communication message, the server will send a ServerPublish object. More detail about the object is given in the table below.

ServerPublish		
Attribute	Datatype	Description
Topic	String	The topic.

Data	Object?	The data that needs to be updated or followed.
AffectedUsers	Guid[]	The users that need to be updated.

ServerTransmission

The server may need to send multiple ServerPublishes and ServerResponses at one time e.g. when adding a new channel message, a response needs to be sent to the client that sent the message and a publish needs to be sent to all the users in the group with the information of the new message. To do this in a single transmission – without having to send multiple transmissions to the router, an object will be used. The class of the object is detailed in the table below.

ServerTransmission		
Attribute	Datatype	Description
Publishes	ServerPublish[]?	The publishes, if any.
Responses	ServerResponse[]?	The responses, if any.

Summary

In summary, the server will always send objects of ServerTransmission to the router.

Router

RESPONSIBILITIES

The router is responsible for ensuring that transmissions reach the correct recipients i.e. transmissions meant for Alice should not be sent to Bob.

This routing of transmissions can be split into 3 main categories:

- **Publish Forwarding** – Publishes from the server are sent to only the desired recipients (and no one else). For example, a new group message transmission should only be sent to the members of the group.
- **Request Forwarding** – Requests sent by the clients should be forwarded to only the server; they should not be sent to any other client.
- **Response Forwarding** – Responses to requests should only be sent to the client that requested it; other clients should not receive those responses.

The router also needs to be able to handle multiple requests and connections at the same time. According to the research done in the analysis section, a good way to do this is with multithreading. If one thread is dedicated to receiving connections and a new thread is created for every connection, the router will be able to effectively handle new connections and client transmissions

at the same time. However, creating a lot of threads can be very expensive on the CPU, so some form of thread-pooling may need to be used to reduce the load on the CPU.

TRANSMISSION TYPES

ClientTransmission

The transmissions sent to and from the 2 portals will be objects of the ClientTransmission class.

ClientTransmission		
Attribute	Datatype	Description
Topic	String	The topic.
Data	Object	The data.

ServerRequest

The transmissions that will be sent from the router to the server will be ServerRequest objects.

ServerRequest		
Attribute	Datatype	Description
Topic	String	The topic.
Data	Object	The data.
RequestID	Guid	Generated Id of the request, explained later.
RequesterID	Guid?	Id of the user that made the request. This might be null since the user may not have signed in yet or the user could be from the admin portal.
IsAdmin	bool	Was the request made from an admin portal.

MANAGING AND GATHERING CLIENT INFORMATION

To send transmissions to the correct clients, the router needs to gather information about the clients that connect to it. The router cannot send messages to Alice when it does not know which client Alice is signed into. Information the router needs to know about clients is listed below:

- Whether the client is a server, user portal or admin portal
- If the client is a user portal, what user is signed into it
- Which client sent a certain request
- The client of the request the server is responding to

Storing Client Information

The router needs a structure to manage client connections. A **TransmissionService** instance handles communications between 2 clients, which for the router will be itself and the connecting client. So, a new instance of the **TransmissionService** will be created for every new connection.

Below, the storage of each type of client connection to the router is discussed:

Server – There will only be a single server connected to the router at any time. Hence, the router will simply store the server's connection in a dedicated variable. As it only needs the connection to send and receive transmissions, the variable can simply be the **TransmissionService** instance created for the server connection.

Admin portal – As with the server, no extra information about an admin portal is required, so only their **TransmissionService** instance needs to be stored. Multiple admin portals can be connected at the same time, so a list of **TransmissionService** instances will be created and maintained to store admin portal connections.

User portal – The router will need to track signed-in users by storing their user ID and connection together. This will be done using a dictionary and a custom structure. The dictionary will allow for fast retrieval of connections by user IDs and the custom structure will store information about the client.

User portal connections will only be stored once a user has signed in.

The custom structure used to store user portal information is called **LifeSupport** and is shown in the diagram. A detailed explanation of its attributes and methods is given in the tables below.

LifeSupport
- isLoggedIn : bool = false - userID : Guid? = null - transmissionService : ITransmissionService {readonly}
+ TransmitData(object data, bool isHighRisk) + ReceiveData<T>() : T + InitialiseConnection() + GetInitialised() : bool + GetUserID() : Guid? + GetIsLoggedIn() : bool + Login(Guid userID)

Attributes	
Attribute	Description
userID	The id of the user signed into the portal. When initially connected, there may not be a user signed in, so this will be null by default.
transmissionService	The instance of TransmissionService for this connection
isLoggedIn	States whether the user portal has been logged into.

Methods	
Method	Description
TransmitData	Exposes transmissionService.TransmitData
ReceiveData	Exposes transmissionService.ReceiveData
InitialiseConnection	Exposes transmissionService.InitialiseConnection
GetInitialised	Exposes transmissionService.Initialised
GetIsLoggedIn	Returns value of isLoggedIn
Login	Sets userID to the given ID and sets isLoggedIn to true.
GetUserId	Returns value of userID

Overall, the router will need to maintain 3 variables to store information about its clients:

- serverConnection : ITransmissionService – stores the connection to the server.
- adminConnections : List<ITransmissionService> – stores the connections to the admin portals.
- userConnections : Dictionary<Guid, LifeSupport> – stores a mapping between user portal connections and the ID of the user signed into it. The userID will be the key and the LifeSupport instance will be the value.

In addition to these, the router will need another variable to maintain its request system (explained in a section below). The variable will be called requests and have the same datatype as userConnections.

Differentiating User Portals, Admin Portals and The Server

Before the router gathers any information about the user signed into the client, it needs to differentiate between which clients are user portals, admin portals and which client is the server.

If the router tries to discern the client type of a client by itself, with no input from the client itself, the only reliable method would be to map every single IP address of computers on the network to a client type. This requires having set IP addresses for every computer on the network and can complicate the process of adding new computers to the network. In addition, it will make it impossible for admins to sign in on any device on the network, reducing the accessibility of the app.

Hence, to identify the type of client, the clients must inform the router of what type of client they are by themselves. This can be done by sending a simple string that corresponds to the client type: "USER" for user portals, "ADMIN" for admin portals, and "SERVER" for the server.

However, this method is vulnerable to spoofing. For example, an attacker could connect to the router and claim to be an admin portal by sending the string "ADMIN".

Since all requests from the user and admin portals must be directed to the server, the server can handle authorisation. Each request should include the user ID of the signed-in user, allowing the server to verify whether the user is authorized to perform the requested operation. The user ID is highly unlikely to be exposed, as all transmissions are encrypted. As such, the only client type that should not be 'spoofable' at all is the server. Additionally, since the server will handle authorisation, the router itself does not need additional security measures for client identification.

Since there will only ever be one instance of the server running, the router should only accept a client as the server if no server is currently connected. In the case of multiple servers in the future, it might be beneficial to map specific IPs to each server and have the router connect directly to them, rather than relying on clients to self-identify however, this is beyond the scope of the current project.

In addition, for extra security the number of admin portals connected at one time could be limited.

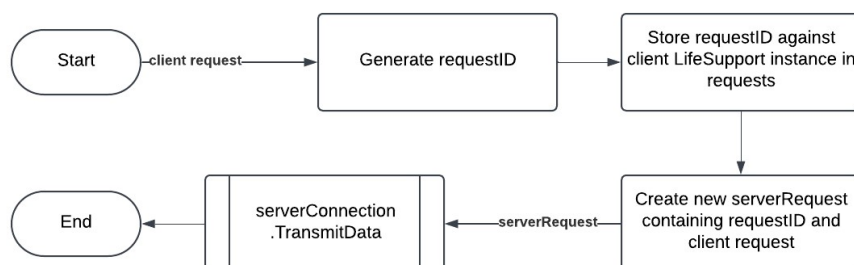
In summary:

- Clients will send a transmission to the router upon connecting, stating what client type they are: "USER" for user portal, "ADMIN" for admin portal, and "SERVER" for the server
- The router will only allow a single server to be connected if a client sends "SERVER" when connecting

Request-Response handling

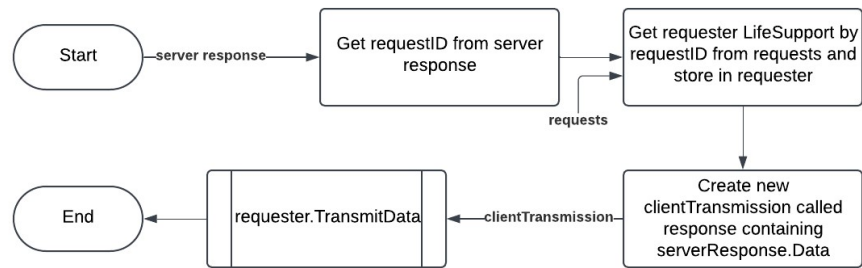
When a user portal connects to the router initially, no information about it is known about the user signing into it. However, the user portal will still need to send requests to the server and receive responses to them. Requests and responses may not always happen in a predictable order, so the router needs to identify which transmissions are responding to which requests.

This can be done by assigning a new unique request id to every request sent to the router from clients and storing it against the client connection – which is best done with a dictionary for quick retrieval. When the server sends a response to a given request, it can attach the request id to the transmission that can be used by the router to deduce the initial requester i.e. the user portal that sent the request.



The flow chart on the left shows the algorithm for forwarding client requests to the server.

This flow chart shows the algorithm for forwarding server responses to the requester.



Getting userID of user signed into User Portal

The router needs to get the userID of the user signed into a user portal. When Bob signs into a user portal correctly, the server will respond with a successful authentication result that will contain Bob's userID. The router can listen for this and add Bob's connection to userConnections with the userID from the server.

This approach removes the need for the user portal to send its userID to the router itself, thus preventing spoofing. In summary, the router should listen for transmissions from the server containing successful authentication results for the user portals, it should then use the userID in those transmissions to store the user portal connection in userConnections.

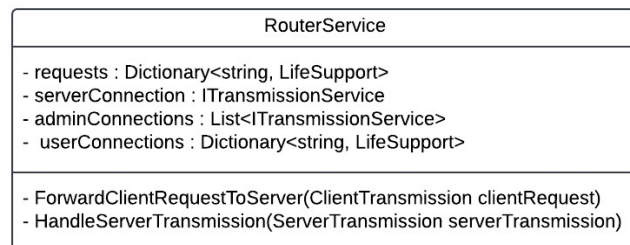
The algorithm for this is shown in the flow chart below: -----
aaaaaaaaaaaa

ROUTER SERVICE

To store all client information along with provide functionalities to start the router, a class will be used called RouterService.

The class is shown in the diagram to the right and is similar to the client service.

A detailed summary of the methods in the class is given in the table below.



Method	Description
ForwardClientRequestToServer	Takes client transmissons, converts them to ServerRequest objects, generates a new requestID and sends the new server request to the server.
HandleServerTransmission	Takes a ServerTransmission, sends all the responses to their requesters based on their requestID, and sends all the publishes to their signed in affected users.

Iterative Development & Testing Plan

TEST STRATEGY

The application will be made using Test Driven Development. This means that every subroutine will be tested and functional by the time it is developed. This is analogous to bottom-up testing.

The user portals and admin portals will have GUIs that need to be tested. This will simply be a case of running the program and ensuring it looks like how its supposed to.

Since the solution will be developed iteratively, the completed work should be tested at the end of each sprint.

Below is shown a table detailing features and the test cases for those features. These features have been deduced by the success criteria.

Feature	Test Case	Expected Outcome
User & admin portal sign in.	Valid credentials	Authentication success
	Invalid credentials	Authentication fail
	Valid username, invalid password	Authentication fail
	Invalid username, valid password	Authentication fail
	For admin portal only: valid credentials of non-admin user	Authentication fail with either "Invalid credentials" or "Unauthorised user"

SPRINTS

The solution will be created in sprints. In each sprint, a small number of features will be developed, and the changes made during that sprint will be tested and evaluated before moving onto the next sprint. The specificities of which features will be developed when will be decided in the development stage.

CHAPTER 3 DEVELOPMENT & TESTING

When developing the code, I have tried my best to make it read ‘like english’ without needing comments to explain things. To aid with future maintenance of the solution, every function and class has been summarised with the <summary> tags in C#.

Sprint 1

CODING

I started off with 3 applications: a client, a router and a sever. The router and server have been heavily discussed in the design. The client is going to be the base for the user portal and admin portal, since they both send and receive transmissions in quite similar formats and are both going to be used by people.

First, I established some basic communication between the applications. For now, all that simply happens is, the user enters a username which is sent to the server via the router. The server then creates a new user with that username and generates the user’s userID before storing it in the database. The userID is then returned to the router where it gets stored against the Socket handler for that user. No messages can be sent yet but the foundations have been laid out. For now, the database is in memory and is implemented with dictionaries.

This is the code for the client:

```
5 Console.WriteLine("CLIENT");
6
7 var hostName = Dns.GetHostName();
8 Console.WriteLine($"On host: {hostName}");
9
10 Console.WriteLine("Broker IP address: ");
11 IPAddress brokerIPAddress = IPAddress.Parse(Console.ReadLine());
12
13 Console.WriteLine("Broker port: ");
14 int brokerPort = Convert.ToInt32(Console.ReadLine());
15
16 var brokerEndPoint = new IPEndPoint(brokerIPAddress, brokerPort);
17
18 using (Socket client = new Socket(brokerEndPoint.AddressFamily, SocketType.Stream, ProtocolType.Tcp))
19 {
20     await client.ConnectAsync(brokerEndPoint);
21
22     await client.SendAsync(Encoding.UTF8.GetBytes("CLIENT"), SocketFlags.None);
23
24     Console.WriteLine("Client connected");
25     Console.WriteLine("Username: ");
26
27     var username = Console.ReadLine();
28
29     var usernameBytes = Encoding.UTF8.GetBytes(username);
30
31     await client.SendAsync(usernameBytes);
32 }
33
34 Console.ReadLine();
```

This connects to the router by getting the user to enter the router's IP address and port and binding a new socket to that endpoint. It then tells the router it's a client by sending the string "CLIENT". Finally, it asks the user for a username and sends it to the router.

This is the initial code for the router:

```
4 Console.WriteLine("IP address: ");
5 IPAddress ipAddress = IPAddress.Parse(Console.ReadLine());
6
7 Console.WriteLine("Port: ");
8 int port = Convert.ToInt32(Console.ReadLine());
9
10
11 var routerService = new RouterService(new IPEndPoint(ipAddress, port));
12
13 Console.WriteLine("Running broker...");
14 routerService.Run();
```

It asks for the IP address to connect to and then parses the given string into an IP address object and also asks for the port number to connect to and parses that into an integer. It then combines these into an IPEndPoint object and passes it into the router service. The RouterService is where the main logic of handling transmissions will occur.

When running the router the following happens:

```
30 public async Task Run()
31 {
32     Console.WriteLine("Broker running...");
33     var listener = new Socket(_iPEndPoint.AddressFamily, SocketType.Stream, ProtocolType.Tcp);
34     listener.Bind(_iPEndPoint);
35
36     listener.Listen(100);
37
38     Task.Run(() =>
39     {
40         while (true)
41         {
42             var handler = listener.Accept();
43
44             if (!handler.Connected)
45             {
46                 Console.WriteLine("Connection attempt failed...");
47                 continue;
48             }
49
50             Console.WriteLine("Connection attempt successful, new client connected...");
51
52             NewConnection(handler);
53         }
54     });
55
56     Console.ReadLine();
57 }
```

First a new stream socket is created (line 33) and is bound to the IP endpoint passed into the constructor (line 34). This socket is called the listener as it will act as the connection listener, anytime a client (the client and the server) wants to connect to the router, they do it through the listener. On line 36, the listener is told to listen for up-to 100 incoming connections.

When a client connects to the listener, the listener has to accept the connection which returns another socket as shown in line 42. This new socket is stored in handler and will be used to receive and transmit data to that client. The handling of a new connection (i.e. the gathering of initial client information) is handled in the NewConnection subroutine.

The infinite while loop running from lines 40 to 54 will run until the app is closed, this is so the router is always able to accept new connections. The while loop is running in its own thread (via the Task.Run subroutine) so the application can perform concurrently – it can handle connections while transmitting data.

The NewConnection subroutine is shown below. It listens for the first transmission from the client, as it is expected to be the initial string identifier: “CLIENT” or “SERVER”.

```
59  public async Task NewConnection(Socket handler)
60  {
61      Console.WriteLine("Handling new connection...");
62
63      string message = await ReceiveTransmission(handler);
64
65      switch (message)
66      {
67          case "SERVER":
68              ServerConnection(handler);
69              break;
70          default:
71              NewClientConnection(handler);
72              break;
73      }
74      Console.WriteLine("Connection handled...");
75  }
```

If the client sends “SERVER”, it is taken to be a sever and is then handled by the ServerConnection subroutine. If the client sends anything else, it is handled by the NewClientConnection subroutine and is assumed to be a client. I used a switch case statement because its faster and cleaner than a series of if else blocks.

I separated the handling of server connections and client connections into their own subroutines to make the code clearer.

The ServerConnection subroutine is shown below. It stores the handler for the server in the `_server` attribute of the RouterService. This is used later to send requests to the server.

```

96  public async Task ServerConnection(Socket serverHandler)
97  {
98      _server = serverHandler;
99      Console.WriteLine("Server connected...");
100
101      while (true)
102      {
103          string message = await ReceiveTransmission(_server);
104          string[] processResult = message.Split(" ");
105
106          _handlerToProcess[processResult[0]].Invoke(processResult[1]);
107      }
108  }

```

Another infinite loop is used to keep listening for transmissions from the server, however, since the entire subroutine is marked as 'async' the subroutine can run asynchronously so the while true loop does not need to be explicitly isolated in its own thread.

The server is expected to send messages in the form "{header} {payload}". The header identifies how the transmission should be handled, and the payload carries the data of the transmission. As shown, the dictionary `_handlerToProcess` will act as a simple version of the TopicHandlerService defined in the design.

The NewClientConnection subroutine is shown below. Since clients cannot function without the server, it stops if there is no server connected.

```

77  public async Task NewClientConnection(Socket handler)
78  {
79      if (_server == null)
80      {
81          Console.WriteLine("No server... Cannot connect client...");
82          return;
83      }
84
85      string username = await ReceiveTransmission(handler);
86
87      Console.WriteLine($"Username received: {username}");
88
89      TransmitData(_server, $"NEW CLIENT: {username}");
90      _handlerToProcess["{username}Connection"] = (userId) => {
91          Console.WriteLine($"New user id: {userId}");
92          _userIdToSocket[Guid.Parse(userId)] = handler;
93      };
94  }

```

The client will send a username, so this subroutine listens for that username. When it is received, the `_handlerToProcess` dictionary is populated with a key of the username + "Connection" and a value of a method that takes the userID given by the server and uses it to store the client's socket handler against.

Both `ServerConnection` and `ClientConnection` rely on the `ReceiveTransmission` method to receive transmissions from the given socket. It is shown below.

```
110  public async Task<string> ReceiveTransmission(Socket handler)
111  {
112      var buffer = new byte[1024];
113
114      int transmissionBytesCount = await handler.ReceiveAsync(buffer, SocketFlags.None);
115
116      return Encoding.UTF8.GetString(buffer, 0, transmissionBytesCount);
117  }
```

It allocates a buffer of 1024 bytes and receives data using the socket and that newly defined buffer. It then converts the received bytes to a string and returns that. Since the buffer is only 1024 bytes and no chunking has been done yet, transmissions over 1024 bytes will get cut off.

To send transmissions to a certain socket, the method `TransmitData` was created. For now, it is only used to send transmissions to the server. The method converts the message to a byte array, sends it via the given socket, and returns the number of bytes sent.

```
119  public async Task<int> TransmitData(Socket handler, string message)
120  {
121      byte[] messageBytes = Encoding.UTF8.GetBytes(message);
122
123      return await handler.SendAsync(messageBytes);
124  }
```

The `RouterService` has private attributes that are used throughout its subroutines, these are shown below.

```
private IPEndPoint _ipEndPoint;
private ConcurrentDictionary<Guid, Socket> _userIdToSocket;
private Socket _server;

private ConcurrentDictionary<string, Action<string>> _handlerToProcess;

public RouterService(IPEndPoint ipEndPoint)
{
    _userIdToSocket = new ConcurrentDictionary<Guid, Socket>();
    _handlerToProcess = new ConcurrentDictionary<string, Action<string>>();

    _ipEndPoint = ipEndPoint;
}
```

The `_ipEndPoint` stores the endpoint of the listener socket. The `_userIdToSocket` stores all the userIDs of the users connected to the router. The `_handlerToProcess` is an extremely simple version of the `TopicHandlerService`.

I chose to use `ConcurrentDictionary` types instead of normal `Dictionary` types since they handle concurrency issues such as race conditions and deadlocking for me. Since each dictionary may be accessed by different threads at the same time, these concurrency issues will have to be handled.

The code for the server is shown below.

```

19  var routerEndPoint = new IPEndPoint(routerIPAddress, routerPort);
20
21  var dataLayer = new DataLayer();
22  var userRepository = new UserRepository(dataLayer);
23
24  using (Socket server = new Socket(routerEndPoint.AddressFamily, SocketType.Stream, ProtocolType.Tcp))
25  {
26      await server.ConnectAsync(routerEndPoint);
27
28      Console.WriteLine("Server connected");
29
30      await server.SendAsync(Encoding.UTF8.GetBytes("SERVER"), SocketFlags.None);
31
32      while (true)
33      {
34          var buffer = new byte[1024];
35          int receivedBytes = await server.ReceiveAsync(buffer, SocketFlags.None);
36
37          string message = Encoding.UTF8.GetString(buffer, 0, receivedBytes);
38          Console.WriteLine($"Transmission received: {message}");
39          string[] splitMessage = message.Split(": ");
40
41          if (splitMessage[0] == "NEW CLIENT")
42          {
43              User user = userRepository.AddUser(splitMessage[1]);
44              Console.WriteLine($"New user: {user.Username}");
45
46              var header = $"{user.Username}Connection";
47
48              var sendingMessage = $"{header} {user.UserId}";
49              var sendingBytes = Encoding.UTF8.GetBytes(sendingMessage);
50
51              await server.SendAsync(sendingBytes, SocketFlags.None);
52              Console.WriteLine("New user information sent...");
53          }
54      }
55  }

```

```

8  Console.WriteLine("SERVER");
9
10 var hostName = Dns.GetHostName();
11 Console.WriteLine($"On host: {hostName}");
12
13 Console.WriteLine("Router IP address: ");
14 IPAddress routerIPAddress = IPAddress.Parse(Console.ReadLine());
15
16 Console.WriteLine("Router port: ");
17 int routerPort = Convert.ToInt32(Console.ReadLine());

```

The routerIPAddress and routerPort are input by the user. The code retrieving that is shown to the right.

The IP address is parsed from the string input by the user and so is the port number.

I decided on using a 'repository' system to access the database. The repositories would be the classes that interact with the database and perform logic on the data (e.g. hashing passwords).

The data layer class is the main interface to the database. For now, it is just an in-memory database. The code for it is shown below.

```
12  public class DataLayer : IDataLayer
13  {
14      private Dictionary<Guid, Message> messages;
15      private Dictionary<Guid, User> users;
16
17      public DataLayer()
18      {
19          messages = new Dictionary<Guid, Message>();
20          users = new Dictionary<Guid, User>();
21      }
22
23      public void AddMessage(Message message)
24      {
25          messages.Add(message.MessageId, message);
26      }
27
28      public void AddUser(User user)
29      {
30          users.Add(user.UserId, user);
31      }
32
33      public dynamic GetMessage(Guid messageId)
34      {
35          messages.TryGetValue(messageId, out var message);
36
37          return message;
38      }
39      public dynamic GetUser(Guid userId)
40      {
41          users.TryGetValue(userId, out var user);
42
43          return user;
44      }
45  }
```

To enable tests to mock the database, I created an abstraction for it which I have shown later.

It uses dictionaries to enable fast retrieval of records by their primary keys.

I also created data models for users and messages. These are shown below and, although incomplete, they can be used to establish basic communication between applications.

```
10  public class User
11  {
12      public Guid UserId { get; set; }
13
14      public string Username { get; set; }
15  }
```

```
12  public class Message
13  {
14      public Guid MessageId { get; set; }
15
16      public string Content { get; set; }
17
18      public Guid SenderId { get; set; }
19  }
```

Since I am using Test Driven Development, I started off by defining the tests for the user and message repositories. I am using the NUnit framework for the tests.

The user repository tests are shown below.

```

19     [Test]
20     public void AddUserAddsUserToDB()
21     {
22         User user = _userRepository.AddUser("User1");
23
24         User result = _dataLayer.GetUser(user.UserId);
25         Assert.That(result?.Username == "User1", $"Username was: {result?.Username}");
26     }
27
28
29     [Test]
30     public void GetUserGetsUserFromDB()
31     {
32         var user = new User()
33         {
34             UserId = new Guid(),
35             Username = "User1"
36         };
37
38         _dataLayer.AddUser(user);
39
40         User result = _userRepository.GetUser(user.UserId);
41         Assert.That(result?.Username == "User1", $"Username was: {result?.Username}");
42     }

```

Currently the only functionality I have developed is adding and getting users, so that is the only functionality that will be tested. I started off with only a single test case for each test.

The setup method for the tests (the method that is called before every test is executed) is shown to the right along with the definition of all the variables the tests use.

```

9         private IDataLayer _dataLayer;
10        private IUserRepository _userRepository;
11
12        [SetUp]
13        public void Setup()
14        {
15            _dataLayer = new MockDB();
16            _userRepository = new UserRepository(_dataLayer);
17        }

```

```

6        public class UserRepository : IUserRepository
7        {
8            private readonly IDataLayer _dataLayer;
9
10           public UserRepository(IDataLayer dataLayer)
11           {
12               _dataLayer = dataLayer;
13           }

```

Currently, the implementation I am testing is the UserRepository class. The constructor and attributes of the class is shown to the left. As shown, it requires an IDataLayer object which is discussed later.

```

15        public dynamic AddUser(string username)
16        {
17            var user = new User() {
18                UserId = Guid.NewGuid(),
19                Username = username
20            };
21
22            _dataLayer.AddUser(user);
23
24            return user;
25        }

```

To test adding a user, the repository is used to add a user of username "User1" and then the database is checked directly through the _dataLayer in memory mock database. If the result from getting the user from the database matches the initial input, the test has passed otherwise it has failed. To pass this test, I wrote the method shown on the left.

To test retrieving a user, a user is added to the database directly with username “User1”. Then the `userRepository` is used to retrieve the user by their `userId`, if the information it returns matches the original input, the test has passed otherwise it has failed.

```
27     public dynamic GetUser(Guid userId)
28     {
29         return _dataLayer.GetUser(userId);
30     }
```

To pass this test, I wrote the following method in the `UserRepository` class.

The actual user repository instance each test uses is of type `IUserRepository` and the implementation being tested is defined in the setup method, this allows me to swap out the implementation I am currently testing with ease, since all the tests should pass for all implementations.

I also wrote tests for the message repository. They are shown below.

```
19     [Test]
20     public void AddMessageAddsMessageToDB()
21     {
22         Message message = _messageRepository.AddMessage(new Guid(), "Message1");
23
24         Message result = _dataLayer.GetMessage(message.MessageId);
25         Assert.That(result?.Content == "Message1", $"Content was: {result?.Content}");
26     }
27
28
29     [Test]
30     public void GetMessageGetsMessageFromDB()
31     {
32         var message = new Message()
33         {
34             MessageId = new Guid(),
35             SenderId = new Guid(),
36             Content = "Message1"
37         };
38
39         _dataLayer.AddMessage(message);
40
41         Message result = _messageRepository.GetMessage(message.MessageId);
42         Assert.That(result?.Content == "Message1", $"Message name was: {result?.Content}");
43     }
```

Currently the only functionality I have developed is adding and getting messages, so that is the only functionality that will be tested. I started off with only a single test case for each test.

The setup method for the tests (the method that is called before every test is executed) is shown to the right along with the definition of all the variables the tests use.

```
9         private IDataLayer _dataLayer;
10        private IMessageRepository _messageRepository;
11
12        [SetUp]
13        public void Setup()
14        {
15            _dataLayer = new MockDB();
16            _messageRepository = new MessageRepository(_dataLayer);
17        }
```

```
8      private readonly IDataLayer _dataLayer;  
9  
10     public MessageRepository(IDataLayer dataLayer)  
11     {  
12         _dataLayer = dataLayer;  
13     }
```

Currently, the implementation I am testing is the MessageRepository class. The constructor and attributes of the class is shown to the left. As shown, it requires an IDataLayer object which is discussed later.

To test adding a message, the repository is used to add a message with content “Message1” and then the database is checked directly through _dataLayer. If the result from getting the message from the database matches the initial input, the test has passed otherwise it has failed. To pass this test, I wrote the method shown to the right.

```
15  public dynamic AddMessage(Guid senderId, string content)  
16  {  
17      var message = new Message() {  
18          MessageId = Guid.NewGuid(),  
19          Content = content,  
20          SenderId = senderId  
21      };  
22  
23      _dataLayer.AddMessage(message);  
24  
25      return message;  
26  }
```

To test retrieving a message, a message is added to the database directly with content “Message1”. Then the repository is used to retrieve the message by its messageId, if the information it returns matches the original input, the test has passed otherwise it has failed.

```
28     public dynamic GetMessage(Guid messageId)  
29     {  
30         return _dataLayer.GetMessage(messageId);  
31     }
```

The code I wrote to pass that test is shown on the left.

```

12  ✓ public class MockDB : IDataLayer
13      {
14          private Dictionary<Guid, Message> messages;
15          private Dictionary<Guid, User> users;
16
17      ✓ public MockDB()
18          {
19              messages = new Dictionary<Guid, Message>();
20              users = new Dictionary<Guid, User>();
21          }
22
23      public void AddMessage(Message message)
24          {
25              messages.Add(message.MessageId, message);
26          }
27
28      public void AddUser(User user)
29          {
30              users.Add(user.UserId, user);
31          }
32
33  ✓ public dynamic GetMessage(Guid messageId)
34      {
35          messages.TryGetValue(messageId, out var message);
36
37          return message;
38      }
39  ✓ public dynamic GetUser(Guid userId)
40      {
41          users.TryGetValue(userId, out var user);
42
43          return user;
44      }
45  }

```

For the testing environment I created a mock data layer (shown to the left).

Although the real data layer will utilise an actual database, the mock data layer simply uses dictionaries to create an in-memory database. This helps me isolate any problems to classes being tested, since the mock database must work without any errors, if there are any errors then I will have to fix them before I continue developing which may prove to be a hassle.

The abstraction of IDataLayer is shown below. For now, a datalayer simply provides functionality to add and get messages and users.

```

11  ✓ public interface IDataLayer
12      {
13          public void AddUser(User user);
14          public void AddMessage(Message message);
15
16          public dynamic GetUser(Guid userId);
17          public dynamic GetMessage(Guid messageId);
18      }

```

Using abstractions allows me to create mocks of all the classes and use them to test other classes, it also enables polymorphism which could prove useful to extend the functionality of the code. The IMessageRepository and IUserRepository abstractions are shown below.

```

5  ✓ public interface IMessageRepository
6      {
7          public dynamic AddMessage(Guid senderId, string content);
8          public dynamic GetMessage(Guid messageId);
9      }

```

```

5  ✓ public interface IUserRepository
6      {
7          public dynamic AddUser(string username);
8          public dynamic GetUser(Guid userId);
9      }

```

I ran the tests and all 4 passed. Then I ran the programs in the following order: router, server, client, client. This is shown below.

```

C:\_Kabushak\Programming\Applications\Orion\Orion\Broker\bin\Debug\net8.0\Bro...
IP address:
192.168.0.26
Port:
50000
Running broker...
Broker running...
Connection attempt successful, new client connected...
Handling new connection...
Server connected...
Connection handled...
Connection attempt successful, new client connected...
Handling new connection...
Connection handled...
Username received: User1
New user id: 5b1f6460-189b-4c47-927a-4aba6684dead

C:\_Kabushak\Programming\Applications\Orion\Orion\Client\bin\Debug\net8.0\Cle...
CLIENT
On host: Peri
Broker IP address:
192.168.0.26
Broker port:
50000
Client connected
Username:
User1

C:\_Kabushak\Programming\Applications\Orion\Orion\Server\bin\Debug\net8.0\Ser...
SERVER
On host: Peri
Broker IP address:
192.168.0.26
Broker port:
50000
Server connected
Transmission received: NEW CLIENT: User1
New user: User1
New user information sent...

C:\_Kabushak\Programming\Applications\Orion\Orion\Client\bin\Debug\net8.0\Cle...
CLIENT
On host: Peri
Broker IP address:

```

As shown, the server, router and two instances of the client were run at the same time. After the client was connected to the router, a username was entered into the client which was sent to the server by the router. The server then created a new user and stored it in an in-memory database after which it returned the user id to the router. The router received the new user id and stored it against the handler. I did not connect the other client just yet.

Now I can work on getting some actual features of the application done. All the transmissions will be done via custom types, to transmit these I will need to convert them to JSON so I created a `JsonService` which can convert to and from JSON. For the actual conversion, I am using a library called `NewtonSoft.Json` and I am wrapping it in my own library, this allows me to easily swap out the library or method I use for serialising and deserialising JSON in just one place, without having to make amends to the whole codebase. To allow for mocking the `JsonService` I made it inherit from an interface. The json service and its interface are both shown below.

```

10  public class JsonService : IJsonService
11  {
12      public T? DeserialiseJson<T>(string json)
13      {
14          return JsonConvert.DeserializeObject<T>(json);
15      }
16
17      public string SerialiseObject(object objectToSerialise)
18      {
19          return JsonConvert.SerializeObject(objectToSerialise);
20      }
21  }

9  public interface IJsonService
10  {
11      public T? DeserialiseJson<T>(string json);
12
13      public string SerialiseObject(object objectToSerialise);
14  }

```

The `Deserialise` method is generic so the JSON can be converted into the desired type directly.

I also created classes for some of the transmission types (ServerResponse, ServerRequest, ClientTransmission). These are shown below.

```
3  ✓ public class ClientTransmission
4      {
5          public string Topic { get; set; }
6
7          public object Data { get; set; }
8
9  ✓      public ClientTransmission(string topic, object data)
10         {
11             Topic = topic;
12             Data = data;
13         }
14     }
```

```
3  ✓ public class ServerRequest
4      {
5          public string Topic { get; set; }
6
7          public object Data { get; set; }
8
9          public Guid? RequestId { get; set; }
10
11  ✓      public ServerRequest(string topic, object data, Guid? requestId = null)
12         {
13             Topic = topic;
14             Data = data;
15             RequestId = requestId;
16         }
17     }
```

```
5  ✓ public class ServerResponse
6      {
7          public Guid? RequestId { get; set; }
8
9          public ServerResult ServerResult { get; set; }
10
11         public string Topic { get; set; }
12
13  ✓      public ServerResponse(string topic, ServerResult serverResult = null, Guid? requestId = null)
14         {
15             Topic = topic;
16             ServerResult = serverResult;
17             RequestId = requestId;
18         }
19     }
```

I also created the OperationInformation and ServerResult classes which are shown below. I called the OperationInformation class OperationResult because I wanted to.

```
3  ✓ public class OperationResult
4      {
5          public OperationMessages OperationMessage { get; set; }
6
7          public OperationResult(OperationMessages operationMessage)
8          {
9              OperationMessage = operationMessage;
10         }
11     }
```

```

3  public class ServerResult
4  {
5      public OperationResult OperationResult { get; set; }
6
7      public object? Data { get; set; }
8
9      public Guid[]? AffectedUsers { get; set; }
10
11  public ServerResult(OperationResult operationResult, object? data = null, Guid[]? affectedUsers = null)
12  {
13      OperationResult = operationResult;
14      Data = data;
15      AffectedUsers = affectedUsers;
16  }
17  }

```

```

3  public enum OperationMessages
4  {
5      VALID_CREDENTIALS,
6      INVALID_CREDENTIALS
7  }

```

OperationResult currently just contains a message code, no message, and the message codes are represented in an enumerator since there is only a fixed set of values it can take. The enumerator is shown to the left.

Since, as of now, there is only one operation, AuthenticateUser, there are only message codes relating to that operation. When there are many operations, I want to have an enumerator for each operation rather than storing all the message codes in a single massive enumerator but I have not done that yet.

As shown, the affected users are contained in the ServerResult class for now, the reason for this is explained later.

I also moved all the code from the Program.cs file into its own ServerService. The constructor for the ServerService is shown below.

```

12  private readonly Dictionary<string, Action<object>> _topicToHandler;
13
14  private readonly IJsonService _jsonService;
15  private readonly IPEndPoint _routerIpEndpoint;
16
17
18  private Socket _router;
19
20
21  public ServerService(IPEndPoint routerIpEndpoint, IJsonService jsonService)
22  {
23      _topicToHandler = new Dictionary<string, Action<object>>();
24      _topicToHandler.Add("AuthenticateClient");
25
26      _router = new Socket(routerIpEndpoint.AddressFamily, SocketType.Stream, ProtocolType.Tcp);
27
28      _routerIpEndpoint = routerIpEndpoint;
29      _jsonService = jsonService;
30  }

```

I created a dictionary _topicToHandler to act as a topic handler service. I also make it require an instance of the json service since it will need this to handle transmissions. I am not creating the layered system yet.

```

32  public void Run()
33  {
34      Console.WriteLine("Server running...");
35
36      _router.ConnectAsync(_routerIpEndpoint);
37
38      Task.Run(() =>
39      {
40          while(true)
41          {
42              var request = await ReceiveRequest();
43
44              if (request == null)
45                  continue;
46
47              HandleRequest(request);
48          }
49      });
50
51      Console.ReadLine();
52  }

```

To the left is the subroutine that runs the server. This method connects to the router and listens for requests from the router. The actual receiving of requests is handled in the ReceiveRequest subroutine and the handling of the requests is done in the HandleRequest subroutine. I am not interested in robustness yet, so if a request is ever null (in which case it means the router has disconnected) I just continue so no errors are thrown, this will be handled better later.

Below is shown the HandleRequest and ReceiveRequest subroutines. As shown, I still need to make a way to handle requests. ReceiveRequest simply takes the bytes converts them to a string and parses the json string into an object of type ServerRequest.

```

54  private async Task HandleRequest(ServerRequest request)
55  {
56  }
57
58
59  private async Task<ServerRequest>> ReceiveRequest()
60  {
61      byte[] buffer = new byte[1024];
62
63      int transmissionLength = await _router.ReceiveAsync(buffer);
64
65      string transmissionJson = Encoding.UTF8.GetString(buffer, 0, transmissionLength);
66
67      ServerRequest? request = _jsonService.DeserialiseJson<ServerRequest>(transmissionJson);
68
69      return request;
70  }

```

Inspired by ASP.NET's MVC approach, I decided that authorisation and the handling of transmissions will be handled in controllers. These controllers would then use the repositories whenever work would need to be done on the database. Initially, I wanted the controllers to return ServerResponse objects (I am not handling publishes just yet) and I wanted the repositories to return ServerResult objects. The controllers would then create the ServerResponse objects based on the ServerResult given by the repository.

To create the controllers, I made custom attributes Controller and Handler. These are shown below.

```

3      public class ControllerAttribute : Attribute
4      {
5
6      }
7
8      public class HandlerAttribute : Attribute
9      {
10         public string Topic { get; set; }
11         public HandlerAttribute(string topic)
12         {
13             Topic = topic;
14         }
15     }

```

The handler attribute takes a topic, this attribute will decorate all the subroutines that handle topics and the topic they handle will be passed into the handler attribute. An example of how these attributes will be used will be shown when working on the UserController.

I then created a class called TopicHandlerService which would use C# reflections to find all the classes decorated with the Controller attribute and retrieve all the methods in that class decorated with the Handler attribute. It uses all of the information it finds to store the handlers against their topics in a dictionary.

The constructor is shown below.

```

10     private readonly IDictionary<string, Func<object, Task<ServerResult>>> topicToHandler;
11
12     public TopicHandlerService(IConfigurationService configurationService)
13     {
14         topicToHandler = new Dictionary<string, Func<object, Task<ServerResult>>>();
15
16         var controllers =
17             from a in AppDomain.CurrentDomain.GetAssemblies()
18             from t in a.GetTypes()
19             let attributes = t.GetCustomAttributes(typeof(ControllerAttribute), false)
20             where attributes != null && attributes.Length > 0
21             select t;
22
23         foreach (var controller in controllers)
24         {
25             var constructor = controller.GetConstructors().Single();
26             var constructorParameters = constructor.GetParameters();
27
28             var arguments = new List<object>();
29
30             foreach (var parameter in constructorParameters)
31             {
32                 var argument = configurationService.GetInstanceOfType(parameter.ParameterType);
33                 arguments.Add(argument);
34             }
35
36             var controllerInstance = constructor.Invoke(arguments.ToArray());
37
38             var handlers = controller.GetMethods()
39                 .Where(m => m.GetCustomAttributes(typeof(HandlerAttribute), false) != null && m.GetCustomAttributes(typeof(HandlerAttribute), false).Length > 0);
40
41             foreach (var handler in handlers)
42             {
43                 topicToHandler.Add(handler.GetCustomAttribute<HandlerAttribute>().Topic, async (x) =>
44                 {
45                     return await ((Task<ServerResult>)handler.Invoke(controllerInstance, new object[] { x }));
46                 });
47             }
48         }
49     }
50 }
51

```

Since the constructor is so big, I will split it up and explain each part.

Below is the definition of the topicToHandler dictionary. This will store the topics against their handlers. And will be used to retrieve the handlers for any topic.

```
10     private readonly IDictionary<string, Func<object, Task<ServerResult>>> topicToHandler;
11
12     public TopicHandlerService(IConfigurationService configurationService)
13     {
14         topicToHandler = new Dictionary<string, Func<object, Task<ServerResult>>>();
15     }
```

I get all the controllers by searching through every single class in the project and finding classes decorated with the Controller attribute.

```
17     var controllers =
18         from a in AppDomain.CurrentDomain.GetAssemblies()
19         from t in a.GetTypes()
20         let attributes = t.GetCustomAttributes(typeof(ControllerAttribute), false)
21         where attributes != null && attributes.Length > 0
22         select t;
```

I then loop through every controller that is found (there will be one for each entity) and I get their first constructor. Doing it this way means controllers can only have a single constructor which should be fine. To store the handlers in the topicToHandler dictionary, I will need to create instances of the controllers, which is why I will need to call the constructors of the controllers.

```
25     foreach (var controller in controllers)
26     {
27         var constructor = controller.GetConstructors().Single();
28         var constructorParameters = constructor.GetParameters();
29
30         var arguments = new List<object>();
```

To pass in arguments into the constructors, I will use a dependency injector that I created.

```
32         foreach (var parameter in constructorParameters)
33         {
34             var argument = configurationService.GetInstanceOfType(parameter.ParameterType);
35             arguments.Add(argument);
36         }
37
38         var controllerInstance = constructor.Invoke(arguments.ToArray());
```

For each parameter in the constructor's parameters, I get an instance of the parameter's type via the dependency injector and store it in the arguments list. Doing it this way means the controllers cannot have 2 arguments of the same type for different things e.g. two integers for addition but due to the nature of controllers, this is unlikely. Using the final arguments list, I instantiate the controller.

Once the controller is instantiated, I get all the methods in the controller decorated with the Handler attribute, loop through them and store each against their topic in the topicToHandler dictionary.

```

40         var handlers = controller.GetMethods()
41             .Where(m => m.GetCustomAttributes(typeof(HandlerAttribute), false) != null && m.GetCustomAttributes(typeof(HandlerAttribute)
42
43         foreach (var handler in handlers)
44         {
45             topicToHandler.Add(handler.GetCustomAttribute<HandlerAttribute>().Topic, async (x) =>
46             {
47                 return await ((Task<ServerResult>)handler.Invoke(controllerInstance, new object[] { x }));
48             });
49     }

```

Each handler can only take in a single argument, this is the data that would be contained in a request to the server. This is the only way to accomplish what I am trying to achieve.

It was either: do what I did above or create and update the topicToHandler dictionary in the server service myself every time I wrote more code to handle more requests. This makes the code more maintainable and makes it easier to add new features as I do not need to worry about manually updating a dictionary for every new feature.

```

53  ✓      public Func<object, Task<ServerResult>>? GetTopicHandler(string topic)
54      {
55          topicToHandler.TryGetValue(topic, out var handler);
56
57          return handler;
58      }

```

To get a handler by its topic, I made a method called GetTopicHandler, shown on the left.

For the topic handler service to work, I built a basic form of a dependency injector. I called it ConfigurationService and its code is displayed below.

```

5         private IDictionary<Type, object> instances;
6
7         public ConfigurationService()
8         {
9             instances = new Dictionary<Type, object>();
10        }
11
12        public void AddInstanceOfType<T>(T instance)
13        {
14            instances.Add(typeof(T), instance);
15        }
16
17  ✓      public T? GetInstanceOfType<T>()
18      {
19          instances.TryGetValue(typeof(T), out var instance);
20
21          return (T?)instance;
22      }
23
24
25  ✓      public object GetInstanceOfType(Type type)
26      {
27          instances.TryGetValue(type, out var instance);
28
29          return instance;
30      }

```

For now, all it does is that it needs to be populated before the application is run using AddInstanceOfType. It then stores each instance against its type. So, the 'dependency injector' does not create any instances it just stores them, the instances are created before the program is run.

I store all dependencies or objects that may become dependencies in the Program.cs file, this is done before the server is run. I have shown the code for this below.

```

12     var configurationService = new ConfigurationService();
13
14     configurationService.AddInstanceOfType<IJsonService>(new JsonService());
15
16     configurationService.AddInstanceOfType<IDataLayer>(new DataLayer());
17     configurationService.AddInstanceOfType<IUserRepository>(new UserRepository(configurationService.GetInstanceOfType<IDataLayer>()));
18     configurationService.AddInstanceOfType<IMessageRepository>(new MessageRepository(configurationService.GetInstanceOfType<IDataLayer>()));
19
20     configurationService.AddInstanceOfType<ITopicHandlerService>(new TopicHandlerService(configurationService));
21

```

For completion, the entire Program.cs file without the using statements is shown below.

```

12     var configurationService = new ConfigurationService();
13
14     configurationService.AddInstanceOfType<IJsonService>(new JsonService());
15
16     configurationService.AddInstanceOfType<IDataLayer>(new DataLayer());
17     configurationService.AddInstanceOfType<IUserRepository>(new UserRepository(configurationService.GetInstanceOfType<IDataLayer>()));
18     configurationService.AddInstanceOfType<IMessageRepository>(new MessageRepository(configurationService.GetInstanceOfType<IDataLayer>()));
19
20     configurationService.AddInstanceOfType<ITopicHandlerService>(new TopicHandlerService(configurationService));
21
22     Console.WriteLine("SERVER");
23
24     var hostName = Dns.GetHostName();
25     Console.WriteLine($"On host: {hostName}");
26
27     Console.WriteLine("Router IP address: ");
28     IPAddress routerIPAddress = IPAddress.Parse(Console.ReadLine());
29
30     Console.WriteLine("Router port: ");
31     int routerPort = Convert.ToInt32(Console.ReadLine());
32
33     var routerEndPoint = new IPEndPoint(routerIPAddress, routerPort);
34
35     var serverService = new ServerService(routerEndPoint,
36     configurationService.GetInstanceOfType<IJsonService>(),
37     configurationService.GetInstanceOfType<ITopicHandlerService>());
38
39     serverService.Run();

```

Using all of this, I started making the first request handling thing in the server, AuthenticateUser. I did this by making a UserController for user-related operations, as shown it takes in a IUserRepository instance.

```

7         [Controller]
8     public class UserController : IUserController
9     {
10         private readonly IUserRepository _userRepository;
11
12         public UserController(IUserRepository userRepository)
13         {
14             _userRepository = userRepository;
15         }
16
17         [Handler("AuthenticateUser")]
18         public async Task<ServerResult?> AuthenticateUser(string username)
19         {
20             return await _userRepository.AuthenticateUser(username);
21         }
22     }

```

Currently, it only has a single handler for the topic "AuthenticateUser".

As stated in the design, all operations from the server will result in a `ServerResult` object, so I made the `userRepository` return a `ServerResult` object. In addition to this, I added another method to the user repository to authenticate users. I started by adding 2 tests for the authentication as shown below.

```

21     [Test]
22     [TestCase("User1")]
23     [TestCase("American bald eagle")]
24     [TestCase("User212343441")]
25     public async Task AuthenticateUser_ReturnsTrueForValidUsername(string username)
26     {
27         await _dataLayer.AddUser(new User()
28         {
29             UserId = Guid.NewGuid(),
30             Username = username
31         });
32
33         ServerResult result = await _userRepository.AuthenticateUser(username);
34
35         Assert.That(result.OperationResult.OperationMessage == OperationMessages.VALID_CREDENTIALS);
36     }
37
38     [Test]
39     [TestCase("User1")]
40     [TestCase("American bald eagle")]
41     [TestCase("User212343441")]
42     public async Task AuthenticateUser_ReturnsFalseForInvalidUsername(string username)
43     {
44         Assert.That((await _dataLayer.GetAllUsers()).Count == 0, "Invalid test conditions");
45
46         ServerResult result = await _userRepository.AuthenticateUser(username);
47
48         Assert.That(result.OperationResult.OperationMessage == OperationMessages.INVALID_CREDENTIALS);
49     }

```

As stated in the tests' names, one test tests if the `userRepository` returns a successful result for valid credentials while the other checks if it returns an unsuccessful result for invalid credentials. Currently, a user is only being validated by their username so we do not need to check that the user repository handles valid and invalid passwords yet. Each test has 3 cases, users with usernames: "User1", "American bald eagle" and "User212343441" to test that the method works for strings with multiple words, random content and any length. These were (definitely) generated by ChatGPT to acquire randomness.

The `AuthenticateUser` method in the user repository is shown below.

```

16     public async Task<ServerResult?> AuthenticateUser(string username)
17     {
18         var users = await _dataLayer.GetAllUsers();
19
20         var user = users?
21             .Select(x => x)
22             .Where(x => x.Username == username)
23             .SingleOrDefault();
24
25         return (user == null)
26             ? ServerResultService.NewServerResult(OperationMessages.INVALID_CREDENTIALS)
27             : ServerResultService.NewServerResult(OperationMessages.VALID_CREDENTIALS, user.UserId, user);
28     }

```

The creation of server result objects is handled by a static class `ServerResultService` to make the code more readable and maintainable. The class is shown below.

```

5  public static class ServerResultService
6  {
7      public static ServerResult NewServerResult(OperationMessages operationMessage, object? data = null, Guid[]? affectedUsers = null)
8      {
9          return new ServerResult(new OperationResult(operationMessage), data, affectedUsers);
10     }
11     public static ServerResult NewServerResult(OperationMessages operationMessage, Guid affectedUser, object? data = null)
12     {
13         return new ServerResult(new OperationResult(operationMessage), data, new Guid[1] { affectedUser });
14     }
15 }

```

As shown, for now, it just makes the code easier to read. There are 2 overloads of the same method, one which takes in a single affected user and one that takes in a collection.

Using all of this, I wrote the `ServerService`'s `HandleRequest` method. It is shown below.

```

71 private async Task<ServerResponse> HandleRequest(ServerRequest request)
72 {
73     var handler = _topicHandlerService.GetTopicHandler(request.Topic);
74
75     if (handler == null)
76         throw new ApplicationException($"No handler found for topic: {request.Topic}");
77
78     ServerResult result = await handler.Invoke(request.Data);
79
80     return new ServerResponse(request.Topic + "Result", result, request.RequestId);
81 }

```

It uses the topic handler service to find handlers for topics and executes the handlers with the data in the request to get `ServerResult` objects. It then packages those `ServerResult` objects into `ServerResponse` objects with the topic of the request concatenated with "Result" as stated in design.

Finally, I tested all this for the first time (bad idea). I did this by adding the following lines in the `Program.cs` file of the server.

```

25 var topicHandlerService = new TopicHandlerService(configurationService);
26
27 var handler = topicHandlerService.GetTopicHandler("AuthenticateUser");
28 var response = await handler.Invoke("Bob");
29
30 Console.WriteLine(response.OperationResult.OperationMessage.ToString());

```

```

11 public DataLayer()
12 {
13     messages = new Dictionary<Guid, Message>();
14     users = new Dictionary<Guid, User>();
15
16     var user = new User()
17     {
18         UserId = Guid.NewGuid(),
19         Username = "User1"
20     };
21
22     users[user.UserId] = user;
23 }

```

I also added a user in the constructor of the in-memory database as shown to the left.

```
INVALID_CREDENTIALS
SERVER
On host: Peri
Router IP address:
```

When running this I got the following result on the left, as required.

I then changed the Program.cs file again:

```
25 var topicHandlerService = new TopicHandlerService(configurationService);
26
27 var handler = topicHandlerService.GetTopicHandler("AuthenticateUser");
28 var response = await handler.Invoke("User1");
29
30 Console.WriteLine(response.OperationResult.OperationMessage.ToString());
```

```
VALID_CREDENTIALS
SERVER
On host: Peri
Router IP address:
```

And this time, I got what is shown on the left, as required.

After removing this testing code. I tried running the router and server, however I got an unexpected result, shown below:

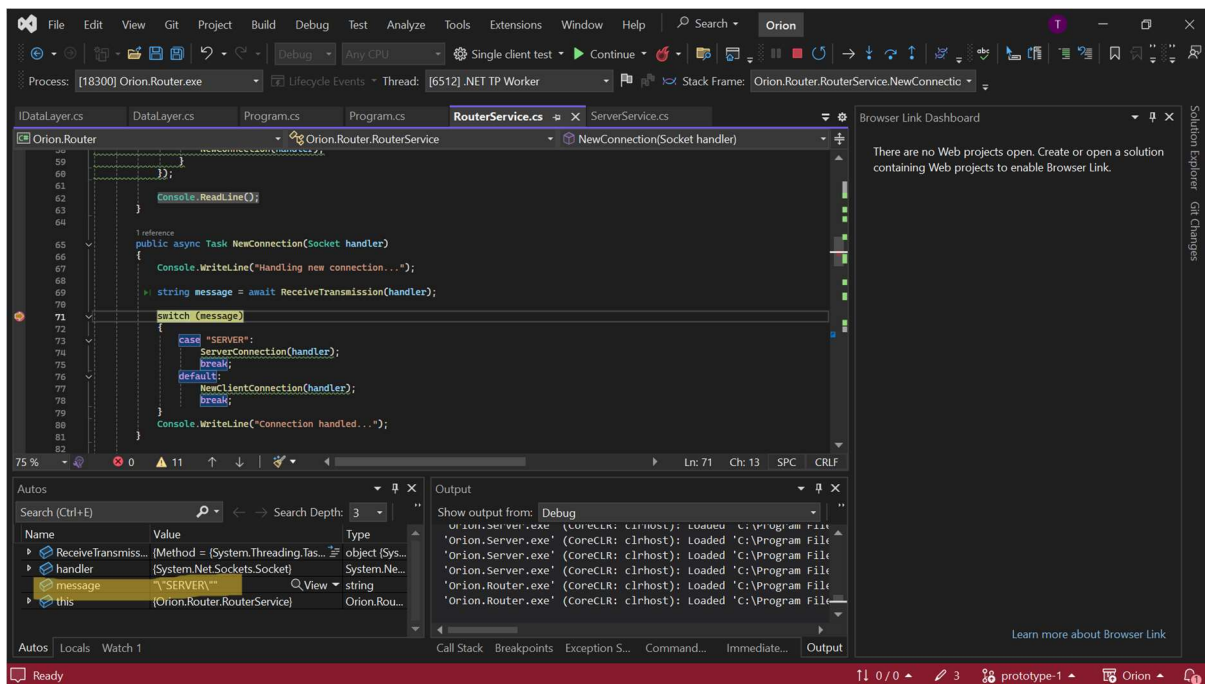
```

C:\_Kabushak_Programming\Applications\Orion\Orion\Broker\bin\Debug\net8.0\Ori...
IP address:
192.168.0.26
Port:
50000
Running broker...
Broker running...
Connection attempt successful, new client connected...
Handling new connection...
No server... Cannot connect client...
Connection handled...

C:\_Kabushak_Programming\Applications\Orion\Orion\Server\bin\Debug\net8.0\Ori...
VALID_CREDENTIALS
SERVER
On host: Peri
Router IP address:
192.168.0.26
Router port:
50000
Server running...

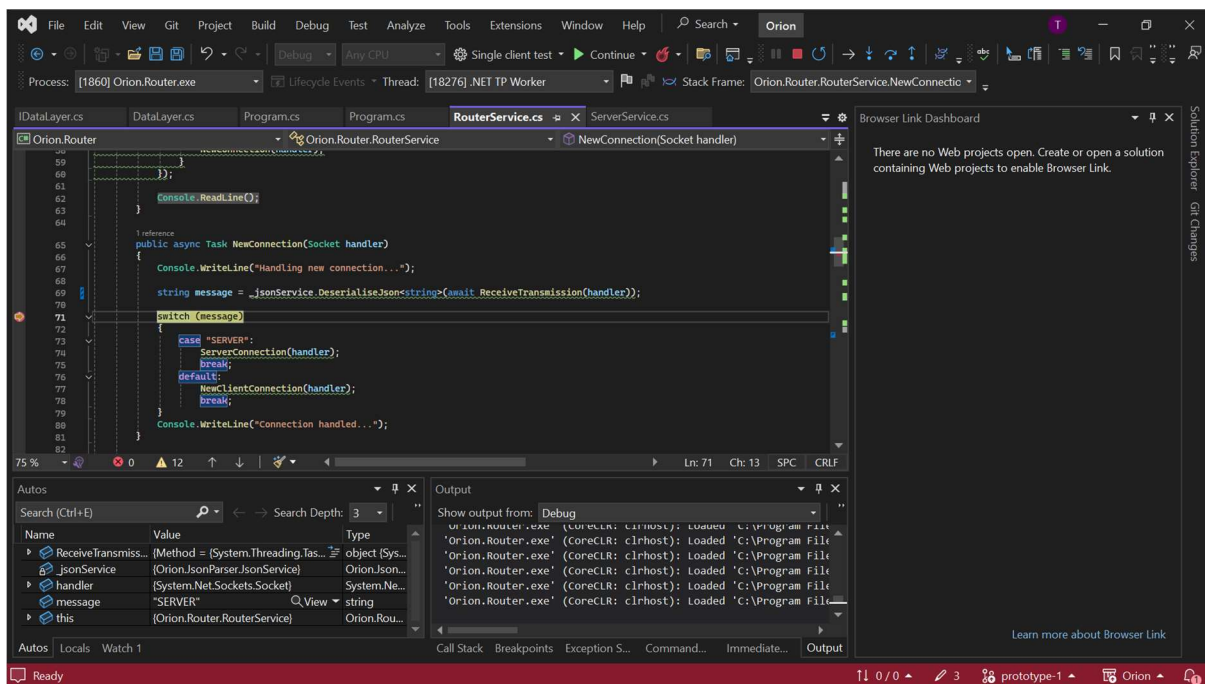
C:\_Kabushak_Programming\Applications\Orion\Orion\Client\bin\Debug\net8.0\Ori...
CLIENT
On host: Peri
Router IP address:
```

As shown, the server was trying to connect to the router, yet the router was treating it as a client connection. I decided to debug to see what was going wrong, I knew the problem was probably something to do with the identifier string sent by the server.



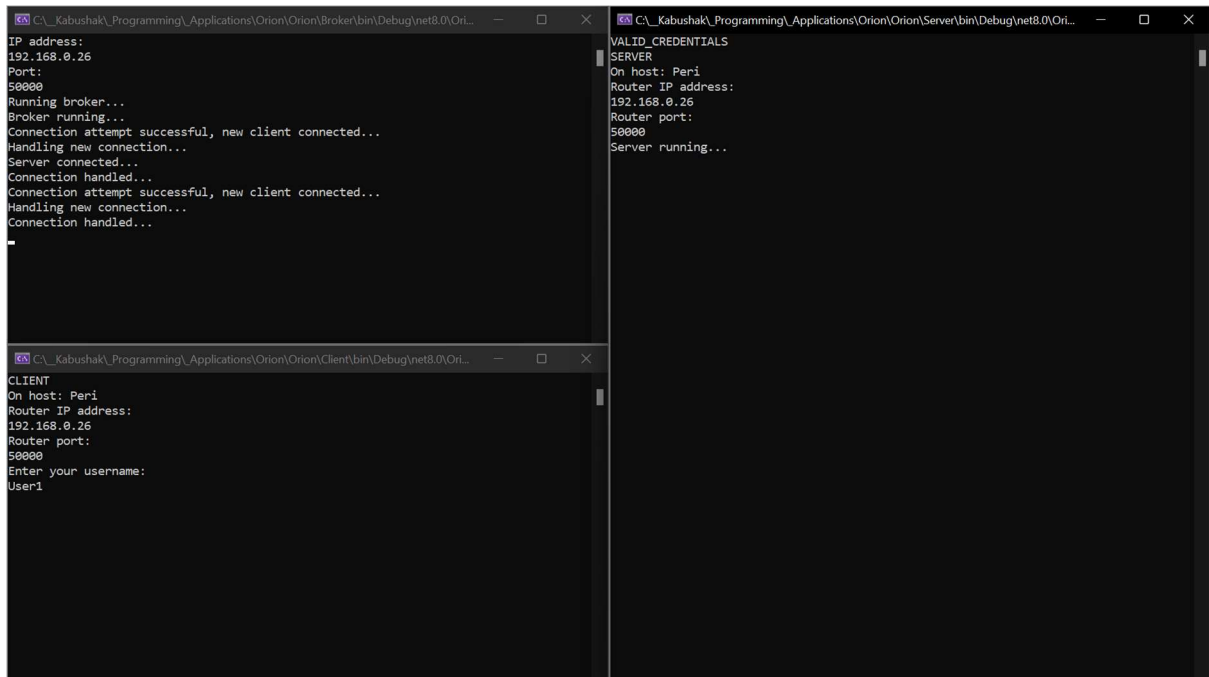
So the router was expecting the server to identify with the string “SERVER” however, the server was instead sending the string “\\SERVER\\”. This is why the router was treating the server as a client.

This unexpected behaviour was due to the JSON serialisation. The server was sending the identifier via its `TransmitData` method which would serialise everything it would send into JSON before actually sending it. The router was not deserialising transmissions that it received, so I made it do just that:



As shown, now the message was being deciphered correctly.

I then tested the client's functionality as well.



The screenshot shows three separate command prompt windows. The top-left window is the Orion Server console, showing logs for IP address (192.168.0.26), Port (50000), and successful connections. The top-right window is the Orion Server console showing 'VALID_CREDENTIALS' and 'SERVER' status, along with host and router information. The bottom window is the Orion Client console, showing host and router information, and a prompt for a username where 'User1' was entered.

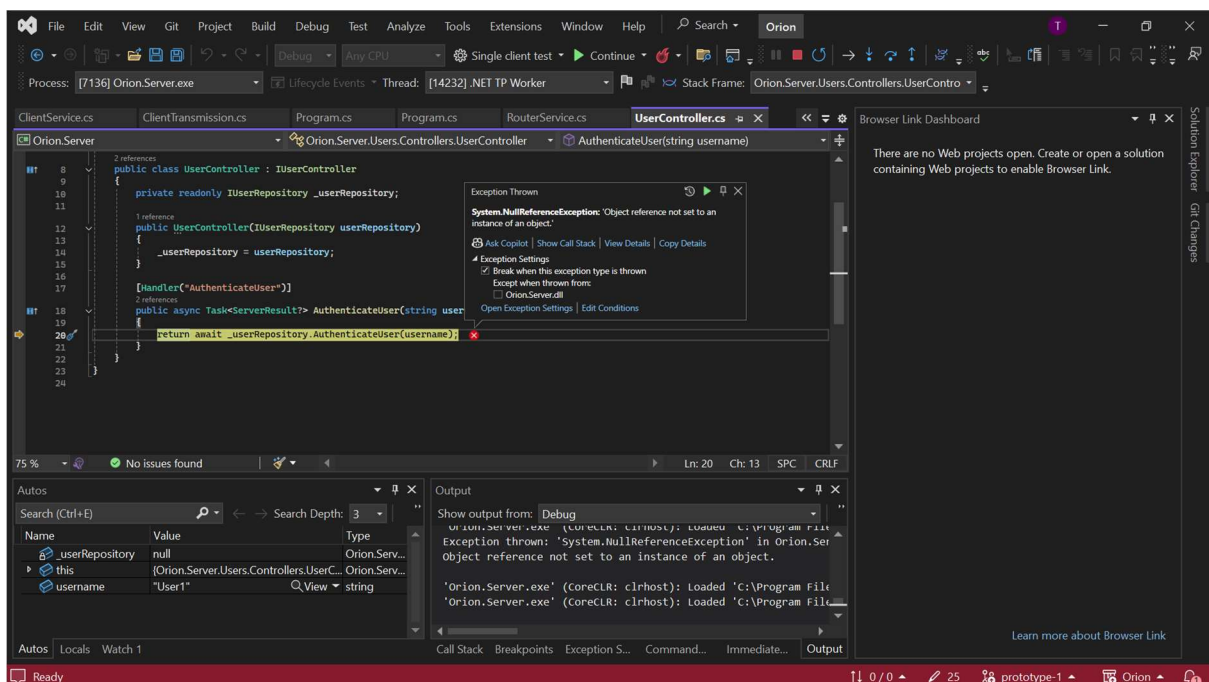
```

C:\_Kabushak_Programming_Applications\Orion\Orion\Broker\bin\Debug\net8.0\Ori...
IP address:
192.168.0.26
Port:
50000
Running broker...
Broker running...
Connection attempt successful, new client connected...
Handling new connection...
Server connected...
Connection handled...
Connection attempt successful, new client connected...
Handling new connection...
Connection handled...

C:\_Kabushak_Programming_Applications\Orion\Orion\Server\bin\Debug\net8.0\Ori...
VALID_CREDENTIALS
SERVER
On host: Peri
Router IP address:
192.168.0.26
Router port:
50000
Server running...

C:\_Kabushak_Programming_Applications\Orion\Orion\Client\bin\Debug\net8.0\Ori...
CLIENT
On host: Peri
Router IP address:
192.168.0.26
Router port:
50000
Enter your username:
User1
  
```

Upon receiving the User1 message, the server encountered an error.



The screenshot shows the Visual Studio IDE with the Orion.Server.exe running in debug mode. The code editor displays the UserController.cs file, specifically the AuthenticateUser method. An exception dialog box is open, showing a 'System.NullReferenceException: Object reference not set to an instance of an object.' The exception was thrown in the AuthenticateUser method. The output window shows the exception details and the loaded assemblies for Orion.Server.exe.

```

Exception Thrown
System.NullReferenceException: Object reference not set to an instance of an object.
Exception Settings
  Break when this exception type is thrown
  Except when thrown from:
    Orion.Server.dll
    Open Exception Settings | Edit Conditions

UserController.cs
public class UserController : IUserController
{
    private readonly IUserRepository _userRepository;

    public UserController(IUserRepository userRepository)
    {
        _userRepository = userRepository;
    }

    [Handler("AuthenticateUser")]
    public async Task<ServerResult> AuthenticateUser(string username)
    {
        return await _userRepository.AuthenticateUser(username);
    }
}

Output
Show output from: Debug
Orion.Server.exe (CORECLR: clrhost): Loaded 'C:\Program File...
Exception thrown: 'System.NullReferenceException' in Orion.Ser...
Object reference not set to an instance of an object.
'Orion.Server.exe' (CORECLR: clrhost): Loaded 'C:\Program File...
'Orion.Server.exe' (CORECLR: clrhost): Loaded 'C:\Program File...
  
```

The controller was not given any instance of IUserRepository. I put a breakpoint where the TopicHandlerService was populating the arguments for the controllers and discovered the problem.

The TopicHandlerService was being instantiated before the configuration service had been populated. So, I simply moved the instantiation of the TopicHandlerService to after the instantiation of the rest of the dependencies. It worked:

The image shows three terminal windows side-by-side, illustrating the execution of a network application. The top-left window shows a broker running on 192.168.0.26:50000, successfully connecting to multiple clients. The top-right window shows a server running on the same IP and port, receiving a request for the 'AuthenticateUser' topic. The bottom window shows a client running on the same IP and port, displaying the host information and a prompt for a username, with 'User1' entered.

```

C:\_Kabushak_Programming\Applications\Orion\Orion\Broker\bin\Debug\net8.0\Ori...
IP address:
192.168.0.26
Port:
50000
Running broker...
Broker running...
Connection attempt successful, new client connected...
Handling new connection...
Server connected...
Connection handled...
Connection attempt successful, new client connected...
Handling new connection...
Connection handled...

C:\_Kabushak_Programming\Applications\Orion\Orion\Server\bin\Debug\net...
SERVER
On host: Peri
Router IP address:
192.168.0.26
Router port:
50000
Server running...
Transmitted response to request of topic: AuthenticateUser

C:\_Kabushak_Programming\Applications\Orion\Orion\Client\bin\Debug\net8.0\Ori...
CLIENT
On host: Peri
Router IP address:
192.168.0.26
Router port:
50000
Enter your username:
User1

```

Now, I needed to make something of the client. As of now, it only sends a single message then completes, it doesn't listen for any messages or anything of the sort. Since the client and the server are so similar, I copied the server service, renamed it and changed a few things as shown in the snippets below.

The identifier was changed from "SERVER" to "CLIENT" and another topicHandlerService was made for the client. This one is defined and explained later.

```

16 public class ClientService
17 {
18     private readonly IJsonService _jsonService;
19     private readonly ITopicHandlerService _topicHandlerService;
20
21     private readonly IPEndPoint _routerIpEndpoint;
22
23     private Socket _router;
24
25     private const string IDENTIFIER = "CLIENT";
26
27
28 public ClientService(IPEndPoint routerIpEndpoint, IJsonService jsonService, ITopicHandlerService topicHandlerService)
29 {
30     _topicHandlerService = topicHandlerService;
31
32     _router = new Socket(routerIpEndpoint.AddressFamily, SocketType.Stream, ProtocolType.Tcp);
33
34     _routerIpEndpoint = routerIpEndpoint;
35     _jsonService = jsonService;
36 }

```

In the Run subroutine, after transmitting the identifier, the user is prompted to enter their username (line 45), this is then transmitted to the router with a topic of “AuthenticateUser” (line 47).

```

38  public async Task Run()
39  {
40      _router.ConnectAsync(_routerIpEndpoint);
41
42      await TransmitData(IDENTIFIER);
43
44      Console.WriteLine("Enter your username: ");
45      string username = Console.ReadLine();
46
47      await TransmitData(new ClientTransmission("AuthenticateUser", username));
48
49      Task.Run(async () =>
50      {
51          while (true)
52          {
53              var request = await ReceiveTransmission();
54
55              Console.WriteLine($"New transmission of topic: {request.Topic}");
56
57              if (request == null)
58                  continue;
59
60              HandleTransmission(request);
61          }
62      });
63
64      Console.ReadLine();
65  }

```

TransmitData remains the same.

```

67  private async Task<int> TransmitData(object data)
68  {
69      string transmissionJson = _jsonService.SerializeObject(data);
70
71      var transmissionBytes = Encoding.UTF8.GetBytes(transmissionJson);
72
73      return await _router.SendAsync(transmissionBytes);
74  }

```

The HandleTransmission method takes in a ClientTransmission and uses a topic handler service to handle server publishes.

```

76  private async Task HandleTransmission(ClientTransmission transmission)
77  {
78      var handler = _topicHandlerService.GetTopicHandler(transmission.Topic);
79
80      if (handler == null)
81          throw new ApplicationException($"No handler found for topic: {transmission.Topic}");
82
83      handler.Invoke(transmission.Data);
84  }

```

ReceiveRequest has been renamed to ReceiveTransmission and it returns a ClientTransmission object instead of a ServerRequest object since the router is going to send ClientTransmissions to the clients, otherwise the underlying logic behind receiving requests remains the same.

```
86  private async Task<ClientTransmission?> ReceiveTransmission()  
87  {  
88      byte[] buffer = new byte[1024];  
89  
90      int transmissionLength = await _router.ReceiveAsync(buffer);  
91  
92      string transmissionJson = Encoding.UTF8.GetString(buffer, 0, transmissionLength);  
93  
94      var transmission = _jsonService.DeserializeJson<ClientTransmission>(transmissionJson);  
95  
96      return transmission;  
97  }
```

The topic handler service for the client is also identical to the one used for the server, it will be used for handling publishes. The topic handlers will also be methods in ‘controllers’.

Since I am building the authentication functionality throughout the app, I also want to make the router listen for AuthenticateUserResults and store client connections against their userIDs. For this, instead of simply using an if statement to check for a response of topic “AuthenticateUserResult” I decided to build a system that could execute a predefined piece of code every time a response of a certain topic is received by the router. This is analogous to the TopicHandlerService in the server and client applications. Because it makes more sense, I called the handlers for each topic ‘interceptors’ as they intercept transmissions being sent from the server to the clients. These interceptors will be methods inside of controllers – I could not think of a better name for that.

The Interceptor attribute is shown below, it is literally the same as the Handler attribute but with a different name.

```
3  public class InterceptorAttribute : Attribute  
4  {  
5      public string Topic { get; set; }  
6  
7      public InterceptorAttribute(string topic)  
8      {  
9          Topic = topic;  
10     }  
11 }
```

The TopicInterceptorService class is shown below.

```

8  public class TopicInterceptorService : ITopicInterceptorService
9  {
10     private readonly IDictionary<string, Action<ServerResponse>> topicToInterceptor;
11
12     public TopicInterceptorService(IConfigurationService configurationService)
13     {
14         topicToInterceptor = new Dictionary<string, Action<ServerResponse>>();
15
16
17         var controllers =
18             from a in AppDomain.CurrentDomain.GetAssemblies()
19             from t in a.GetTypes()
20             let attributes = t.GetCustomAttributes(typeof(ControllerAttribute), false)
21             where attributes != null && attributes.Length > 0
22             select t;
23
24
25         foreach (var controller in controllers)
26         {
27             var constructor = controller.GetConstructors().Single();
28             var constructorParameters = constructor.GetParameters();
29
30             var arguments = new List<object>();
31
32             foreach (var parameter in constructorParameters)
33             {
34                 var argument = configurationService.GetInstanceOfType(parameter.ParameterType);
35                 arguments.Add(argument);
36             }
37
38             var controllerInstance = constructor.Invoke(arguments.ToArray());
39
40             var interceptors = controller.GetMethods()
41                 .Where(m => m.GetCustomAttributes(typeof(InterceptorAttribute), false) != null && m.GetCustomAttributes(typeof(InterceptorAttribute), false).Length > 0);
42
43             foreach (var interceptor in interceptors)
44             {
45                 topicToInterceptor.Add(interceptor.GetCustomAttribute<InterceptorAttribute>().Topic, async (x) =>
46                 {
47                     await (Task)interceptor.Invoke(controllerInstance, new object[] { x });
48                 });
49             }
50         }
51     }
52
53     public bool TryGetTopicInterceptor(string topic, out Action<ServerResponse>? interceptor)
54     {
55         var result = topicToInterceptor.TryGetValue(topic, out interceptor);
56
57         return result;
58     }
59 }

```

The interceptor for gathering the user id based off a successful authentication response is shown below.

```

21 [Interceptor("AuthenticateUserResult")]
22 public async Task InitialiseClient(ServerResponse authenticationResponse)
23 {
24     if (authenticationResponse.ServerResult.OperationResult.OperationMessage != OperationMessages.VALID_CREDENTIALS)
25         return;
26
27     if (authenticationResponse.RequestId == null)
28         throw new ArgumentNullException(nameof(authenticationResponse.RequestId));
29
30     bool requestIsValid = _requestService.TryGetRequester(authenticationResponse.RequestId.Value, out var client);
31
32     if (!requestIsValid)
33         throw new Exception($"Request, {authenticationResponse.RequestId}, not found");
34
35     if (authenticationResponse.ServerResult.AffectedUsers == null || authenticationResponse.ServerResult.AffectedUsers.Length == 0)
36         throw new Exception("AffectedUsers was null");
37
38     var userId = authenticationResponse.ServerResult.AffectedUsers[0];
39
40     _connectionService.TryAddConnection(client, userId);
41 }

```

As shown, the interceptor requires the full serverResponse to retrieve the requester the server is responding to via the requestService. This is also where connections are added to the connection service.

First, it is checked whether the authentication result is for a passed or failed authentication. If it has failed, the interceptor stops otherwise it continues. Then it is ensured that the requestID in the serverResponse is not null, if it was this would be a problem so the interceptor will throw an error (this is not the best practice but it is useful for debugging since the server should not be sending a serverResponse with no requestID in the first place). After retrieving the requester, it is stored against the userID (which is fetched via the affectedUsers in the serverResponse). I chose to use the AffectedUsers to retrieve the userID however I should have used the data inside the response to get it, I used the AffectedUsers so that I do not have to change this interceptor whenever I make a change to the data model that the server returns when authenticating a user.

The interceptor was written inside a UserController in the router, the constructor for which is shown below.

```
9      [Controller]
10     public class UserController
11     {
12         private readonly IRequestService _requestService;
13         private readonly IConnectionService _connectionService;
14
15         public UserController(IRequestService requestService, IConnectionService connectionService)
16         {
17             _requestService = requestService;
18             _connectionService = connectionService;
19         }

```

I decided that, to separate logic and make the code more

maintainable, the routerService should not handle the storage of user connections, only the server connection since there is only ever going to be a single server connection. So, instead of having a dictionary userConnections (as mentioned in the design), I made a class called ConnectionService which handles the storage of user connections. I did the same thing to the requests dictionary: instead of generating request ids and maintaining requests in the router service, I made another class called RequestService which handles the creation and maintenance of the requests.

The connection service code is shown below.

```
6  public class ConnectionService : IConnectionService
7  {
8      private ConcurrentDictionary<Guid, Socket> _userIdToHandler;
9
10     public ConnectionService()
11     {
12         _userIdToHandler = new ConcurrentDictionary<Guid, Socket>();
13     }
14
15     public bool TryAddConnection(Socket handler, Guid userId)
16     {
17         return _userIdToHandler.TryAdd(userId, handler);
18     }
19
20     public bool TryGetConnectionHandler(Guid requestId, out Socket? handler)
21     {
22         return _userIdToHandler.TryGetValue(requestId, out handler);
23     }
24 }
25
26
27 5  public interface IConnectionService
28  {
29     public bool TryAddConnection(Socket handler, Guid userId);
30
31     public bool TryGetConnectionHandler(Guid requestId, out Socket? handler);
32 }
33
```

It provides methods to add a connection and get connection handlers (in this case the sockets). It works by maintaining a dictionary.

I made it inherit an interface `IConnectionService` so that, if in the future, I want to implement this a different way I can create another class, and I do not have to modify this one.

The interface is shown to the left.

You can only add a connection if the `userId` associated with it is not associated with a preexisting connection. You can only get a connection by a `userId` (which I believe I misspelled as 'requestID') if the `userId` has been stored, obviously.

The method `TryAddConnection` attempts to store the given connection, if it succeeds, it returns `true` otherwise it returns `false`.

The method `TryGetConnectionHandler` attempts to get a connection handler by the given `userId`. If the given `userId` has a connection associated with it, the method returns `true`, and the `out` parameter will be populated with the connection handler. Otherwise, it returns `false`, and the `out` parameter will become (or stay) `null`.

The request service is shown below.

```

6  public class RequestService : IRequestService
7  {
8      private ConcurrentDictionary<Guid, Socket> _requestIdToRequester;
9
10     public RequestService()
11     {
12         _requestIdToRequester = new ConcurrentDictionary<Guid, Socket>();
13     }
14     public Guid AddRequest(Socket requester)
15     {
16         var requestId = Guid.NewGuid();
17         _requestIdToRequester[requestId] = requester;
18
19         return requestId;
20     }
21
22     public bool TryAddRequest(Socket requester, Guid requestId)
23     {
24         return _requestIdToRequester.TryAdd(requestId, requester);
25     }
26
27     public bool TryGetRequester(Guid requestId, out Socket? requester)
28     {
29         return _requestIdToRequester.TryGetValue(requestId, out requester);
30     }
31 }

```

It inherits from an interface for the same reason as the connection service.

It provides method to add requests and to get the requester of a request (the connection handler for the client that made the request).

The interface IRequestService is shown below.

```

5  public interface IRequestService
6  {
7      public bool TryAddRequest(Socket requester, Guid requestId);
8
9      public Guid AddRequest(Socket requester);
10
11     public bool TryGetRequester(Guid requestId, out Socket? requester);
12 }

```

I included a method to attempt to add a request by a pre-generated requestId since I thought it might be useful. It was not useful and should be deleted.

The method AddRequest generates a new requestId and stores the given requester connection handler against it. It returns the newly generated requestId.

The method TryGetRequester attempts to get a requester's connection handler by the given requestId. If the given requestId has a connection associated with it, the method returns true, and the out parameter will be populated with the connection handler. Otherwise, it returns false, and the out parameter will become (or stay) null.

Now, the router service. Its constructor is shown below with its new attributes.

```

14 public class RouterService
15 {
16     private IPEndPoint _iEndPoint;
17     private Socket _server;
18
19     private IRequestService _requestService;
20     private IConnectionService _connectionService;
21     private ITopicInterceptorService _topicInterceptorService;
22
23     private readonly IJsonService _jsonService;
24
25     public RouterService(IPEndPoint iEndPoint, ITopicInterceptorService topicInterceptorService, IConnectionService connectionService, IRequestService requestService, IJsonService jsonService)
26     {
27         _iEndPoint = iEndPoint;
28
29         _topicInterceptorService = topicInterceptorService;
30         _connectionService = connectionService;
31         _requestService = requestService;
32
33         _jsonService = jsonService;
34 }

```

The purpose of all these attributes has already been mentioned so I will not go into them any further.

The run method has remained the same and is shown below.

```
36  public async Task Run()
37  {
38      Console.WriteLine("Broker running...");
39      var listener = new Socket(_ipEndPoint.AddressFamily, SocketType.Stream, ProtocolType.Tcp);
40      listener.Bind(_ipEndPoint);
41
42      listener.Listen(100);
43
44      Task.Run(() =>
45      {
46          while (true)
47          {
48              var handler = listener.Accept();
49
50              if (!handler.Connected)
51              {
52                  Console.WriteLine("Connection attempt failed...");
53                  continue;
54              }
55
56              Console.WriteLine("Connection attempt successful, new client connected...");
57
58              NewConnection(handler);
59          }
60      });
61
62      Console.ReadLine();
63  }
```

The NewConnection method only has a single change on line 69 which was explained previously when debugging a logic error.

```
65  public async Task NewConnection(Socket handler)
66  {
67      Console.WriteLine("Handling new connection...");
68
69      string message = _jsonService.DeserialiseJson<string>(await ReceiveTransmission(handler));
70
71      switch (message)
72      {
73          case "SERVER":
74              ServerConnection(handler);
75              break;
76          default:
77              NewClientConnection(handler);
78              break;
79      }
80      Console.WriteLine("Connection handled...");
81  }
```

The NewClientConnection method is shown below. Now, every time the client sends a request, the requestService is used to generate a new requestId and store the request information before sending the request to the server via the SendServiceRequest method.

```
83  public async Task NewClientConnection(Socket handler)
84  {
85      if (_server == null)
86      {
87          Console.WriteLine("No server... Cannot connect client...");
88          return;
89      }
90
91      var clientTransmission = await ReceiveClientTransmission(handler);
92
93      var requestId = _requestService.AddRequest(handler);
94
95      SendServerRequest(_server, "AuthenticateUser", clientTransmission.Data, requestId);
96  }
```

The ServerConnection method is quite big, so I will split it up and explain every part of it. These sections should have been split up into their own methods, but I did not have enough time in the sprint to do that.

First the server connection handler is stored in the server attribute.

```
98  public async Task ServerConnection(Socket serverHandler)
99  {
100     _server = serverHandler;
101     Console.WriteLine("Server connected...");
102 }
```

Since I am not checking whether a server has already been connected, a malicious actor could easily set up a little script which overrides the server which is not good and something I will sort out later.

The main server loop then starts, where the router listens for and handles all the transmissions sent by the server. Every server transmission is received by the ReceiveServerTransmission method. The topicIntercetorService is then used to check for any interceptors for the topic of the transmission sent by the server, if any is found, it is invoked with the serverResponse object.

```
103  while (true)
104  {
105      ServerResponse serverResponse = await ReceiveServerTransmission();
106
107      bool topicHasInterceptor = _topicIntercetorService.TryGetTopicInterceptor(serverResponse.Topic, out var topicInterceptor);
108
109      if (topicHasInterceptor)
110      {
111          topicInterceptor.Invoke(serverResponse);
112      }
```

Since the `ServerResponse` class represents both server responses and publishes in a single class (for now), the router handles it as both.

```
114         var broadcastList = serverResponse.ServerResult.AffectedUsers;
115
116         if (broadcastList == null || broadcastList.Length == 0)
117         {
118             if (serverResponse.RequestId == null)
119                 continue;
120
121             bool wasRequested = _requestService.TryGetRequester(serverResponse.RequestId.Value, out var client);
122
123             if (wasRequested)
124             {
125                 await ForwardServerResponse(serverResponse, client);
126             }
127
128             continue;
129         }
130
131         foreach (var userId in broadcastList)
132         {
133             bool isConnected = _connectionService.TryGetConnectionHandler(userId, out var client);
134
135             if (isConnected)
136             {
137                 await ForwardServerResponse(serverResponse, client);
138             }
139         }
```

First the broadcast list is set to the affected users in the server result (line 114), this is the list of users the publish has to be sent to.

If there is no broadcast list (i.e. it has a length of 0 or is null – line 116), the server transmission is treated as a response. In this case, the requester is retrieved based on the given requester (line 121). Then, the response is forwarded to the requester via the `ForwardServerResponse` method (line 125).

Otherwise, if there is a broadcast list, it is iterated through (line 131) and every `userId` in the list is checked for an associated connection handler (lines 133 to 135 inclusive). Every time an associated connection handler is found, the server response is forwarded to the client via that connection handler using the `ForwardServerResponse` method (line 137).

The methods for receiving transmissions are shown below.

```
143  ✓      public async Task<string> ReceiveTransmission(Socket handler)
144      {
145          var buffer = new byte[2048];
146
147          int transmissionBytesCount = await handler.ReceiveAsync(buffer, SocketFlags.None);
148
149          string transmission = Encoding.UTF8.GetString(buffer, 0, transmissionBytesCount);
150
151          return transmission;
152      }
153
154  ✓      public async Task<ServerResponse?> ReceiveServerTransmission()
155      {
156          var buffer = new byte[2048];
157
158          int transmissionBytesCount = await _server.ReceiveAsync(buffer, SocketFlags.None);
159
160          string transmissionJson = Encoding.UTF8.GetString(buffer, 0, transmissionBytesCount);
161          var serverResponse = _jsonService.DeserialiseJson<ServerResponse>(transmissionJson);
162
163          return serverResponse;
164      }
165
166  ✓      public async Task<ClientTransmission?> ReceiveClientTransmission(Socket client)
167      {
168          var buffer = new byte[2048];
169
170          int transmissionBytesCount = await client.ReceiveAsync(buffer, SocketFlags.None);
171
172          string transmissionJson = Encoding.UTF8.GetString(buffer, 0, transmissionBytesCount);
173          var clientTransmission = _jsonService.DeserialiseJson<ClientTransmission>(transmissionJson);
174
175          return clientTransmission;
176      }
```

ReceiveTransmission receives a transmission from a given socket and converts it to a UTF8 string, it should also deserialise the string using the jsonService but I forgot to do that.

ReceiveServerTransmission receives a transmission from the server and converts it to a ServerResponse object.

ReceiveClientTransmission receives a transmission from a given client and converts it to a ClientTransmission object.

There is a lot of redundant code here which could be fixed in numerous ways. I was running out of time for this sprint, so I did not fix that.

The methods that send transmissions are shown below.

```
178 ✓ public async Task<int> ForwardServerResponse(ServerResponse serverResponse, Socket client)
179 {
180     string responseJson = _jsonService.SerialiseObject(serverResponse);
181
182     byte[] transmissionBytes = Encoding.UTF8.GetBytes(responseJson);
183
184     return await client.SendAsync(transmissionBytes);
185 }
186
187 ✓ public async Task<int> SendServerRequest(ServerRequest serverRequest)
188 {
189     byte[] transmissionBytes = Encoding.UTF8.GetBytes(_jsonService.SerialiseObject(serverRequest));
190
191     return await _server.SendAsync(transmissionBytes);
192 }
193
194 ✓ public async Task<int> SendServerRequest(Socket handler, string topic, object data, Guid? requestId = null)
195 {
196     var serverRequest = new ServerRequest(topic, data, requestId);
197
198     byte[] transmissionBytes = Encoding.UTF8.GetBytes(_jsonService.SerialiseObject(serverRequest));
199
200     return await handler.SendAsync(transmissionBytes);
201 }
```

The ForwardServerResponse method takes a server response object and simply sends that to the given client after converting it to JSON.

The SendServerRequest method sends a ServerRequest object to the server.

There is an overload for the SendServerRequest method which takes in the topic, data and requestId of a request, converts them into a ServerRequest object and then sends it to the server.

There is also a lot of redundancy in this set of code which should be fixed. Once again, I was running low on time.

Finally, I tested the whole system, and the result is shown below.

```

C:\_Kabushak\Programming\Applications\Orion\Orion\Broker\bin\Debug\net8.0\Ori...
IP address:
192.168.0.26
Port:
50000
Running broker...
Broker running...
Connection attempt successful, new client connected...
Handling new connection...
Server connected...
Connection handled...
Connection attempt successful, new client connected...
Handling new connection...
Connection handled...

Select C:\_Kabushak\Programming\Applications\Orion\Orion\Server\bin\Debug\net...
SERVER
On host: Peri
Router IP address:
192.168.0.26
Router port:
50000
Server running...
Transmitted response to request of topic: AuthenticateUser

Select C:\_Kabushak\Programming\Applications\Orion\Orion\Client\bin\Debug\net8.0\Ori...
CLIENT
On host: Peri
Router IP address:
192.168.0.26
Router port:
50000
Enter your username:
User1
New transmission of topic: AuthenticateUserResult
  
```

IT WORKS! As shown, the client entered a username, sent a request of topic `AuthenticateUser` and received an `AuthenticateUserResult` from the server, this all happened through the router.

With multiple clients:

```

C:\_Kabushak\Programming\Applications\Orion\Orion\Server\bin\Debug\net8.0\Ori...
SERVER
On host: Peri
On IP address: 192.168.0.26 and port: 50000
Server running...
New request of topic: AuthenticateUser
Transmitted response to request of topic: AuthenticateUser
New request of topic: AuthenticateUser
Transmitted response to request of topic: AuthenticateUser

C:\_Kabushak\Programming\Applications\Orion\Orion\Client\bin\Debug\net8.0\Ori...
CLIENT
On host: Peri
On IP address: 192.168.0.26 and port: 50000
Enter your username:
User1
New transmission of topic: AuthenticateUserResult

C:\_Kabushak\Programming\Applications\Orion\Orion\Broker\bin\Debug\net8.0\Ori...
On IP address: 192.168.0.26 and port: 50000
Running broker...
Broker running...
Connection attempt successful, new client connected...
Handling new connection...
Server connected...
Connection handled...
Connection attempt successful, new client connected...
Handling new connection...
Connection handled...
Connection attempt successful, new client connected...
Handling new connection...
Connection handled...

C:\_Kabushak\Programming\Applications\Orion\Orion\Client\bin\Debug\net8.0\Ori...
CLIENT
On host: Peri
On IP address: 192.168.0.26 and port: 50000
Enter your username:
User2
New transmission of topic: AuthenticateUserResult
  
```

IT STILL WORKS!

REVIEW

The sprint produced a good foundation for communication between the applications, this will allow me to build up new features very quickly.

Since everything is still on the console and there no usability features so, the stakeholders were not shown this very early prototype.

Here is a table detailing what was expected versus what was actually accomplished.

Expected	Actual
Direct messaging	No
Database connectivity	In-memory database
Login with just username	Login with just username

Sprint 2

GOALS

For this sprint, I want to get some functionality built in. The applications can stay in the console for now (if I have time, I will make a GUI in this sprint). The functionality I want to focus on is:

- Direct Communications
 - Users adding direct communications
 - Users sending direct messages

In summary, this sprint should satisfy success criteria 2a and 2b. Since messages will also be sent, this sprint may satisfy success criteria 1a and 1b as well.

CODING

I started off by refactoring the data layer. It is not very easy to add new entities with the current system since I need to add multiple methods for every new entity in IDataLayer, DataLayer and MockDB.

```

10  public class Entity<T> : IEntity<T> where T : class
11  {
12      private IDictionary<Guid, T> _idToRecord;
13
14      public Entity()
15      {
16          _idToRecord = new ConcurrentDictionary<Guid, T>();
17      }
18
19      public async Task<IQueryable<T>> GetAllRecords()
20      {
21          return _idToRecord.Values.AsQueryable();
22      }
23
24      public async Task<T> GetRecordById(Guid id)
25      {
26          return _idToRecord[id];
27      }
28
29      public async Task<bool> DeleteRecordById(Guid id)
30      {
31          return _idToRecord.Remove(id);
32      }
33
34      public async Task<bool> UpdateRecord(Guid id, T record)
35      {
36          _idToRecord[id] = record;
37          return true;
38      }
39
40      public async Task<bool> AddRecord(Guid id, T record)
41      {
42          return _idToRecord.TryAdd(id, record);
43      }
44  }

```

I created a generic Entity class with methods to get all records, get a single record by an id, update a record and add a record. This is shown on the left. It is generic so that any entity can be represented, its use is shown later.

It inherits from an interface so that it can be mocked for tests. The interface is called IEntity and is shown below.

```

9  public interface IEntity<T> where T : class
10  {
11      public Task<IQueryable<T>> GetAllRecords();
12
13      public Task<T> GetRecordById(Guid id);
14
15      public Task<bool> DeleteRecordById(Guid id);
16
17      public Task<bool> UpdateRecord(Guid id, T record);
18
19      public Task<bool> AddRecord(Guid id, T record);
20  }

```

The Entity class is what will handle the querying of the database so that the repositories only have to focus on business logic and they do not have to worry communicating with a database directly.

As shown, the database is still in-memory for now.

```

12  public class Database
13  {
14      public IEntity<User> UserEntity { get; set; }
15      public IEntity<Message> MessageEntity { get; set; }
16
17      public Database()
18      {
19          UserEntity = new Entity<User>();
20
21          MessageEntity = new Entity<Message>();
22      }
23  }

```

I removed the DataLayer class and its interface and replaced it with the Database class to the left.

There is no interface for this class since the repositories will have to use its properties and it requires an implementation to do so. To mock this class in the tests, I will inherit from it.

I mocked the Database with the MockDB to the right. I overrode the entity properties so I could replace them with a MockEntity type.

The MockEntity is simply in-memory and it will remain so throughout the development of the application since an in-memory database servers as a good mock database.

```

9  class MockDB : Database
10  {
11      public IEntity<User> UserEntity { get; set; }
12      public IEntity<Message> MessageEntity { get; set; }
13
14      public MockDB()
15      {
16          UserEntity = new MockEntity<User>();
17
18          MessageEntity = new MockEntity<Message>();
19      }
20  }

```

```

11  public class MockEntity<T> : IEntity<T> where T : class
12  {
13      private IDictionary<Guid, T> _idToRecord;
14
15      public MockEntity()
16      {
17          _idToRecord = new ConcurrentDictionary<Guid, T>();
18      }
19
20      public async Task<IQueryable<T>> GetAllRecords()
21      {
22          return _idToRecord.Values.AsQueryable();
23      }
24
25      public async Task<T> GetRecordById(Guid id)
26      {
27          return _idToRecord[id];
28      }
29
30      public async Task<bool> DeleteRecordById(Guid id)
31      {
32          return _idToRecord.Remove(id);
33      }
34
35      public async Task<bool> UpdateRecord(Guid id, T record)
36      {
37          _idToRecord[id] = record;
38          return true;
39      }
40
41      public async Task<bool> AddRecord(Guid id, T record)
42      {
43          return _idToRecord.TryAdd(id, record);
44      }
45  }

```

MockEntity is shown to the left and is, currently, exactly the same as the Entity class.

I will not go into detail into the changes I made throughout the application to accommodate this new system. I will just post the screen shots of the git hub change log.

MessageRepository:

```

5   6       public class MessageRepository : IMessageRepository
6   7       {
7   8       -       private readonly IDataLayer _dataLayer;
8   9       +       private readonly Database _database;
9   10
10  11      -       public MessageRepository(IDataLayer dataLayer)
11  12      +       public MessageRepository(Database database)
12  13      {
13  14
14  15          public async Task<Message?> AddMessage(Guid senderId, string content)
@@ -20,14 +21,14 @@ public MessageRepository(IDataLayer dataLayer)
20  21              SenderId = senderId
21  22          };
22  23
23  24      -       await _dataLayer.AddMessage(message);
24  25      +       bool wasSuccessful = await _database.MessageEntity.AddRecord(message.MessageId, message);
25  26
26  27      -       return message;
27  28      +       return wasSuccessful ? message : null;
28  29
29  30      }
30  31
31  32      public async Task<Message?> GetMessage(Guid messageId)
32  33      {
33  34      -       return await _dataLayer.GetMessage(messageId);
34  35      +       return await _database.MessageEntity.GetRecordById(messageId);
35  36
36  37      }
37  38
38  39      }
39  40
40  41      }

```

UserRepository:

```

7 8      public class UserRepository : IUserRepository
8 9      {
9      -      private readonly IDatalayer _datalayer;
10 +      private readonly Database _database;
10 11
11 -      public UserRepository(IDatalayer datalayer)
12 +      public UserRepository(Database database)
12 13      {
13 -      _datalayer = datalayer;
14 +      _database = database;
14 15      }
15 16
16 17      public async Task<ServerResult?> AuthenticateUser(string username)
17 18      {
18      -      var users = await _datalayer.GetAllUsers();
19      -
20      -      var user = users?
21      -          .Select(x => x)
22      -          .Where(x => x.Username == username)
23      -          .SingleOrDefault();
19 +      var user = (await _database.UserEntity.GetAllRecords())
20 +          .Select(x => x)
21 +          .Where(x => x.Username == username)
22 +          .SingleOrDefault();
24 23
25 24      return (user == null)
26 25          ? ServerResultService.NewServerResult(OperationMessages.INVALID_CREDENTIALS)
27 26      @@ -35,14 +34,14 @@ public UserRepository(IDatalayer datalayer)
35 34          Username = username
36 35      };
37 36
38 -      await _datalayer.AddUser(user);
37 +      await _database.UserEntity.AddRecord(user.UserId, user);
39 38
40 39      return user;
41 40      }
42 41
43 42      public async Task<User?> GetUser(Guid userId)
44 43      {
45 -      return await _datalayer.GetUser(userId);
44 +      return await _database.UserEntity.GetRecordById(userId);
46 45      }

```

MessageRepositoryTests:

```

8      8      public class MessageRepositoryTests
9      9      {
10     -      private IDatalayer _datalayer;
10     +      private Database _mockDB;
11     11      private IMessageRepository _messageRepository;
12     12
13     13      [SetUp]
14     14      public void Setup()
15     15      {
16     -      _datalayer = new MockDB();
17     -      _messageRepository = new MessageRepository(_datalayer);
16     +      _mockDB = new MockDB();
17     +      _messageRepository = new MessageRepository(_mockDB);
18     18      }
19     19
20     20      [Test]
21     21      public async Task AddMessageAddsMessageToDB()
22     22      {
23     -      Message message = await _messageRepository.AddMessage(new Guid(), "Message1");
23     +      Message message = await _messageRepository.AddMessage(Guid.NewGuid(), "Message1");
24     24
25     -      Message result = await _datalayer.GetMessage(message.MessageId);
25     +      Message result = await _mockDB.MessageEntity.GetRecordById(message.MessageId);
26     26      Assert.That(result?.Content == "Message1", $"Content was: {result?.Content}");
27     27      }
28     28
29     @@ -32,12 +32,12 @@ public async Task GetMessageGetsMessageFromDB()
30     30      {
31     31      var message = new Message()
32     32      {
33     -      MessageId = new Guid(),
34     -      SenderId = new Guid(),
35     +      MessageId = Guid.NewGuid(),
36     +      SenderId = Guid.NewGuid(),
37     37      Content = "Message1"
38     38      };
39     39
40     -      await _datalayer.AddMessage(message);
40     +      await _mockDB.MessageEntity.AddRecord(message.MessageId, message);
41     41
42     42      Message result = await _messageRepository.GetMessage(message.MessageId);
43     43      Assert.That(result?.Content == "Message1", $"Message name was: {result?.Content}");

```

In the MessageRepositoryTests changes, you may notice changes to lines 23, 35 and 36 where I replaced new Guid() with Guid.NewGuid(). This was because I needed to generate new ids but using new Guid() did not generate a new id, it just made another Guid with the default value.

UserRepositoryTests:

```

9 9      public class UserRepositoryTests
10 10     {
11 11         - private IDatalayer _datalayer;
11 11         + private Database _mockDB;
12 12         private IUserRepository _userRepository;
13 13
14 14         [SetUp]
15 15         public void Setup()
16 16         {
17 17         -         _datalayer = new MockDB();
18 18         -         _userRepository = new UserRepository(_datalayer);
17 17         +         _mockDB = new MockDB();
18 18         +         _userRepository = new UserRepository(_mockDB);
19 19     }

24 24         [TestCase("User212343441")]
25 25         public async Task AuthenticateUser_ReturnsTrueForValidUsername(string username)
26 26         {
27 27         -         await _datalayer.AddUser(new User())
27 27         +         var newUser = new User()
28 28         {
29 29             UserId = Guid.NewGuid(),
30 30             Username = username
31 31         -         });
31 31         +         });
32 32         +         await _mockDB.UserEntity.AddRecord(newUser.UserId, newUser);
32 34
33 35         ServerResult result = await _userRepository.AuthenticateUser(username);
34 36
35 36         @@ -41,7 +43,7 @@ await _datalayer.AddUser(new User())
41 43         [TestCase("User212343441")]
42 44         public async Task AuthenticateUser_ReturnsFalseForInvalidUsername(string username)
43 45         {
44 46         -         Assert.That(await _datalayer.GetAllUsers().Count == 0, "Invalid test conditions");
46 46         +         Assert.That(await _mockDB.UserEntity.GetAllRecords().ToList().Count == 0, "Invalid test conditions");
45 47
46 48         ServerResult result = await _userRepository.AuthenticateUser(username);
47 49
48 49         @@ -52,7 +54,7 @@ public async Task AddUserAddsUserToDB()
52 54         {
53 55         User user = await _userRepository.AddUser("User1");
54 56
55 56         -         User result = await _datalayer.GetUser(user.UserId);
57 57         +         User result = await _mockDB.UserEntity.GetRecordById(user.UserId);
56 58         Assert.That(result?.Username == "User1", $"Username was: {result?.Username}");
57 59         }
58 60
59 60         @@ -66,7 +68,7 @@ public async Task GetUserGetsUserFromDB()
66 68         Username = "User1"
67 69         };
68 70
69 70         -         await _datalayer.AddUser(user);
71 71         +         await _mockDB.UserEntity.AddRecord(user.UserId, user);
70 72
71 73         User result = await _userRepository.GetUser(user.UserId);
72 74         Assert.That(result?.Username == "User1", $"Username was: {result?.Username}");

```

In the previous sprint I discussed how I wanted to keep one enumerator for each operation. These enumerators would contain the message codes the operation could produce. To make this, I replaced the `OperationResult` class with a generic `OperationInformation` class. This is shown below.

```

6  public class OperationInformation<T> where T : Enum
7  {
8      public Statuses Status { get; set; }
9
10     public T OperationMessageCode { get; set; }
11
12     public string? OperationMessage { get; set; }
13
14     public OperationInformation(Statuses status, T operationMessageCode, string? operationMessage = null)
15     {
16         Status = status;
17         OperationMessageCode = operationMessageCode;
18         OperationMessage = operationMessage;
19     }

```

I added a field for the operation message as well, it is just a string.

I also added a field called `Status` which shows whether the server encountered an error while performing the operation. The `Status` is represented by an enumerator called `Statuses` (shown on the right).

```

9  public enum Statuses
10 {
11     SUCCEEDED,
12     FAILED
13 }

```

To accommodate this change, I had to make the `ServerResult` class generic as well. It is shown below.

```

3  public class ServerResult<T> where T : Enum
4  {
5      public OperationInformation<T> OperationInformation { get; set; }
6
7      public object? Data { get; set; }
8
9      public Guid[]? AffectedUsers { get; set; }
10
11     public ServerResult(OperationInformation<T> operationInformation, object? data = null, Guid[]? affectedUsers = null)
12     {
13         OperationInformation = operationInformation;
14         Data = data;
15         AffectedUsers = affectedUsers;
16     }

```

I did not want to make the `ServerResponse` class generic because then I would have to change a lot of things in the router. To make this happen, I made an overload of the `OperationInformation` class that was not generic, the operation message code was stored as a string.

This is shown below.

```

35  public class OperationInformation
36  {
37      public Statuses Status { get; set; }
38
39      public string OperationMessageCode { get; set; }
40
41      public string? OperationMessage { get; set; }
42
43  public OperationInformation(Statuses status, string operationMessageCode, string? operationMessage = null)
44  {
45      Status = status;
46      OperationMessageCode = operationMessageCode;
47      OperationMessage = operationMessage;
48  }
49  }

```

I also wrote some implicit converters for the converting between the 2 overloads since I would need to convert between them a lot so it would clean up the code. These are shown below.

```

21  public static implicit operator OperationInformation<T>(OperationInformation value)
22  {
23      object result;
24
25      if (!Enum.TryParse(typeof(T), value.OperationMessageCode, out result))
26          throw new Exception($"{value.OperationMessageCode} is not a value represented by enumeration {nameof(T)}");
27
28      return new OperationInformation<T>(value.Status, (T)result, value.OperationMessage);
29  }
30
31  public static implicit operator OperationInformation(OperationInformation<T> value) =>
32      new OperationInformation(value.Status, value.OperationMessageCode.ToString(), value.OperationMessage);

```

I also made an overload for the ServerResult class that was not generic, which stored its operation information as the non-generic overload of OperationInformation. This is shown below.

```

22  public class ServerResult
23  {
24      public OperationInformation OperationInformation { get; set; }
25
26      public object? Data { get; set; }
27
28      public Guid[]? AffectedUsers { get; set; }
29
30  public ServerResult(OperationInformation operationInformation, object? data = null, Guid[]? affectedUsers = null)
31  {
32      OperationInformation = operationInformation;
33      Data = data;
34      AffectedUsers = affectedUsers;
35  }
36  }

```

I also wrote an implicit converter for converting the generic ServerResult to a non-generic ServerResult. This is shown below.

```

18  public static implicit operator ServerResult(ServerResult<T> value) =>
19      new ServerResult(value.OperationInformation, value.Data, value.AffectedUsers);

```

I did not write one for going the other way because I did not want to accidentally convert the non-generic ServerResult to a generic ServerResult.

Due to all of this, I had to change the ServerResultService. The changes are shown below.

```

5 5      public static class ServerResultService
6 6      {
7 7          - public static ServerResult NewServerResult(OperationMessages operationMessage, Guid[]? affectedUsers = null, object? data = null)
8 8          + public static ServerResult<T> NewServerResult<T>(Statuses status, T operationMessageCode, string? operationMessage = null, Guid[]? affectedUsers = null, object? data = null) where T : Enum
9 9          {
10 10             - return new ServerResult(new OperationResult(operationMessage), data, affectedUsers);
11 11             + return new ServerResult<T>(new OperationInformation<T>(status, operationMessageCode, operationMessage), data, affectedUsers);
12 12         }
13 13         - public static ServerResult NewServerResult(OperationMessages operationMessage, Guid affectedUser, object? data = null)
14 14         + public static ServerResult<T> NewServerResult<T>(Statuses status, T operationMessageCode, Guid affectedUser, string? operationMessage = null, object? data = null) where T : Enum =>
15 15             NewServerResult(status, operationMessageCode, operationMessage, [ affectedUser ], data);
16 16         + public static ServerResult NewServerResult(Statuses status, Enum operationMessageCode, string? operationMessage = null, Guid[]? affectedUsers = null, object? data = null)
17 17         {
18 18             - return new ServerResult(new OperationResult(operationMessage), data, [ affectedUser ]);
19 19             + return new ServerResult(new OperationInformation(status, operationMessageCode.ToString(), operationMessage), data, affectedUsers);
20 20         }
21 21         + public static ServerResult NewServerResult(Statuses status, Enum operationMessageCode, Guid affectedUser, string? operationMessage = null, object? data = null) =>
22 22             NewServerResult(status, operationMessageCode, operationMessage, [ affectedUser ], data);
23 23         + public static ServerResult<T> NewSuccessfulServerResult<T>(T operationMessageCode, string? operationMessage = null, Guid[]? affectedUsers = null, object? data = null) where T : Enum =>
24 24             NewServerResult(Statuses.SUCCEEDED, operationMessageCode, operationMessage, affectedUsers, data);
25 25         + public static ServerResult<T> NewSuccessfulServerResult<T>(T operationMessageCode, Guid affectedUser, string? operationMessage = null, object? data = null) where T : Enum =>
26 26             NewServerResult(Statuses.SUCCEEDED, operationMessageCode, affectedUser, operationMessage, data);
27 27         + public static ServerResult NewSuccessfulServerResult(Enum operationMessageCode, string? operationMessage = null, Guid[]? affectedUsers = null, object? data = null) =>
28 28             NewServerResult(Statuses.SUCCEEDED, operationMessageCode, operationMessage, affectedUsers, data);
29 29         + public static ServerResult NewSuccessfulServerResult(Enum operationMessageCode, Guid affectedUser, string? operationMessage = null, object? data = null) =>
30 30             NewServerResult(Statuses.SUCCEEDED, operationMessageCode, affectedUser, operationMessage, data);
31 31         + public static ServerResult NewSuccessfulServerResult(Enum operationMessageCode, Guid affectedUser, string? operationMessage = null, object? data = null) =>
32 32             NewServerResult(Statuses.SUCCEEDED, operationMessageCode, affectedUser, operationMessage, data);

```

```

10 10      public enum AuthenticationMessages
11 11      {
12 12          VALID_CREDENTIALS,
13 13          INVALID_CREDENTIALS,
14 14          USER_ALREADY_SIGNED_IN
15 15      }

```

Finally, I created a new enumerator for AuthenticationMessages, it is shown on the right.

I then changed the UserRepository AuthenticateUser method to utilise this enumerator. This is shown below.

```

25 26      return (user == null)
26 26      - ? ServerResultService.NewServerResult(OperationMessages.INVALID_CREDENTIALS)
27 27      - : ServerResultService.NewServerResult(OperationMessages.VALID_CREDENTIALS, user.UserId, user);
27 27      + ? ServerResultService.NewSuccessfulServerResult(AuthenticationMessages.INVALID_CREDENTIALS, "Invalid credentials given")
28 28      + : ServerResultService.NewSuccessfulServerResult(AuthenticationMessages.VALID_CREDENTIALS, user.UserId, "Valid credentials", user);

```

As you can see, I also wrote more detailed messages.

After integrating this new database system, I created new classes for the direct communications.

On the right is shown the data model for direct communications. I initially thought it would be a good idea to store the members of the direct communication in a tuple of length 2, since there can only ever be 2 members in a direct communication.

```

10 10      public class DirectCommunication
11 11      {
12 12          public Guid DirectCommunicationId { get; set; }
13 13
14 14          public Tuple<Guid, Guid> UserIds { get; set; }
15 15
16 16          public List<Guid>? DirectMessageIds { get; set; }
17 17      }

```

```

9 9      public class DirectMessage
10 10      {
11 11          public Guid DirectMessageId { get; set; }
12 12
13 13          public Guid DirectCommunicationId { get; set; }
14 14
15 15          public Guid MessageId { get; set; }
16 16      }

```

The direct message class shown on the left represents the intermediary between messages and their direct communications.

This was to keep direct communication messages and channel messages separate.

I added a property to the User data model to reflect a user's direct communications. This is shown on the right. I am storing the unique ids of all the entities rather than instances of them. This is because it is easier to work with.

```

@@ -5,5 +5,7 @@ public class User
5      5      public Guid UserId { get; set; }
6      6
7      7      public string Username { get; set; }
8      8      +
9      9      +      public List<Guid> DirectCommunicationIds { get; set; }
10     10     }

```

The last model I added was one which would be the data for a new direct communication request. Since the user portal will be sending these requests, all they will use is the username of the recipient (the other member of the direct communication) and the userID of the sender (the user logged into the user portal making the request).

```

9      public class NewDirectCommunication
10     {
11         public Guid SenderId { get; set; }
12         public string ReceiverName { get; set; }
13     }

```

Although, as specified in the design, I should be sending 2 requests, one to check if the recipient exists and another to request the addition of the direct communication. I will only send one for now.

I also created a new Credentials model and updated all the authentication methods to use that over just using a plain string username. It is shown to the right.

```

9      public class Credentials
10     {
11         public string Username { get; set; }
12     }

```

Now that I have the models set up, I can start work on the operations. First, I want to focus on the adding of direct communications.

```

9      public enum DirectCommunicationMessages
10     {
11         DIRECT_COMMUNICATION_CREATION_SUCCEEDED,
12         DIRECT_COMMUNICATION_CREATION_FAILED
13     }

```

To start off, I added another enumerator for adding direct communications which I called DirectCommunicationMessages. This is shown on the left.

The method that creates a direct communication will need to do 3 things: it adds the direct communication to the database with the correct information; it updates the users that are members of the new direct communication; and it returns the newly created direct communication.

All the tests for the direct communication repository (there is now another repository for direct communications) will be contained in the DirectCommunicationRepositoryTests class. The definition of the private properties and the implementation of the SetUp method for this class is shown below.

```

13  ✓    public class DirectCommunicationRepositoryTests
14      {
15          private Database _mockDB;
16          private IDirectCommunicationRepository _directCommunicationRepository;
17
18          [SetUp]
19  ✓    public void Setup()
20      {
21          _mockDB = new MockDB();
22          _directCommunicationRepository = new DirectCommunicationRepository(_mockDB);
23      }

```

The test for adding the direct communication to the database is shown below.

```

142      [Test]
143  ✓    public async Task AddDirectCommunication_AddsDirectCommunicationToDBWithCorrectAttributes()
144      {
145          var sender = new User() { UserId = Guid.NewGuid() };
146          var receiver = new User() { UserId = Guid.NewGuid() };
147
148          await _mockDB.UserEntity.AddRecord(sender.UserId, sender);
149          await _mockDB.UserEntity.AddRecord(receiver.UserId, receiver);
150
151          var newDirectCommunication = new NewDirectCommunication() { SenderId = sender.UserId, ReceiverName = receiver.Username };
152
153          ServerResult? result = await _directCommunicationRepository.AddDirectCommunication(newDirectCommunication);
154
155          DirectCommunication directCommunication = result.Data as DirectCommunication;
156
157          DirectCommunication? storedDirectCommunication = await _mockDB.DirectCommunicationEntity.GetRecordById(directCommunication.DirectCommunicationId);
158
159          Assert.That(storedDirectCommunication.UserIds.Item1 == sender.UserId && storedDirectCommunication.UserIds.Item2 == receiver.UserId,
160                      "Wrong DirectCommunication stored...");
161      }

```

The test for returning the correct direct communication is shown below.

```

115      [Test]
116  ✓    public async Task AddDirectCommunication_ReturnsValidDirectCommunication()
117      {
118          var sender = new User() { UserId = Guid.NewGuid() };
119          var receiver = new User() { UserId = Guid.NewGuid(), Username = "User" };
120
121          await _mockDB.UserEntity.AddRecord(sender.UserId, sender);
122          await _mockDB.UserEntity.AddRecord(receiver.UserId, receiver);
123
124          var newDirectCommunication = new NewDirectCommunication() { SenderId = sender.UserId, ReceiverName = receiver.Username };
125
126          ServerResult? result = await _directCommunicationRepository.AddDirectCommunication(newDirectCommunication);
127
128          Assert.IsNotNull(result, "Result was null");
129
130          DirectCommunication? directCommunication = (DirectCommunication?)result.Data;
131
132          Assert.IsNotNull(directCommunication, "DirectCommunication was null");
133
134          DirectCommunication? storedDirectCommunication = await _mockDB.DirectCommunicationEntity.GetRecordById(directCommunication.DirectCommunicationId);
135
136          Assert.That(directCommunication.DirectCommunicationId == storedDirectCommunication.DirectCommunicationId
137                      && directCommunication.UserIds.Item1 == sender.UserId
138                      && directCommunication.UserIds.Item2 == receiver.UserId,
139                      "Wrong DirectCommunication returned...");
140      }

```

The test for updating the members of the direct communication is shown below.

```

163     [Test]
164     public async Task AddDirectCommunication_UpdatesUserEntities()
165     {
166         var sender = new User() { UserId = Guid.NewGuid() };
167         var receiver = new User() { UserId = Guid.NewGuid() };
168
169         await _mockDB.UserEntity.AddRecord(sender.UserId, sender);
170         await _mockDB.UserEntity.AddRecord(receiver.UserId, receiver);
171
172         var newDirectCommunication = new NewDirectCommunication() { SenderId = sender.UserId, ReceiverName = receiver.Username };
173
174         ServerResult? result = await _directCommunicationRepository.AddDirectCommunication(newDirectCommunication);
175
176         DirectCommunication directCommunication = result.Data as DirectCommunication;
177
178         User? storedSender = await _mockDB.UserEntity.GetRecordById(sender.UserId);
179
180         Assert.IsNotNull(storedSender, "Stored sender was null");
181
182         Assert.That(storedSender.DirectCommunicationIds.Contains(directCommunication.DirectCommunicationId), "Sender did not contain directCommunication");
183
184         User? storedReceiver = await _mockDB.UserEntity.GetRecordById(receiver.UserId);
185
186         Assert.IsNotNull(storedReceiver, "Stored receiver was null");
187
188         Assert.That(storedReceiver.DirectCommunicationIds.Contains(directCommunication.DirectCommunicationId), "Receiver did not contain directCommunication");
189     }

```

I then added an AddDirectCommunication method into the direct communication repository interface. This is shown below.

```

13     public interface IDirectCommunicationRepository
14     {
15         public Task<ServerResult<DirectCommunicationMessages>>> AddDirectCommunication(NewDirectCommunication newDirectCommunication);
16     }

```

The implementation of this method is shown below. It is quite big, so it has been split up and explained.

First, the recipient is found with the given receiver's username. The id of the receiver is then stored in receiverId. If the user id is null, the user was not found in the database so the wasSuccessful variable is immediately set to false otherwise its set to true.

```

26     public async Task<ServerResult<DirectCommunicationMessages>>> AddDirectCommunication(NewDirectCommunication newDirectCommunication)
27     {
28         var receiverId = (await _database.UserEntity.GetAllRecords())
29             .Where(x => x.Username == newDirectCommunication.ReceiverName)
30             .Single()
31             .UserId;
32
33         bool wasSuccessful = receiverId != null;

```

Then, a new direct communication is made containing the user ids. This is then added to the database. If the receiver was not found, the method should not have continued but I forgot to add that. Additionally, if the user was not found the Single method in line 30 would have thrown an error.

```

35         var directCommunication = new DirectCommunication()
36         {
37             DirectCommunicationId = Guid.NewGuid(),
38             UserIds = new Tuple<Guid, Guid>(newDirectCommunication.SenderId, receiverId)
39         };
40
41         wasSuccessful &= await _database.DirectCommunicationEntity.AddRecord(directCommunication.DirectCommunicationId, directCommunication);

```

The result of the record added is added to the `wasSuccessful` variable, this is because, if `wasSuccessful` was false, it should stay false otherwise it should be set to the result of the next operation.

Now, the users need to be updated. First the sender is updated:

```
43         var sender = await _database.UserEntity.GetRecordById(newDirectCommunication.SenderId);
44
45         wasSuccessful &= sender != null;
46
47         sender.DirectCommunicationIds = sender.DirectCommunicationIds == null
48             ? new List<Guid> { directCommunication.DirectCommunicationId }
49             : sender.DirectCommunicationIds.Append(directCommunication.DirectCommunicationId).ToList();
50
51         wasSuccessful &= await _database.UserEntity.UpdateRecord(sender.UserId, sender);
```

Then, the receiver is updated:

```
53         var receiver = await _database.UserEntity.GetRecordById(receiverId);
54
55         wasSuccessful &= receiver != null;
56
57         receiver.DirectCommunicationIds = receiver.DirectCommunicationIds == null
58             ? new List<Guid> { directCommunication.DirectCommunicationId }
59             : receiver.DirectCommunicationIds.Append(directCommunication.DirectCommunicationId).ToList();
60
61         wasSuccessful &= await _database.UserEntity.UpdateRecord(receiver.UserId, receiver);
```

These 2 contain a lot of redundancy and can be easily reduced by adding another method to update the direct communications of the users however, I just wanted to get this done quickly.

Then, the following is returned:

```
return wasSuccessful
    ? ServerResultService.NewSuccessfulServerResult(
        operationMessageCode: DirectCommunicationMessages.DIRECT_COMMUNICATION_CREATION_SUCCEEDED,
        affectedUsers: [newDirectCommunication.SenderId, receiver.UserId],
        data: directCommunication)
    : ServerResultService.NewSuccessfulServerResult(DirectCommunicationMessages.DIRECT_COMMUNICATION_CREATION_FAILED, "Creation of direct communication failed", [newDirectCommunication.SenderId, receiver.UserId]);
```

The affected users are always set to the members of the direct communication.

The only thing left for the server to add direct communications is the direct communication controller. It is shown below and is self-explanatory.

```
13     [Controller]
14     public class DirectCommunicationController : IDirectCommunicationController
15     {
16         private readonly IDirectCommunicationRepository _directCommunicationRepository;
17
18         public DirectCommunicationController(IDirectCommunicationRepository directCommunicationRepository)
19         {
20             _directCommunicationRepository = directCommunicationRepository;
21         }
22
23         [Handler("NewDirectCommunication")]
24         public async Task<ServerResult<DirectCommunicationMessages>> AddDirectCommunication(NewDirectCommunication newDirectCommunication)
25         {
26             return await _directCommunicationRepository.AddDirectCommunication(newDirectCommunication);
27         }
28     }
```

Now I can work on the client changes. First, the client needs to be able to receive responses for its requests. To do this I need to make the subscription service. This has been explained in the design in detail.

The subscribable class has been made generic, the type of it is the operation message code enum type.

```

5  public class Subscribable<T> where T : Enum
6  {
7      private readonly List<Action<ServerResult<T>>> _subscriptions;
8
9      public Subscribable()
10     {
11         _subscriptions = new List<Action<ServerResult<T>>>();
12     }
13
14     public Subscribable(List<Action<ServerResult<T>>> subscriptions)
15     {
16         _subscriptions = subscriptions;
17     }
18
19     public void Subscribe(Action<ServerResult<T>> subscription)
20     {
21         _subscriptions.Add(subscription);
22     }
23

```

As shown, it stores all its subscriptions in a list. It has 2 overloads in its constructor, one creates a new list while another allows me to pass in a premade subscriptions list, I thought this might be useful in the future.

The subscriptions service is quite big so I will split it up and explain it.

The constructor and attributes of the class are shown below.

```

6  public class SubscriptionService : ISubscriptionService
7  {
8      private readonly HashSet<string> _topics;
9      private readonly Dictionary<string, Type> _topicToOperationMessageCodeType;
10     private readonly Dictionary<string, Action<object>>> _topicToHandler;
11     private readonly Dictionary<string, Subscribable<Enum>> _topicToSubscribable;
12
13     public SubscriptionService()
14     {
15         _topics = new HashSet<string>();
16         _topicToOperationMessageCodeType = new Dictionary<string, Type>();
17         _topicToHandler = new Dictionary<string, Action<object>>>();
18         _topicToSubscribable = new Dictionary<string, Subscribable<Enum>>();
19     }

```

I use a hash set to store a unique collection of all the topics the subscription service has subscribable for, this is so I can quickly lookup which topics have subscribables and which don't.

I have 3 dictionaries, one which stores each topic against the type of operation message code it will return. Another dictionary stores each topic against a delegate that should be invoked when that

topic is received by the client, this delegate will simply wrap the `Subscribable.Publish` method. The last dictionary stores the actual subscribables against topics.

The `GetOrCreateSbscribableForTopic` method is shown below.

```

21  public Subscribable<T> GetOrCreateSubscribableForTopic<T>(string topic) where T : Enum
22  {
23      if(!_topics.Contains(topic))
24          return _topicToSubscribable[topic];
25
26      var subscribable = new Subscribable<T>();
27
28      _topics.Add(topic);
29      _topicToOperationMessageCodeType[topic] = typeof(T);
30      _topicToSubscribable[topic] = subscribable;
31      _topicToHandler[topic] = data =>
32      {
33          ServerResult baseResult = (ServerResult)data;
34          ServerResult<T> result = (ServerResult<T>)baseResult;
35          subscribable.Publish(result);
36      };
37
38      return subscribable;
39  }

```

It is generic so that the correct type of subscribable can be returned. First the hashset is used to see if the topic already has a subscribable. If it does, the subscribable for that topic is retrieved and returned. Otherwise, a new subscribable is created, all the dictionaries are updated accordingly and the subscribable is returned. Note lines 31 to 36 (inclusive) where the data is converted to a `ServerResult` of the same type as the subscribable.

The `TryPublishDataForTopic` is shown below.

```

41  public bool TryPublishDataForTopic(string topic, object? data)
42  {
43      if(!_topics.Contains(topic))
44          return false;
45
46      _topicToHandler[topic].Invoke(data);
47      return true;
48  }

```

This returns true if there was a subscribable for the given topic, otherwise it returns false. If a subscribable for the given topic was found, it invokes the handler for that topic.

I also decided to create a mini transmission service and connection service in the client because I wanted to.

The transmission service only provides the function `TransmitDataOfTopic`.

```

11  public class TransmissionService : ITransmissionService
12  {
13      private readonly IConnectionService _connectionService;
14      private readonly IJsonService _jsonService;
15      private readonly ISubscriptionService _subscriptionService;
16
17      public TransmissionService(IConnectionService connectionService, IJsonService jsonService, ISubscriptionService subscriptionService)
18      {
19          _connectionService = connectionService;
20          _jsonService = jsonService;
21          _subscriptionService = subscriptionService;
22      }
23
24      public async Task<Subscribable<T>> TransmitDataOfTopic<T>(object? data, string topic) where T : Enum
25      {
26          var transmission = new ClientTransmission(topic, data);
27
28          var transmissionJson = _jsonService.SerialiseObject(transmission);
29
30          var transmissionBytes = Encoding.UTF8.GetBytes(transmissionJson);
31
32          var subscribable = _subscriptionService.GetOrCreateSubscribableForTopic<T>(topic + "Result");
33
34          _connectionService.SendMessage(transmissionBytes);
35
36          return subscribable;
37      }
38  }

```

Since this is how all transmissions are going to be sent in the client (for now) and every transmission sent by the client is a request, a new subscribable is created for every transmission.

This transmission service uses a connection service which is just the `clientService` renamed. This strays far from what was decided in the design but, for now, it gets the job done. To let the transmission service send an array of bytes, I added another method called `SendMessage` which is shown below.

```

92  public async Task<bool> SendMessage(byte[] data)
93  {
94      int sentBytes = await _router.SendAsync(data);
95
96      return sentBytes == data.Length;
97  }

```

To send the requests, I wrote user and direct communication services. These services just encapsulate which topic to send the request with.

The user service is shown below.

```

8  public class UserService : IUserService
9  {
10     private readonly ITransmissionService _transmissionService;
11
12     public UserService(ITransmissionService transmissionService)
13     {
14         _transmissionService = transmissionService;
15     }
16
17     public async Task<Subscribable<AuthenticationMessages>> AuthenticateUser(Credentials credentials)
18     {
19         return await _transmissionService.TransmitDataOfTopic<AuthenticationMessages>(credentials, "AuthenticateUser");
20     }
21 }

```

The direct communication service is shown below.

```

8  public class DirectCommunicationService : IDirectCommunicationService
9  {
10     private readonly ITransmissionService _transmissionService;
11
12     public DirectCommunicationService(ITransmissionService transmissionService)
13     {
14         _transmissionService = transmissionService;
15     }
16
17     public async Task<Subscribable<DirectCommunicationMessages>> NewDirectCommunication(NewDirectCommunication newDirectCommunication)
18     {
19         return await _transmissionService.TransmitDataOfTopic<DirectCommunicationMessages>(newDirectCommunication, "NewDirectCommunication");
20     }
21 }

```

The actual execution of the client application will now take place in the Application class. Since this is a console program (currently), I decided I needed a way to store the options a user can choose from. The users options will change as they do different things i.e. once a user is signed in they cannot sign in again.

The constructor for the application is shown below. Options are numbered and stored in a dictionary because I might want to letter them instead in the future. The tuples store the display content of each option against the action to take when that option has been chosen. The application will also need to use the user and direct communication services.

```

15 public class Application
16 {
17
18     private Dictionary<int, Tuple<string, Action>> _availableOptions;
19
20     private readonly IUserService _userService;
21     private readonly IDirectCommunicationService _directCommunicationService;
22
23     private User sender;
24
25     public Application(IUserService userService, IDirectCommunicationService directCommunicationService)
26     {
27         _availableOptions = new Dictionary<int, Tuple<string, Action>>();
28         _userService = userService;
29         _directCommunicationService = directCommunicationService;
30     }

```

In the application run method (which will be called in the program to run the app), the options are populated.

The first option is the ability for the user to login, when selected it asks the user to enter their username then sends an AuthenticateUser request to the server.

```

34     _availableOptions[0] = new Tuple<string, Action>("Login", async () =>
35     {
36         Console.WriteLine("Enter username: ");
37         string username = Console.ReadLine();
38
39         var subscribable = await _userService.AuthenticateUser(new Credentials() { Username = username });
40         subscribable.Subscribe(result =>
41         {
42             Console.WriteLine(result.OperationInformation.OperationMessage);
43             if (result.OperationInformation.OperationMessageCode != AuthenticationMessages.VALID_CREDENTIALS)
44                 return;
45
46             sender = result.Data as User;
47
48             _availableOptions.Add(2, new Tuple<string, Action>("NewDirectCommunication", async () =>
49             {
50                 Console.WriteLine("Receiver Name: ");
51                 string name = Console.ReadLine();
52
53                 var directCommunication = new NewDirectCommunication()
54                 {
55                     SenderId = sender.UserId,
56                     ReceiverName = name
57                 };
58
59                 var subscribable = await _directCommunicationService.NewDirectCommunication(directCommunication);
60                 subscribable.Subscribe(result =>
61                 {
62                     if (result.OperationInformation.OperationMessageCode != DirectCommunicationMessages.DIRECT_COMMUNICATION_CREATION_SUCCEEDED)
63                         return;
64
65                 });
66             });
67         });
68     });
69 });

```

It then subscribes to the response for this, when received, if the response states that invalid credentials were entered, it doesn't do anything otherwise it stores the given user (the server sends the user's information in a User object) in sender and then adds another option to the options that allows a user to add a direct communication.

In this second option, the user is asked to enter the other user's name and a request is sent to the server to add that direct communication.

The last option allows the user to quit the application, it is shown below.

```

71     _availableOptions[1] = new Tuple<string, Action>("Quit", () => Environment.Exit(0));

```

After the options are defined, the main-loop of the application begins where it keeps asking the user to choose an option until they choose to quit.

```

73         while (true)
74         {
75             Console.WriteLine("Pick an option: ");
76
77             for (int i = 0; i < _availableOptions.Count; i++)
78             {
79                 Console.WriteLine($"{i}. {_availableOptions[i].Item1}");
80             }
81
82             bool validOptionSelected = int.TryParse(Console.ReadLine(), out var option);
83
84             if (!validOptionSelected)
85                 continue;
86
87             _availableOptions[option].Item2.Invoke();
88         }

```

The Program.cs file in the client app is shown below.

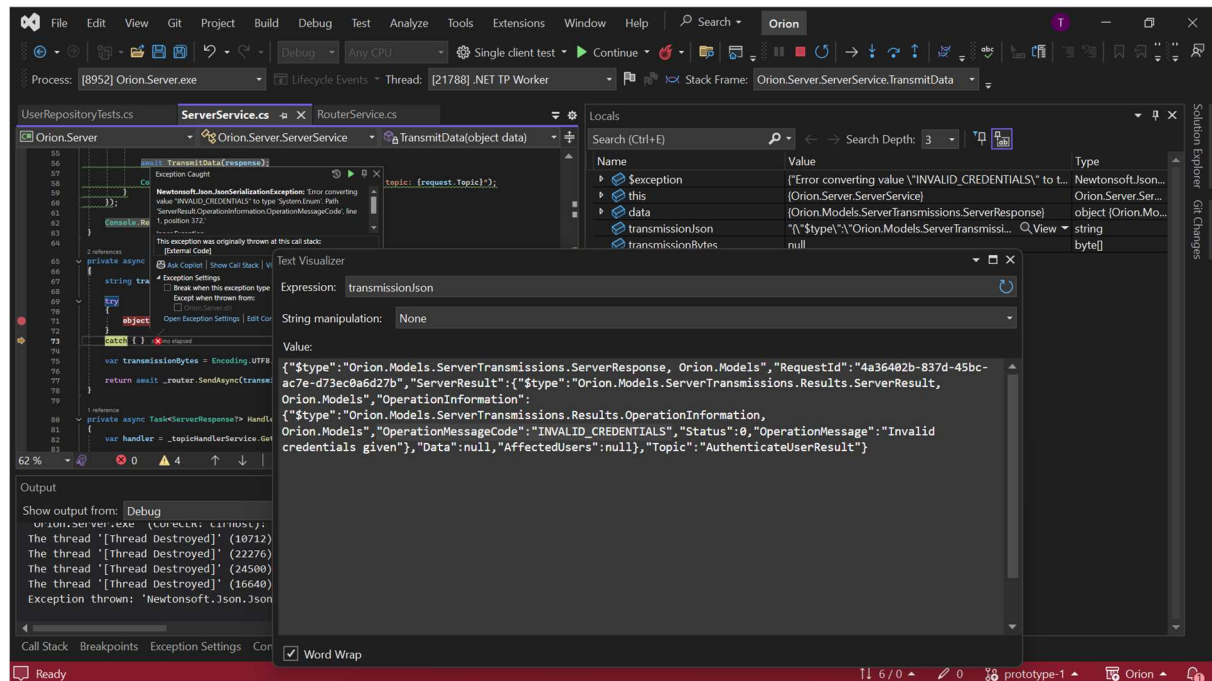
```

18 Console.WriteLine("CLIENT");
19
20 var hostName = Dns.GetHostName();
21 Console.WriteLine($"On host: {hostName}");
22
23 //Console.WriteLine("Router IP address: ");
24 IPAddress routerIPAddress = IPAddress.Parse("192.168.0.28");
25
26 //Console.WriteLine("Router port: ");
27 int routerPort = Convert.ToInt32(50000);
28
29 Console.WriteLine($"On IP address: {routerIPAddress} and port: {routerPort}");
30
31 var routerEndPoint = new IPEndPoint(routerIPAddress, routerPort);
32
33
34 var configurationService = new ConfigurationService();
35
36 configurationService.AddInstanceOfType<IJsonService>(new JsonService());
37
38 configurationService.AddInstanceOfType<ISubscriptionService>(new SubscriptionService());
39 configurationService.AddInstanceOfType<ITopicHandlerService>(new TopicHandlerService(configurationService));
40
41 var connectionService = new ConnectionService
42 (
43     routerEndPoint,
44     configurationService.GetInstanceOfType<IJsonService>(),
45     configurationService.GetInstanceOfType<ISubscriptionService>()
46 );
47
48 configurationService.AddInstanceOfType<IConnectionService>(connectionService);
49
50 configurationService.AddInstanceOfType<ITransmissionService>(new TransmissionService(
51     configurationService.GetInstanceOfType<IConnectionService>(),
52     configurationService.GetInstanceOfType<IJsonService>(),
53     configurationService.GetInstanceOfType<ISubscriptionService>()
54 ));
55
56 configurationService.AddInstanceOfType<IUserService>(new UserService(
57     configurationService.GetInstanceOfType<ITransmissionService>()
58 ));
59
60 configurationService.AddInstanceOfType<IDirectCommunicationService>(new DirectCommunicationService(
61     configurationService.GetInstanceOfType<ITransmissionService>()
62 ));
63
64 connectionService.Run();
65
66 var application = new Application(
67     configurationService.GetInstanceOfType<IUserService>(),
68     configurationService.GetInstanceOfType<IDirectCommunicationService>()
69 );
70 application.Run();
71
72 Console.ReadLine();

```

I decided to stop asking the user to enter the ip address and port number of the router and instead built it in, otherwise the app simply adds all the instances to the configuration service and calls application.run.

When I ran the application initially, I got a JSON exception. After some further investigation, I uncovered that the JSON being sent was flawless.



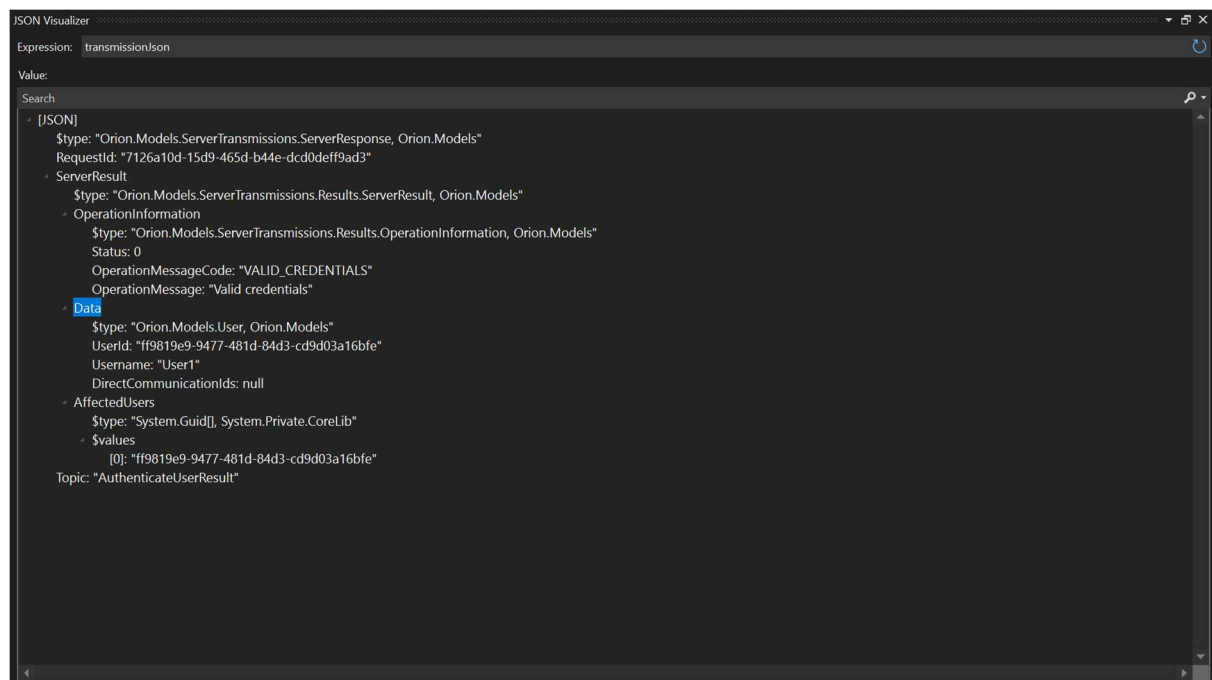
Even then, the exception persisted so I did some research. That lead me to find out an intricacy of Newtonsoft.Json which essentially meant the type of data being sent (i.e. the class type) is not being transmitted along with the JSON so the json service cannot deserialise it properly.

To send the type data, I had to create a JsonSerializerSettings object (provided by Newtonsoft.Json). In this object, I had to specify the typeNameHandling setting, I set it to all so all the type data is sent along with the JSON. I then pass in this JsonSerializerSettings to the serialise and deserialise

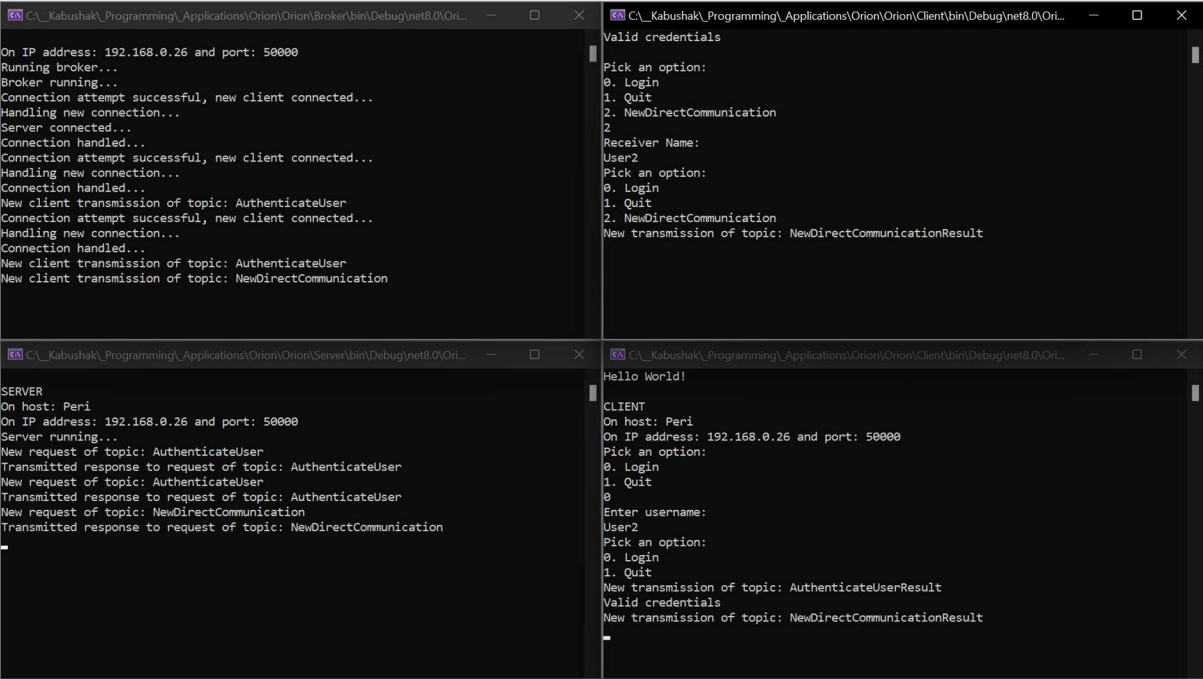
methods.

```
8         private readonly JsonSerializerSettings _jsonSerialiserSettings;
9
10        public JsonService()
11        {
12            _jsonSerialiserSettings = new()
13            {
14                TypeNameHandling = TypeNameHandling.All
15            };
16        }
17
18        public T? DeserializeJson<T>(string json)
19        {
20            return JsonConvert.DeserializeObject<T>(json, _jsonSerialiserSettings);
21        }
22
23        public string SerializeObject(object objectToSerialize)
24        {
25            return JsonConvert.SerializeObject(objectToSerialize, Formatting.None, _jsonSerialiserSettings);
26        }
```

When doing this, the JSON being send changes, and I can also clearly see what type each object is. This is shown in the screenshot below.



The application is shown to run below:



It works with multiple clients, when a new direct communication is added, all members of the direct communication are notified that a direct communication has been added.

REVIEW

The sprint provided more foundation to build more features on and the client is now ready to have its own interface since all its logic code is separate from all the code that runs and displays it.

Since everything is still on the console and there no usability features so, the stakeholders were not shown this very early prototype.

Here is a table detailing what was expected versus what was actually accomplished.

Expected	Actual
Users adding direct communications	Users adding direct communications
Users sending direct messages	No

Sprint 3

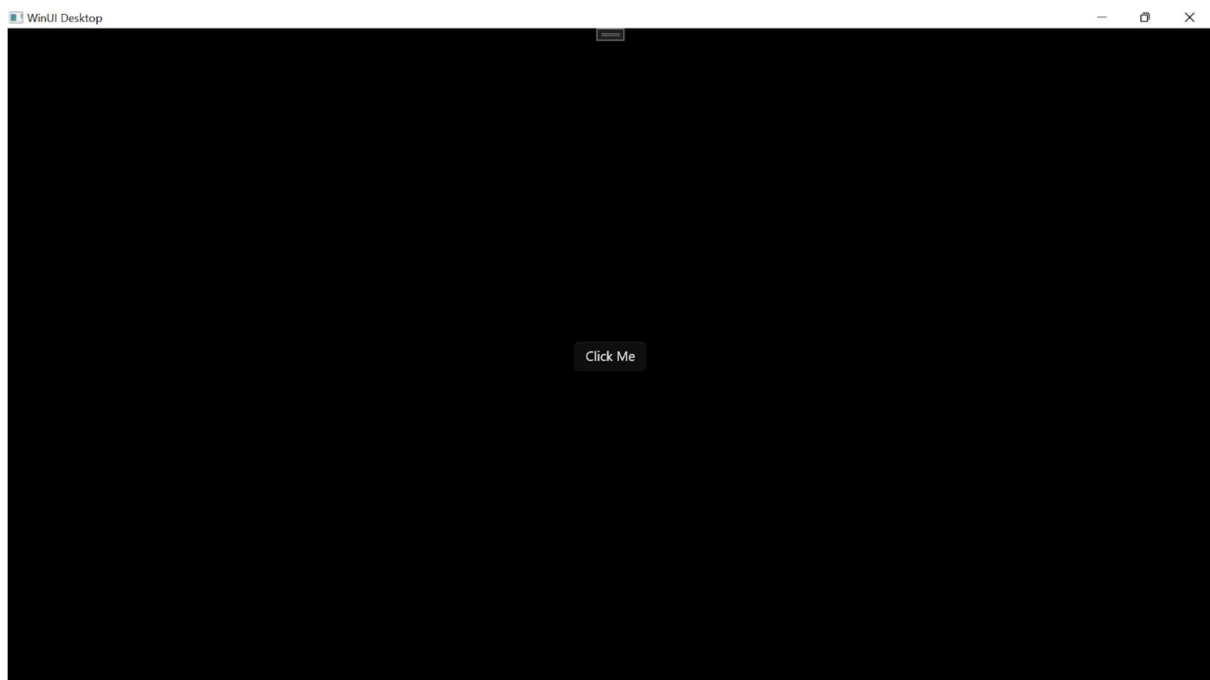
GOALS

During this sprint I want to start the client UI. Along with this I want to implement the following features:

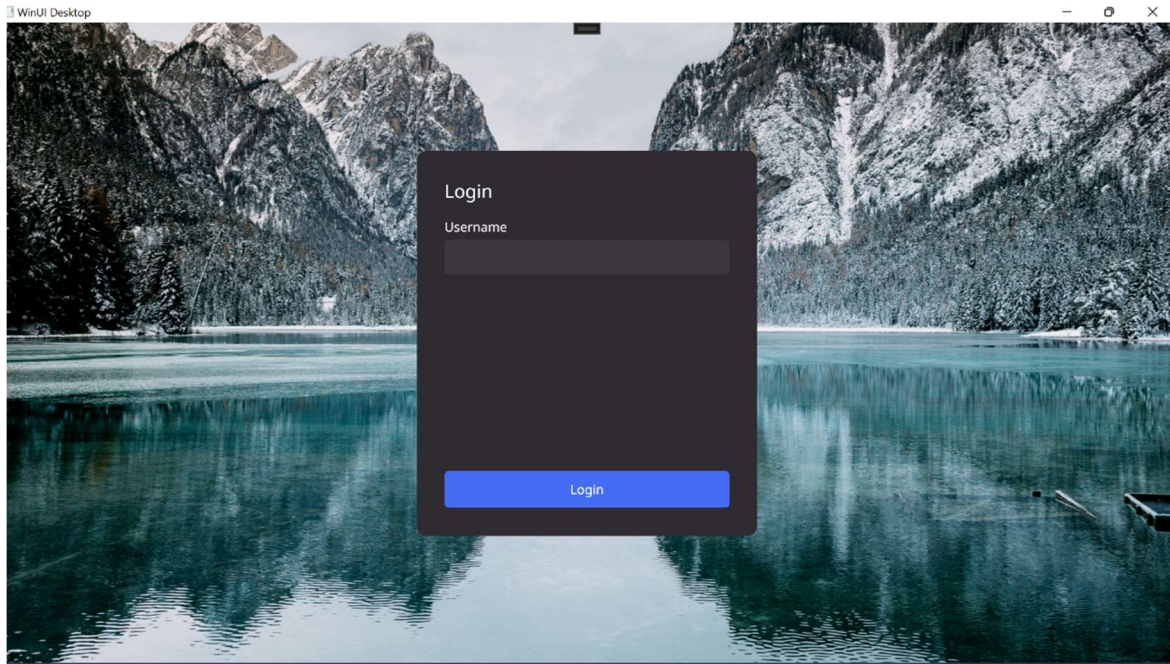
- Direct messages
- Login with password and validations

CODING

I started off by making a new WinUI3 project for the client to use as an interface. The default winui3 application is shown below.



I got to work setting up a login screen. I made it so upon starting up, the app would navigate to the login page. Although no functionality was built into it just yet, and the user can only send a username, I have shown the design I came up with below.



To add the functionality into the login screen, I had to setup everything else first. In the entry point of the project (the `OnLaunch` method of the `App` class in `winui3`) I moved all the instantiation of objects in the configurations service – I copied the contents of the `Program.cs` file into the `OnLaunch` method of the `App.xaml.cs` file. This is shown below.

```

56  protected override void OnLaunched(Microsoft.UI.Xaml.LaunchActivatedEventArgs args)
57  {
58      var hostName = Dns.GetHostName();
59
60      //Console.WriteLine("Router IP address: ");
61      IPAddress routerIPAddress = IPAddress.Parse("192.168.0.26");
62
63      //Console.WriteLine("Router port: ");
64      int routerPort = Convert.ToInt32(50000);
65
66      var routerEndPoint = new IPEndPoint(routerIPAddress, routerPort);
67
68
69      var configurationService = new ConfigurationService();
70
71      configurationService.AddInstanceOfType<IJsonService>(new JsonService());
72
73      configurationService.AddInstanceOfType<ISubscriptionService>(new SubscriptionService());
74      configurationService.AddInstanceOfType<ITopicHandlerService>(new TopicHandlerService(configurationService));
75
76      var connectionService = new ConnectionService
77      (
78          routerEndPoint
79      );
80
81      configurationService.AddInstanceOfType<IConnectionService>(connectionService);
82
83      configurationService.AddInstanceOfType<ITransmissionService>(new TransmissionService(
84          configurationService.GetInstanceOfType<IConnectionService>(),
85          configurationService.GetInstanceOfType<IJsonService>(),
86          configurationService.GetInstanceOfType<ISubscriptionService>()
87      ));
88
89      configurationService.AddInstanceOfType<IUserService>(new UserService(
90          configurationService.GetInstanceOfType<ITransmissionService>()
91      ));
92
93      configurationService.AddInstanceOfType<IDirectCommunicationService>(new DirectCommunicationService(
94          configurationService.GetInstanceOfType<ITransmissionService>()
95      ));
96
97      ConfigurationService = configurationService;
98
99      m_window = new MainWindow();
100     var rootFrame = new Frame();
101     rootFrame.NavigationFailed += OnNavigationFailed;
102
103     rootFrame.Navigate(typeof(Initial), args.Arguments);
104
105     m_window.Content = rootFrame;
106     m_window.Activate();
107 }

```

As shown above, the first page I navigate to is the Initial page. What this will do is connect to the router and start listening for transmissions from the server.

The Initial page consists of a single button in the middle that, when clicked, executes the Work method below.

```

33  private void Work(object sender, RoutedEventArgs e)
34  {
35      App.Current.ConfigurationService.GetInstanceOfType<ITransmissionService>().InitialiseRouterConnection();
36
37      Task.Run(async () =>
38      {
39          while (true)
40          {
41              App.Current.ConfigurationService.GetInstanceOfType<ITransmissionService>().ReceiveData();
42          }
43      });
44
45      Frame.Navigate(typeof(Login));
46  }

```

As shown, all the Work method does is connect to the router via the InitialiseRouterConnection method in the transmission service (line 35) and start listening for transmissions from the router on another thread (lines 37 to 43). After this, it navigates to the login page (line 45).

The InitialiseRouterConnection method is shown below.

```

52  public async Task InitialiseRouterConnection()
53  {
54      var initialiseMessage = _jsonService.SerializeObject("CLIENT");
55      var initialiseMessageBytes = Encoding.UTF8.GetBytes(initialiseMessage);
56
57      await _connectionService.SendMessage(initialiseMessageBytes);
58  }

```

All it does is send the identifier "CLIENT" to the router via the connection service.

In the login page, I wrote an OnLogin method that is called when the user presses the login button. This method is shown below.

```

53  private async void OnLogin(object sender, RoutedEventArgs e)
54  {
55      var username = usernameTextBox.Text;
56
57      var credentials = new Credentials() { Username = username };
58
59      var subscribable = await _userService.AuthenticateUser(credentials);
60      subscribable.Subscribe(result =>
61      {
62          if (result.OperationInformation.OperationMessageCode == AuthenticationMessages.VALID_CREDENTIALS)
63              return;
64          else
65          {
66              SetVisible();
67          }
68      });
69  }

```

The method retrieves the entered username from the username text box, stores it in a credentials object and sends an AuthenticateUser request to the server via the userService. It then subscribes to the response from the server and, if it is valid, it does nothing otherwise it sets the error message

to visible. I did this because I wanted to check if I could get basic communication set up between the new client application, router and server.

```

71     public void SetVisible()
72     {
73         InvalidMessageVisibility = Visibility.Visible;
74     }

```

I have bound the visibility error message to a field in the code behind file of the login page. In the SetVisible method on the left, I set the field to visible.

The userService is populated via the configuration service exposed by the App class, which I can access in every page, its assignment is shown below.

```

50     _userService = App.Current.ConfigurationService.GetInstanceOfType<IUserService>();

```

The App.ConfigurationService field is defined and assigned in the App class.

When testing this code, winui3 threw an error saying that I could not make changes to variables used by the UI in threads other than the main UI thread. Since I was listening for and handling transmissions on another thread, I was also attempting to set the visibility of the error messages in a thread other than the main UI thread. To overcome this, I found out about the dispatcher queue which allowed me to enqueue tasks to perform on the main UI thread.

Using this newfound knowledge, I made the following changes to the Work method in the Initial page.

```

34 35     {
35 36         App.Current.ConfigurationService.GetInstanceOfType<ITransmissionService>().InitialiseRouterConnection();
36 37
37 38     - Task.Run(async () =>
38 39         + DispatcherQueueTimer d = DispatcherQueue.GetForCurrentThread().CreateTimer();
39 40         + d.Interval = TimeSpan.Zero;
40 41         + d.Tick += (s, e) =>
41 42         + {
42 43             while (true)
43 44             {
44 45                 App.Current.ConfigurationService.GetInstanceOfType<ITransmissionService>().ReceiveData();
45 46             }
46 47         });
47 48         App.Current.ConfigurationService.GetInstanceOfType<ITransmissionService>().ReceiveData();
48 49         };
49 50         d.Start();
50 51     }
51 52     Frame.Navigate(typeof(Login));
52 53 }

```

This resulted in a final Work method of:

```
34  private void Work(object sender, RoutedEventArgs e)
35  {
36      App.Current.ConfigurationService.GetInstanceOfType<ITransmissionService>().InitialiseRouterConnection();
37
38      DispatcherQueueTimer d = DispatcherQueue.GetForCurrentThread().CreateTimer();
39
40      d.Interval = TimeSpan.Zero;
41
42      d.Tick += (s, e) =>
43      {
44          App.Current.ConfigurationService.GetInstanceOfType<ITransmissionService>().ReceiveData();
45      };
46
47      d.Start();
48
49      Frame.Navigate(typeof(Login));
50  }
```

What this does is now, instead of creating a new thread and running an infinite loop inside of it to listen for requests, I start a timer and calls the ReceiveData method at the end of the timer, after which the timer is started again. Using this, the execution of the ReceiveData method occurs within the main UI thread. I set the interval of the timer to 0 (an incredibly bad idea) which enqueues the action of calling the ReceiveData method on the main UI thread a lot of times.

This approach worked but also cause the app to crash after 5 minutes because it “attempted to write to unauthorised memory”. For now, to get features working, this method is fine however, in the future, to make a robust application that can last for more than 5 minutes I want to have the listening for messages occur on its own thread then have every transmission received enqueued onto some transmission queue then I can dequeue these received transmissions in the main UI thread or something along those lines. This way I should not end up overloading the main UI thread.

Now that I know communication is working, I scrapped the idea of the Initial page since it was not very usable. I moved the code in the Work method to the entry point of the application (it will connect to the router and start listening for the transmissions before the GUI has been presented) and made the first page that shows up the login page.

I also added an ApplicationPage to shown when the user logs in successfully. I changed the OnLogin method to accommodate this, as shown below. It also now stores the data given by the server in the

App.Current property so that the user's information can be used throughout the rest of the app.

```

58 private async void OnLogin(object sender, RoutedEventArgs e)
59 {
60     var username = usernameTextBox.Text;
61
62     var credentials = new Credentials() { Username = username };
63
64     var subscribable = await _userService.AuthenticateUser(credentials);
65     subscribable.Subscribe(result =>
66     {
67         if (result.OperationInformation.OperationMessageCode == AuthenticationMessages.VALID_CREDENTIALS)
68         {
69             SetInvisible();
70             App.Current.User = result.Data as User;
71             Frame.Navigate(typeof(ApplicationPage));
72         }
73         else
74             SetVisible();
75     });
76 }

```

The application page simply holds frames for the navbar and the main content, as shown to the right. This is to maintain consistency throughout the app. The navbar will stay fixed while the content frame will be populated with the currently selected page i.e. the direct communications or groups page.

```

<Grid ColumnSpacing="20" Padding="30, 30, 0, 0">
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="50" />
    <ColumnDefinition Width="*" />
  </Grid.ColumnDefinitions>

  <Frame x:Name="NavbarFrame" Grid.Column="0">

  </Frame>
  <Frame x:Name="ContentFrame" Grid.Column="1">

  </Frame>
</Grid>

```

```

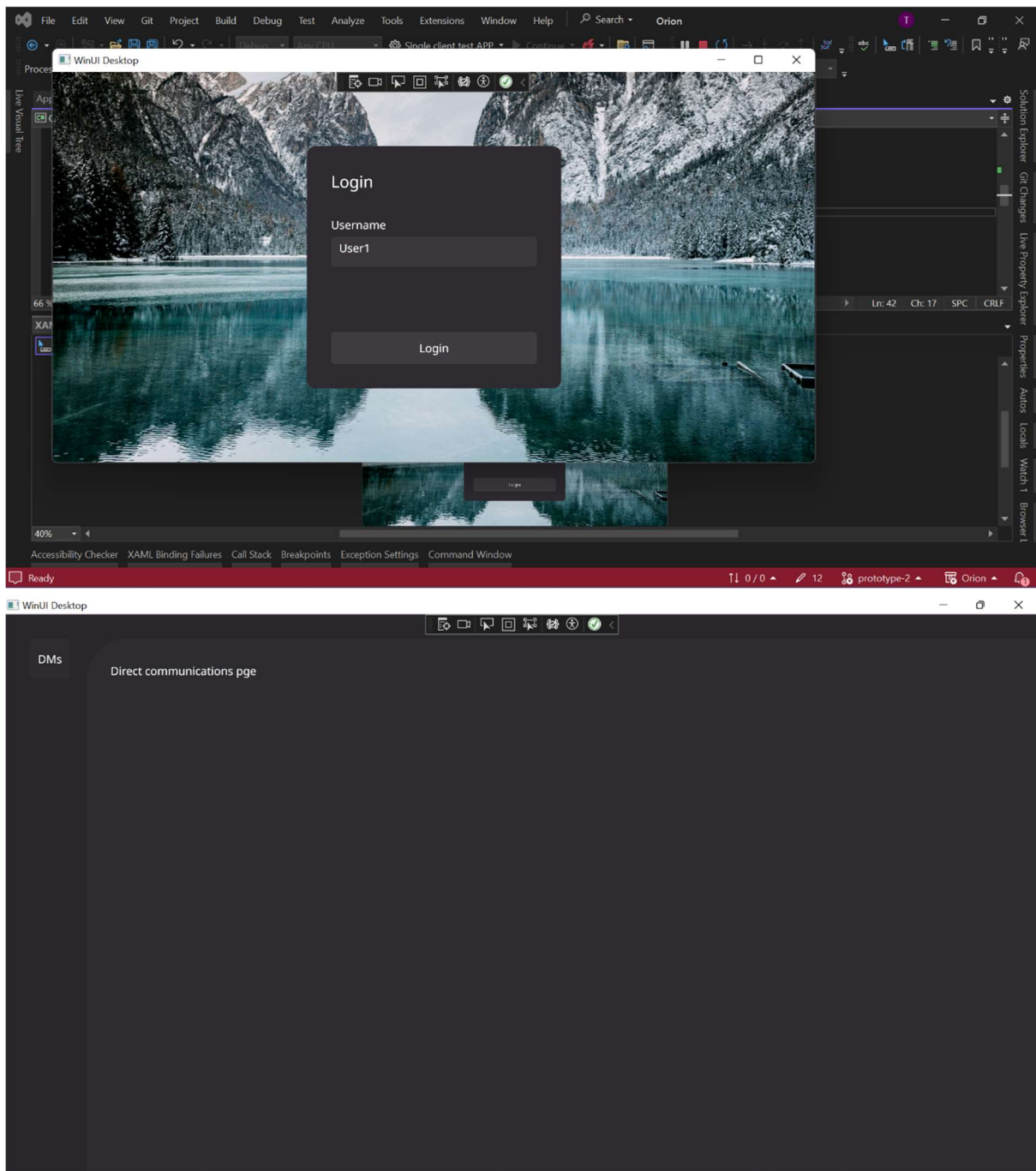
26 public ApplicationPage()
27 {
28     InitializeComponent();
29
30     App.Current.ContentFrame = ContentFrame;
31     App.Current.NavbarFrame = NavbarFrame;
32
33     App.Current.NavbarFrame.Navigate(typeof(Navbar));
34 }

```

When the application page is initialised, it navigates the NavbarFrame to the Navbar page, as shown to the left.

The Navbar page, for now, has a single button that navigates to the DirectCommunications Page.

The direct communications page is currently (pretty much) empty, I just want to see the general flow of the application.



It worked.

Now, I decided to refactor the server transmissions so that I could easily distinguish a server response from a server publish. I added a `ServerTransmission` class as shown below.

```

9  public class ServerTransmission
10 {
11     public ServerResponse Response { get; set; }
12
13     public ServerResponse? Publish { get; set; }
14
15     public ServerTransmission(ServerResponse response, ServerResponse? publish = null)
16     {
17         Response = response;
18         Publish = publish;
19     }
20 }

```

I kept the `Response` and `Publish` both objects of `ServerResponse` and for now only up-to a single response and up-to a single publish can be sent at a time. I also moved the `AffectedUsers` field into the `ServerResponse` object and out of the `ServerResults` object, this change is shown in the snippets below.

```

5  public class ServerResponse
6  {
7      public Guid? RequestId { get; set; }
8
9      public ServerResult? ServerResult { get; set; }
10
11     public Guid[]? AffectedUsers { get; set; }
12
13     public string Topic { get; set; }
14
15     public ServerResponse(string topic, ServerResult? serverResult = null, Guid? requestId = null, Guid[]? affectedUsers = null)
16     {
17         Topic = topic;
18         ServerResult = serverResult;
19         RequestId = requestId;
20         AffectedUsers = affectedUsers;
21     }
22 }

```

```

3  public class ServerResult<T> where T : Enum
4  {
5      public OperationInformation<T> OperationInformation { get; set; }
6
7      public object? Data { get; set; }
8
9      public ServerResult(OperationInformation<T> operationInformation, object? data = null)
10     {
11         OperationInformation = operationInformation;
12         Data = data;
13     }

```

```

22 public class ServerResult
23 {
24     public OperationInformation? OperationInformation { get; set; }
25
26     public object? Data { get; set; }
27
28     public ServerResult(OperationInformation? operationInformation = null, object? data = null)
29     {
30         OperationInformation = operationInformation;
31         Data = data;
32     }
33 }

```

The repositories should now return ServerTrasmissions and not ServerResults, this is because the repositories will populate the AffectedUsers list. To enable this, I changed the ServerResultsService to create server transmissions as shown below.

```

6  ✓ public static class ServerResultService
7  {
8      public static ServerTransmission NewSuccessfulResponseServerTransmission<T>(string topic, T operationMessageCode, object? data = null) where T : Enum
9      {
10         return new ServerTransmission(new ServerResponse(topic, new ServerResult<T>(new OperationInformation<T>(Statuses.SUCCEEDED, operationMessageCode), data)));
11     }
12
13  ✓ public static ServerTransmission AddResponseOperationMessage(this ServerTransmission serverTransmission, string? operationMessage = null)
14  {
15     serverTransmission.Response.ServerResult.OperationInformation.OperationMessage = operationMessage;
16
17     return serverTransmission;
18 }
19
20  ✓ public static ServerTransmission AddResponseAffectedUser(this ServerTransmission serverTransmission, Guid affectedUser)
21  {
22     serverTransmission.Response.AffectedUsers = [affectedUser];
23
24     return serverTransmission;
25 }
26
27  ✓ public static ServerTransmission AddPublish(this ServerTransmission serverTransmission, string topic, object? data, Guid[]? affectedUsers)
28  {
29     serverTransmission.Publish = new ServerResponse(topic, new ServerResult(null, data), null, affectedUsers);
30
31     return serverTransmission;
32 }
33 public static ServerTransmission AddPublish(this ServerTransmission serverTransmission, string topic, object? data, Guid affectedUser) =>
34     serverTransmission.AddPublish(topic, data, [affectedUser]);
35 }

```

Now the service works as follows: a new successful response can be created, on-top of that response an operation message can be added along with affected users and publishes. This is better than having to enter all the data into a single subroutine as it produces cleaner code. The difference is shown below.

The red is the code before and the green is the code after, as shown the newer version is much more readable and, thus, maintainable.

```

64 -         ? ServerResultService.NewSuccessfulServerResult(
65 -             operationMessageCode: DirectCommunicationMessages.DIRECT_COMMUNICATION_CREATION_SUCCEEDED,
66 -             affectedUsers: [newDirectCommunication.SenderId, receiver.UserId],
67 -             data: directCommunication)
68 -         : ServerResultService.NewSuccessfulServerResult(DirectCommunicationMessages.DIRECT_COMMUNICATION_CREATION_FAILED, "Creation of direct communication failed",
        [newDirectCommunication.SenderId, receiver.UserId]);
65 +         ? ServerResultService.NewSuccessfulResponseServerTransmission("NewDirectCommunicationResult", DirectCommunicationMessages.DIRECT_COMMUNICATION_CREATION_FAILED)
66 +         .AddResponseOperationMessage("Creation of new direct communication failed...")
67 +         .AddResponseAffectedUser(newDirectCommunication.SenderId)
68 +         : ServerResultService.NewSuccessfulResponseServerTransmission("NewDirectCommunicationResult", DirectCommunicationMessages.DIRECT_COMMUNICATION_CREATION_SUCCEEDED)
69 +         .AddResponseOperationMessage("New direct communication created...")
70 +         .AddResponseAffectedUser(newDirectCommunication.SenderId)
71 +         .AddPublish("NewDirectCommunication", directCommunication, receiverId);

```

I had to change the router a bit to accommodate this ServerTransmission system, the changes are shown below and, since they are quite a bit, they are split up and explained.

The changes inside the ServerConnection method from lines 108 to 128 are self-explanatory since they are just the replacing of serverResponse.Topic with serverTransmission.Response.Topic. This is shown below.

```

108 - ServerResponse serverResponse = await ReceiveServerTransmission();
108 + ServerTransmission serverTransmission = await ReceiveServerTransmission();
109 109
110 - bool topicHasInterceptor = _topicInterceptorService.TryGetTopicInterceptor(serverResponse.Topic, out var topicInterceptor);
110 + bool topicHasInterceptor = _topicInterceptorService.TryGetTopicInterceptor(serverTransmission.Response.Topic, out var topicInterceptor);
111 111
112 112         if (topicHasInterceptor)
113 113         {
114 - topicInterceptor.Invoke(serverResponse);
114 + topicInterceptor.Invoke(serverTransmission.Response);
115 115         }
116 116
117 - var broadcastList = serverResponse.ServerResult.AffectedUsers;
117 + var broadcastList = serverTransmission.Response.AffectedUsers;
118 118
119 119         if (broadcastList == null || broadcastList.Length == 0)
120 120         {
121 - if (serverResponse.RequestId == null)
121 + if (serverTransmission.Response.RequestId == null)
122 122             continue;
123 123
124 - bool wasRequested = _requestService.TryGetRequester(serverResponse.RequestId.Value, out var client);
124 + bool wasRequested = _requestService.TryGetRequester(serverTransmission.Response.RequestId.Value, out var requester);
125 125
126 126         if (wasRequested)
127 127         {
128 - await ForwardServerResponse(serverResponse, client);
128 + await ForwardServerResponse(serverTransmission.Response, requester);
129 129         }
130 130
131 131         continue;
132 132     }

```

I also changed the ReceiveServerTransmission method so that it returns a ServerTransmission instead of a ServerResponse. This is shown below.

```

157 - public async Task<ServerResponse?> ReceiveServerTransmission()
167 + public async Task<ServerTransmission?> ReceiveServerTransmission()
158 168     {
159 169         var buffer = new byte[2048];
160 170
161 171         int transmissionBytesCount = await _server.ReceiveAsync(buffer, SocketFlags.None);
162 172
163 173         string transmissionJson = Encoding.UTF8.GetString(buffer, 0, transmissionBytesCount);
164 - ServerResponse? serverResponse = (_jsonService.DeserializeJson<ServerResponse?>)(transmissionJson);
174 + ServerTransmission? serverTransmission = _jsonService.DeserializeJson<ServerTransmission?>(transmissionJson);
165 175
166 - return serverResponse;
176 + return serverTransmission;
167 177     }

```

Sprint 4

GOALS

In this sprint, I want to connect the server to a database.

CODING

I needed to connect the server to the database. I used Entity Framework Core. To do this, I had to change my current DataLayer system. The DbContext is shown below.

```

15  public class OrionDbContext : DbContext
16  {
17      public DbSet<User> Users { get; set; }
18      public DbSet<DirectCommunication> DirectCommunications { get; set; }
19      public DbSet<Message> Messages { get; set; }
20
21      private readonly string _connectionString;
22
23      public OrionDbContext() : base()
24      {
25          _connectionString = ConfigurationManager.ConnectionStrings["OrionDatabase"].ToString();
26      }
27
28      protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
29      {
30          optionsBuilder.UseSqlServer(_connectionString);
31      }
32
33  protected override void OnModelCreating(ModelBuilder modelBuilder)
34  {
35      modelBuilder.Entity<User>()
36          .HasIndex(e => e.Username)
37          .IsUnique();
38  }
39  }

```

Because of the nature of DbContext, I had to change the ConfigurationService to have scoped and singleton instances. Essentially, this is because a new DbContext has to be created for every “small unit of work” [INSERT MICROSOFT DOCUMENTATION LINK]. To do this, the changes I made are shown below.

This is how a scoped instance is added.

```

20  public void AddScoped<Type, InstanceType>() where InstanceType : Type
21  {
22      scopedTypesOrder.Add(typeof(Type));
23      scopedTypes.Add(typeof(Type), typeof(InstanceType));
24
25      var constructors = typeof(InstanceType).GetConstructors();
26      var constructorInfo = constructors.First();
27
28      var parameters = constructorInfo.GetParameters();
29      object[] arguments = new object[parameters.Length];
30
31      for (int i = 0; i < arguments.Length; i++)
32      {
33          arguments[i] = GetScopedOfType(parameters[i].ParameterType) ?? GetSingletonOfType(parameters[i].ParameterType);
34      }
35
36      scopedInstances.Add(typeof(Type), constructorInfo.Invoke(arguments));
37  }

```

It does not make sense for scoped instances to be added with an instance, since they will be reinstantiated later.

This is how a singleton is added.

```
39  public void AddSingleton<Type, InstanceType>() where InstanceType : Type
40  {
41      var constructorInfo = typeof(InstanceType).GetConstructors().Single();
42
43      var parameters = constructorInfo.GetParameters();
44      object[] arguments = new object[parameters.Length];
45
46      for(int i = 0; i < arguments.Length; i++)
47      {
48          arguments[i] = GetSingletonOfType(parameters[i].ParameterType);
49      }
50
51      singletons.Add(typeof(Type), constructorInfo.Invoke(arguments));
52  }
53
54  public void AddSingleton<T>(T instance)
55  {
56      singletons.Add(typeof(T), instance);
57  }
```

There are 2 ways to add a singleton, either the same way as adding a scoped instance or by passing in an object to store.

To 'reset' the scope (every scoped instance will be reinstantiated) the following method is shown below.

```
98  public void ResetScope()
99  {
100     foreach(var type in scopedTypesOrder)
101     {
102         var constructorInfo = scopedTypes[type].GetConstructors().First();
103
104         var parameters = constructorInfo.GetParameters();
105         object[] arguments = new object[parameters.Length];
106
107         for (int i = 0; i < arguments.Length; i++)
108         {
109             arguments[i] = GetScopedOfType(parameters[i].ParameterType) ?? GetSingletonOfType(parameters[i].ParameterType);
110         }
111
112         scopedInstances[type] = constructorInfo.Invoke(arguments);
113     }
114 }
```

Because of how this works, singleton instances can only depend on singleton objects.

In the server, the scope is reset whenever GetTopicHandler is called, this is because the method will only be called when handling a request, and the scope needs to be reset for every request.

```
54  public Func<object, Task<ServerTransmission>>> GetTopicHandler(string topic)
55  {
56      _configurationService.ResetScope();
```

```
10  public class HashingService : IHashingService
11  {
12      private readonly SHA256 _sha256;
13
14      public HashingService()
15      {
16          _sha256 = SHA256.Create();
17      }
18
19      public byte[] Hash(byte[] bytes)
20      {
21          return _sha256.ComputeHash(bytes);
22      }
23  }
```

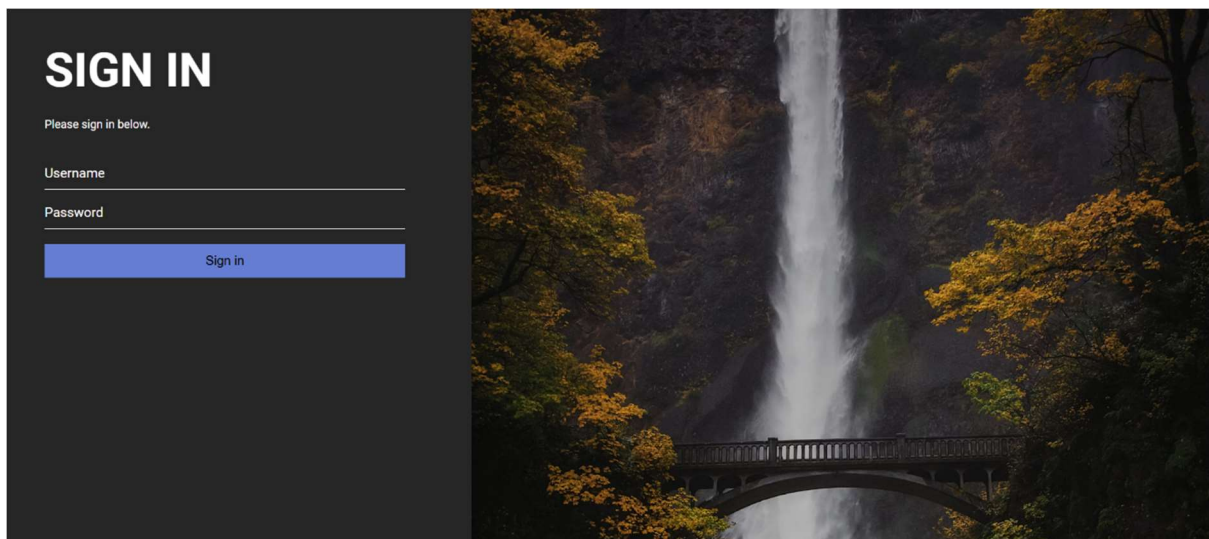
Now that the database is set up, I decided to add passwords to users. This involves hashing so I created a new HashingService that wraps a library that handles hashing for me.

I am using the SHA256 algorithm.

This is how the user repository uses the hashing service.

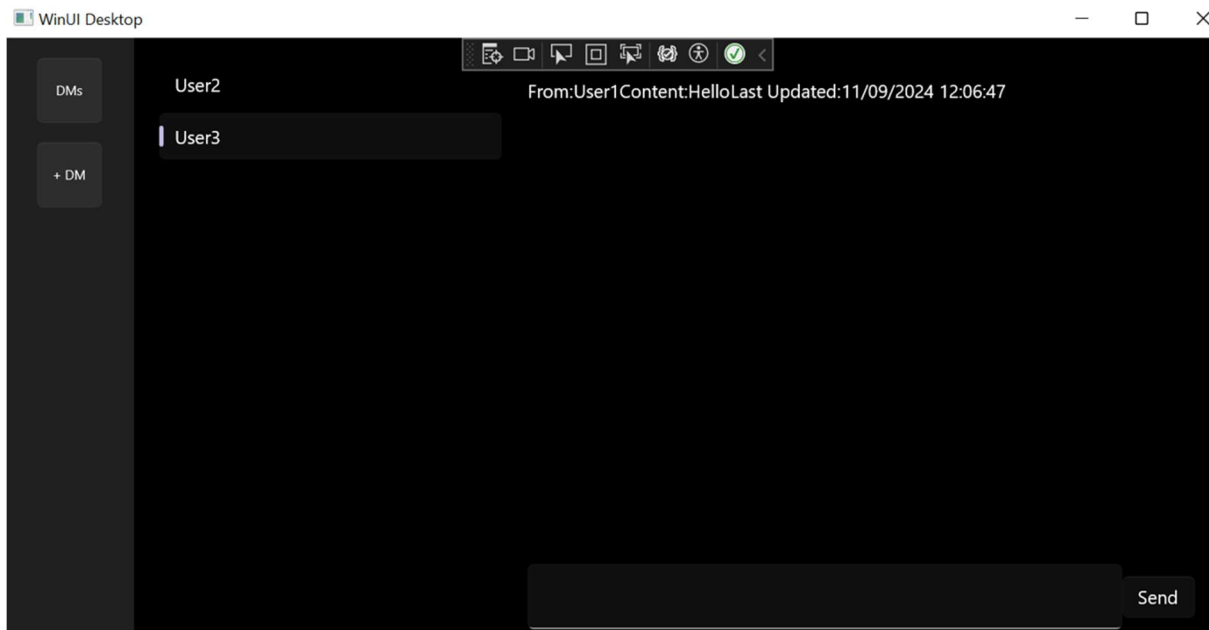
```
25  public async Task<ServerTransmission> AuthenticateUser(Credentials credentials)
26  {
27      var passwordHash = _hashingService.Hash(Encoding.UTF8.GetBytes(credentials.Password));
28
29      var getDataQuery = _database.Users
30          .Where(user => user.Username == credentials.Username && user.PasswordHash == passwordHash)
31          .Include(user => user.DirectCommunications)
32          .ThenInclude(d => d.Members);
```

Along with adding the password in the login screen, I changed the user login design. This was greatly favoured by the stakeholders.



When the user logs in successfully, the following screen is shown.

Sending direct messages works now.



Due to entity framework core, I could not simply add the user ids when adding a message to the database, I had to begin tracking them first. This is shown below.

```
27         var sender = await _database.Users.FindAsync(newDirectCommunication.SenderId);
28         var receiver = await _database.Users.FindAsync(newDirectCommunication.ReceiverId);
29
30         if (sender == null)
31             throw new UserNotFoundException("Sender was null");
32
33         if (receiver == null)
34             throw new UserNotFoundException("Receiver was null");
35
36         var directCommunication = new DirectCommunication()
37         {
38             DirectCommunicationId = Guid.NewGuid(),
39             Members = new List<User>() { sender, receiver }
40         };
41
42         await _database.DirectCommunications.AddAsync(directCommunication);
43
44         int appliedChanges = await _database.SaveChangesAsync();
```

Although it queries the database every time a message is sent, it is necessary to ensure that the users given in the direct communication are real users and enforces data integrity.

I now want to work on the Admin Portal, to do this I will need user roles so I wrote that.

```
11 references
public enum Roles
{
    USER,
    ADMIN,
    SUPERADMIN
}
```

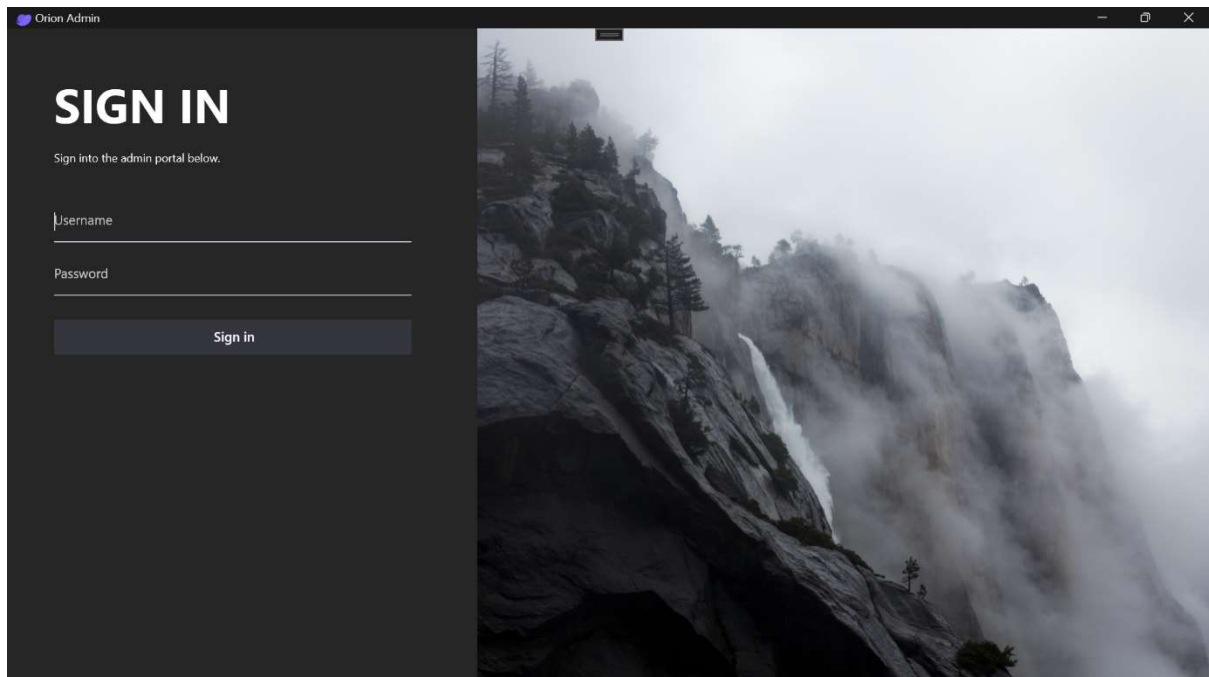
Sprint 5

GOALS

In this sprint I need to get the admin portal set up. To do this quickly, I will merge the admin portal and the server apps into one. This means I do not have to handle as many security issues since the admin will not be making requests over the network that could be intercepted or spoofed by malicious parties.

CODING

I started by copying the login page from the user portal, I changed the image and the colour of the button to make it easy to differentiate between the 2.



I added a users list page, its not pretty but I am focusing on the functionality.

d36edf7b-648f-483b-81ec-b60a7d001878	User1
05766fe1-8400-4504-9f1d-d39f6eee27d8	User2
6dd362d7-e8d7-4ef1-a354-53bfdfa00a20	User3

The users list page is populated by accessing the user repository as shown below.

```
49         var users = await App.Current.ConfigurationService.GetInstanceOfType<IUserRepository>()
50             .GetAllUsers();
```

Currently users are simply displayed with their userID and username. You may notice a log page on the top left but it has not been implemented yet.

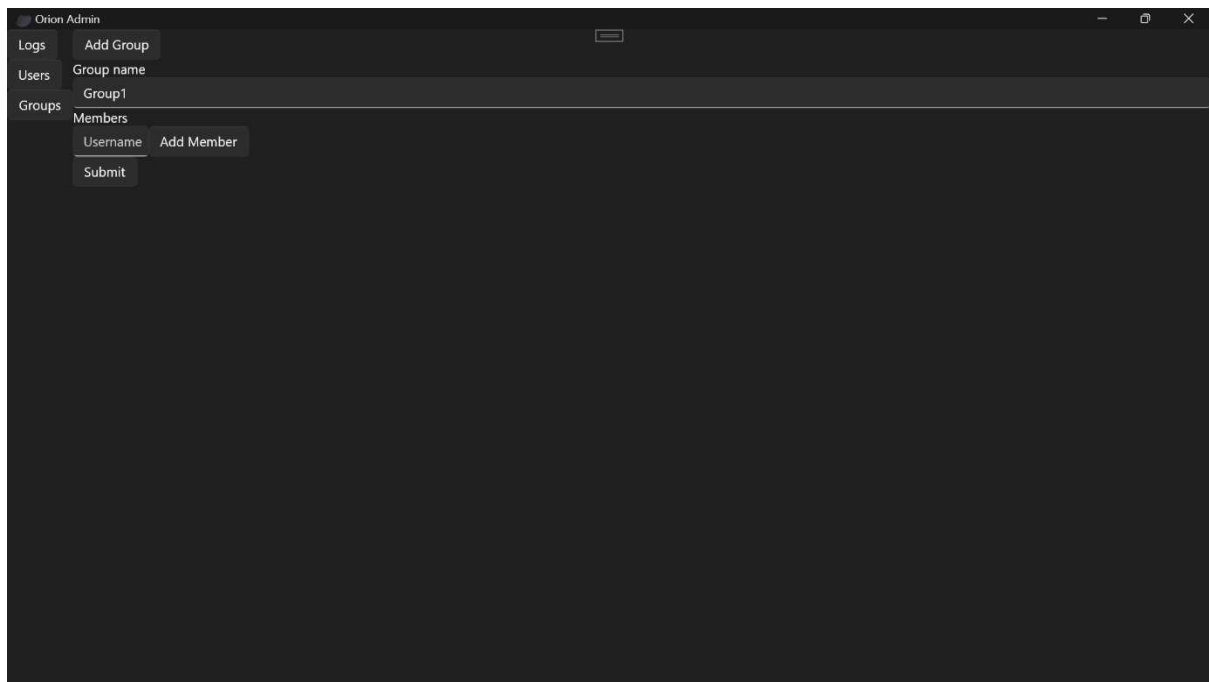
The AddUser button opens the add user form. I wan to reuse the same form for adding and editing users. This is why I provided a public event to bind to when submitting the form, as shown below.

```
38         public event Action<string, string, Roles> OnSubmit = delegate { };
```

I bind to it in the code below.

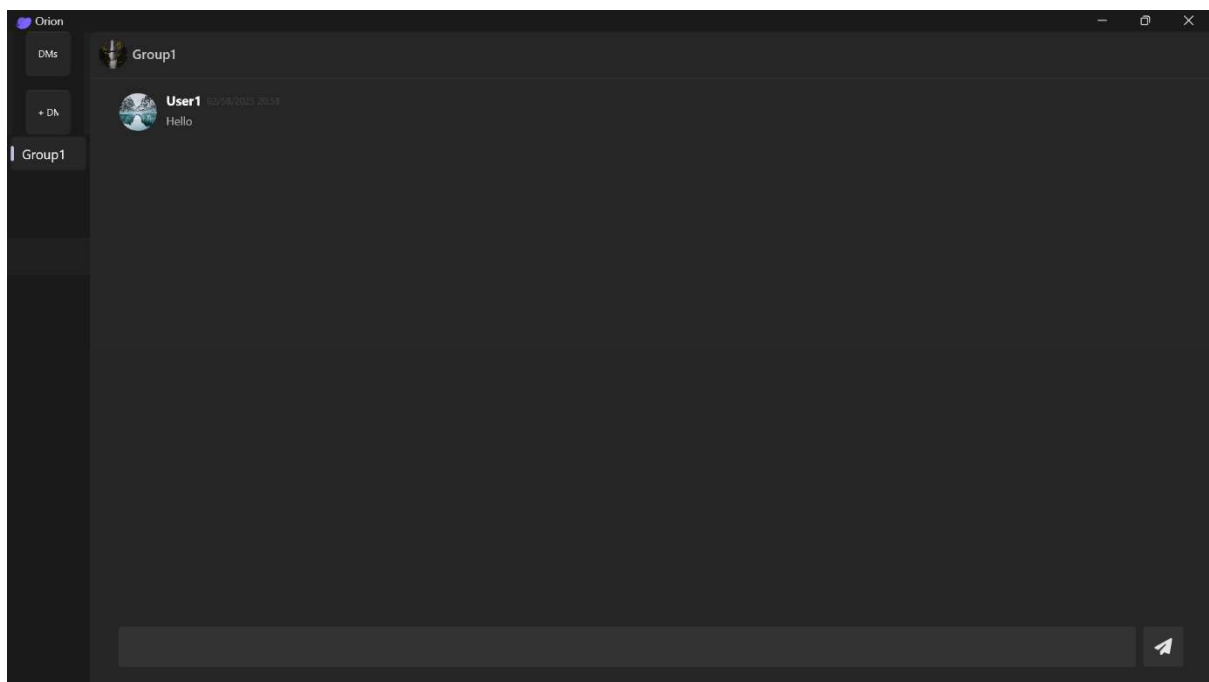
```
26         <local:UserForm Visibility="{x:Bind Model.HideAddUserForm, Converter={StaticResource BoolToVisibilityConverter}, Mode=OneWay}" OnSubmit="{x:Bind Model.AddUser}" />
```

I also got to work on a group lists page.



The groups list page is populated via a group repository just like the users list page.

I also got groups working on the client and made some design changes to the client app.



Sprint 6

CODING

I decided to make the transmission and connection services.

The connection service is shown below.

The send message function:

```
47  public async Task<int> SendMessage(byte[] data)
48  {
49      if (_socket == null)
50          throw new Exception("socket was null...");
51
52      byte[] finalTransmission = new byte[4 + data.Length];
53
54      Buffer.BlockCopy(BitConverter.GetBytes(data.Length), 0, finalTransmission, 0, 4);
55      Buffer.BlockCopy(data, 0, finalTransmission, 4, data.Length);
56
57      int sentBytes = 0;
58
59      byte[] sendBuffer = new byte[_bufferLength];
60
61      do
62      {
63          if (finalTransmission.Length - sentBytes < _bufferLength)
64              sendBuffer = new byte[finalTransmission.Length - sentBytes];
65
66          Buffer.BlockCopy(finalTransmission, sentBytes, sendBuffer, 0, sendBuffer.Length);
67
68          sentBytes += await _socket.SendAsync(sendBuffer, SocketFlags.None);
69      }
70      while (sentBytes < finalTransmission.Length);
71
72      return sentBytes;
73  }
```

The receive message function:

```
public async Task<MessageBytes> ReceiveMessage()

    byte[] receiveBuffer = new byte[_bufferLength];

    int previousReceivedBytes = 0;
    int receivedBytes = 0;
    int transmissionLength = 4;
    byte[] lengthBuffer = new byte[4];
    int someRandomNumber = 0;

    while (receivedBytes < transmissionLength)
    {
        previousReceivedBytes = await _socket.ReceiveAsync(receiveBuffer, SocketFlags.None);

        if (previousReceivedBytes == 0)
            return new MessageBytes(lengthBuffer, receivedBytes);

        someRandomNumber = Math.Min(Math.Max(4 - receivedBytes, 0), receiveBuffer.Length);
        Buffer.BlockCopy(receiveBuffer, 0, lengthBuffer, receivedBytes, someRandomNumber);
        receivedBytes += previousReceivedBytes;
    }

    transmissionLength = BitConverter.ToInt32(lengthBuffer);
    byte[] finalBuffer = new byte[transmissionLength];
    receivedBytes -= 4;

    Buffer.BlockCopy(receiveBuffer, someRandomNumber, finalBuffer, 0, receivedBytes);

    while (receivedBytes < transmissionLength)
    {
        previousReceivedBytes = await _socket.ReceiveAsync(receiveBuffer, SocketFlags.None);

        if (previousReceivedBytes == 0)
            return new MessageBytes(finalBuffer, receivedBytes);

        Buffer.BlockCopy(receiveBuffer, 0, finalBuffer, receivedBytes, Math.Min(receiveBuffer.Length, transmissionLength - receivedBytes));
        receivedBytes += previousReceivedBytes;
    }
}
```

I use Buffer.BlockCopy to copy chunks of data into the send and receive buffers.

The transmission service is shown below, without any encryption.

```
33  public async Task<T> ReceiveTransmission<T>() where T : class
34  {
35      if (!Initialised)
36          throw new Exception("Not initialised...");
37
38      var transmissionBytes = await _connectionService.ReceiveMessage();
39
40      string json = Encoding.UTF8.GetString(transmissionBytes.Data, 0, transmissionBytes.DataByteLength);
41
42      return _jsonService.DeserialiseJson<T>(json);
43  }
44
45  public async Task<int> SendTransmission(object transmission)
46  {
47      if (!Initialised)
48          throw new Exception("Not initialised...");
49
50      string transmissionJson = _jsonService.SerialiseObject(transmission);
51      byte[] transmissionJsonBytes = Encoding.UTF8.GetBytes(transmissionJson);
52
53      return await _connectionService.SendMessage(transmissionJsonBytes);
54  }
```

Now I set up the LifeSupport class.

```
14  public class LifeSupport
15  {
16      private Guid _userId;
17
18      private readonly ITransmissionService _transmissionService;
19
20      private bool _isLoggedIn;
21
22      public bool IsLoggedIn => _isLoggedIn;
23
24      public bool Initialised => _transmissionService.Initialised;
25
26      public event Action<Guid> OnLogIn = delegate { };
27
28      public Guid UserId => _userId;
29
30      private LifeSupport()
31      {
32          _isLoggedIn = false;
33      }
34
35      public LifeSupport(Socket socket, IJsonService jsonService, int bufferLength = 16192) : this()
36      {
37          _transmissionService = new TransmissionService(jsonService, new ConnectionService(socket, bufferLength));
38      }
39
40      public LifeSupport(ITransmissionService transmissionService) : this()
41      {
42          _transmissionService = transmissionService;
43      }
44
45  public void LogIn(Guid userId)
46  {
47      if (_isLoggedIn)
48          throw new InvalidOperationException("Already logged in...");
49
50      _isLoggedIn = true;
51      _userId = userId;
52      OnLogIn.Invoke(_userId);
53  }
54
55  public async Task<bool> InitialiseConnection()
56  {
57      return await _transmissionService.InitialiseConnection();
58  }
59
60  public async Task<T> ReceiveTransmission<T>() where T : class
61  {
62      return await _transmissionService.ReceiveTransmission<T>();
63  }
64
65  public async Task<int> SendTransmission(object data)
66  {
67      return await _transmissionService.SendTransmission(data);
68  }
```

I changed the router's connection service to a client service and is shown below.

```
10  public class ClientService : IClientService
11  {
12      private readonly IJsonService _jsonService;
13
14      private readonly ConcurrentDictionary<Guid, LifeSupport> _userIdToLifeSupport;
15
16  public ClientService(IJsonService jsonService)
17  {
18      _jsonService = jsonService;
19      _userIdToLifeSupport = new();
20  }
21
22  public async Task<LifeSupport> InstantiateConnection(Socket socket)
23  {
24      var lifeSupport = new LifeSupport(socket, _jsonService);
25
26      await lifeSupport.InitialiseConnection();
27
28      lifeSupport.OnLogIn += userId => _userIdToLifeSupport.TryAdd(userId, lifeSupport);
29
30      return lifeSupport;
31  }
32
33  public bool TerminateConnection(ref LifeSupport connection)
34  {
35      return _userIdToLifeSupport.TryRemove(connection.UserId, out _);
36  }
37
38  public bool TryGetClientConnection(Guid userId, out LifeSupport connection)
39  {
40      return _userIdToLifeSupport.TryGetValue(userId, out connection);
41  }
42  }
```

The request service was also changed, as shown below.

```
9      private readonly ConcurrentDictionary<Guid, LifeSupport> _requestIdToRequester;
10
11  public RequestService()
12  {
13      _requestIdToRequester = new ConcurrentDictionary<Guid, LifeSupport>();
14  }
15
16  public bool TryEndRequest(Guid requestId, out LifeSupport connection)
17  {
18      return _requestIdToRequester.Remove(requestId, out connection);
19  }
20
21  public bool TryGetRequestConnection(Guid requestId, out LifeSupport connection)
22  {
23      return _requestIdToRequester.TryGetValue(requestId, out connection);
24  }
25
26  public bool NewRequest(ref LifeSupport connection, out Guid requestId)
27  {
28      requestId = Guid.NewGuid();
29
30      return _requestIdToRequester.TryAdd(requestId, connection);
31  }
```

I also implemented encryption. This is shown below.

```
10  ✓ public class AesEncryptionService : IEncryptionService
11      {
12          private int _ivSize = 16;
13
14          public int IVSize => _ivSize;
15
16  ✓ public byte[] Decrypt(byte[] cipherText, byte[] key, byte[] iv)
17      {
18          using Aes aes = Aes.Create();
19          aes.Key = key;
20          aes.IV = iv;
21
22          return aes.DecryptCbc(cipherText, iv, PaddingMode.PKCS7);
23      }
24
25  ✓ public byte[] Encrypt(byte[] plainText, byte[] key, byte[] iv)
26      {
27          using Aes aes = Aes.Create();
28          aes.Key = key;
29          aes.IV = iv;
30
31          return aes.EncryptCbc(plainText, iv, PaddingMode.PKCS7);
32      }
33
34  ✓ public byte[] Encrypt(byte[] plainText, byte[] key, out byte[] iv)
35      {
36          using Aes aes = Aes.Create();
37          aes.Key = key;
38          iv = aes.IV;
39
40          return aes.EncryptCbc(plainText, iv, PaddingMode.PKCS7);
41      }
42  }
```

The library I used required an iv to make the encryption more secure. To accommodate this, I prefixed every message with the iv so that it could be used during encryption and decryption.

The changes to the transmission service to implement this is shown below.

```

62  public async Task<T> ReceiveTransmission<T>() where T : class
63  {
64      if (!Initialised)
65          throw new Exception("Not initialised...");
66
67      var transmissionBytes = await _connectionService.ReceiveMessage();
68
69      byte[] iv = new byte[_encryptionService.IVSize];
70      byte[] cipherText = new byte[transmissionBytes.DataByteLength - iv.Length];
71
72      Buffer.BlockCopy(transmissionBytes.Data, 0, iv, 0, iv.Length);
73      Buffer.BlockCopy(transmissionBytes.Data, iv.Length, cipherText, 0, cipherText.Length);
74
75      byte[] plainText = _encryptionService.Decrypt(cipherText, _encryptionKey, iv);
76
77      string json = Encoding.UTF8.GetString(plainText, 0, plainText.Length);
78
79      return _jsonService.DeserializeJson<T>(json);
80  }
81
82  public async Task<int> SendTransmission(object transmission)
83  {
84      if (!Initialised)
85          throw new Exception("Not initialised...");
86
87      string transmissionJson = _jsonService.SerializeObject(transmission);
88      byte[] transmissionJsonBytes = Encoding.UTF8.GetBytes(transmissionJson);
89
90      byte[] cipherText = _encryptionService.Encrypt(transmissionJsonBytes, _encryptionKey, out byte[] iv);
91      byte[] finalTransmission = new byte[cipherText.Length + iv.Length];
92
93      Buffer.BlockCopy(iv, 0, finalTransmission, 0, iv.Length);
94      Buffer.BlockCopy(cipherText, 0, finalTransmission, iv.Length, cipherText.Length);
95
96      return await _connectionService.SendMessage(finalTransmission);
97  }

```

I also implemented a key exchanger, shown below. This was using an external library.

```

12  public byte[] CombineKeys(byte[] otherPublicKey, byte[] privateKey, byte[] publicKey)
13  {
14      using ECDiffieHellman exchanger = ECDiffieHellman.Create();
15
16      exchanger.ImportSubjectPublicKeyInfo(publicKey, out _);
17      exchanger.ImportECPrivateKey(privateKey, out _);
18
19      using ECDiffieHellman exchanger2 = ECDiffieHellman.Create();
20
21      exchanger2.ImportSubjectPublicKeyInfo(otherPublicKey, out _);
22
23      return exchanger.DeriveKeyMaterial(exchanger2.PublicKey);
24  }
25
26  public bool GeneratePublicPrivateKeyPair(out byte[] privateKey, out byte[] publicKey)
27  {
28      using ECDiffieHellman exchanger = ECDiffieHellman.Create();
29
30      publicKey = exchanger.ExportSubjectPublicKeyInfo();
31      privateKey = exchanger.ExportECPrivateKey();
32
33      return true;
34  }

```

To carry out the key exchange, I did the following in the InitialiseConnection method of the transmission service.

```
35 public async Task<bool> InitialiseConnection()
36 {
37     if (!_initialised)
38         return true;
39
40     try
41     {
42         _keyExchanger.GeneratePublicPrivateKeyPair(out var privateKey, out var publicKey);
43
44         await _connectionService.SendMessage(publicKey);
45
46         var messageBytes = await _connectionService.ReceiveMessage();
47
48         var otherPublicKey = messageBytes.Data;
49
50         _encryptionKey = _keyExchanger.CombineKeys(otherPublicKey, privateKey, publicKey);
51
52         _initialised = true;
53     }
54     catch
55     {
56     }
57
58     return _initialised;
59 }
60 }
```

I tested the whole system with wireshark.

Before encryption, the messages could be read very easily, as shown below:

The image shows a Wireshark packet capture of a TCP connection. The packet list on the left shows a sequence of packets, including a FIN, ACK, and a PSH, ACK packet. The packet details pane on the right shows the structure of the PSH, ACK packet, including the acknowledgment number (20357), window size (306944), and the data payload (405 bytes). The data payload is shown in hexadecimal and ASCII, and it contains a JSON message that is easily readable before encryption.

Packet 112: 91.533880 → 192.168.0.26 [PSH, ACK] Seq=489 Ack=20357 Win=306944 Len=405

Packet details:

- Acknowledgment Number: 20357 (relative ack num)
- Acknowledgment number (raw): 4072767208
- 0101 ... = Header Length: 20 bytes (5)
- Flags: 0x018 (PSH, ACK)
 - 000 ... = Reserved: Not set
 - 000 ... = Accurate ECN: Not set
 - 000 ... = Congestion Window Reduced: Not set
 - 000 ... = ECN-Echo: Not set
 - 000 ... = Urgent: Not set
 - 000 ... = Acknowledgment: Set
 - 000 ... = Push: Set
 - 000 ... = Reset: Not set
 - 000 ... = Syn: Not set
 - 000 ... = Fin: Not set
 - [TCP Flags: ...AP...]
- Window: 1199
 - [Calculated window size: 306944]
 - [Window size scaling factor: 256]
 - Checksum: 0x3f4c [unverified]
 - [Checksum Status: Unverified]
 - Urgent Pointer: 0
 - [Timestamps]
 - [SEQ/ACK analysis]
 - TCP payload (405 bytes)
 - Data (405 bytes)
 - Data [..]: 910100007b222474797065223a224f726966e
 - [Length: 405]

The data payload is shown in hexadecimal and ASCII, and it contains a JSON message that is easily readable before encryption.

After encryption was implemented, the messages could not be understood, as shown below.

```
E<?:@PP?aP{f1cSbmzqT6Ke
Mv
|2^G ,/)`\! [j=uG*51|w~&6GfK'T 16DzdK?o+B]Mc[W -=Ea 7CEkBHq{#q[T$*0%6F{
[['1"m%k/^
AzK 6?Ro#>c
Z-yurF`XN*k(Qq(
GfP(LhZk&4N(c#,!LvGg9wE[V89m_M<Yf@~kUZTpY;MZ4yMma\s1
yLSR/]3R
jH@(%?H-TTZhml/dwep8Qpf9h`GXEmNJ{ips#>eemKFP9/Y/!_1!WjuPOAiRWGAB.*Z9* [87S$ox
qI{Yh%8z'!%nf3:-%">crYq.@)Z}33.|le23J[*s(>%"
^[4]nbt bGK}%H;%1}E!sC<yGWg-HtoI
)\uWUY^
W}
a7c_2KycL_&4
YX9n8qain3;VD>j)M[)/dk98; ,9sM=i}?]Td
v?{/bF%[ZP {Y_E8a
4G9n<^C?|b=0ez^^E4kxctG1k),
L+,m4QMy&3N?@Hj$?W.,hdOS5mB")t,? <2T5G2e1"06jN%-S Y4jpE+0K\qv?y8w
1
```

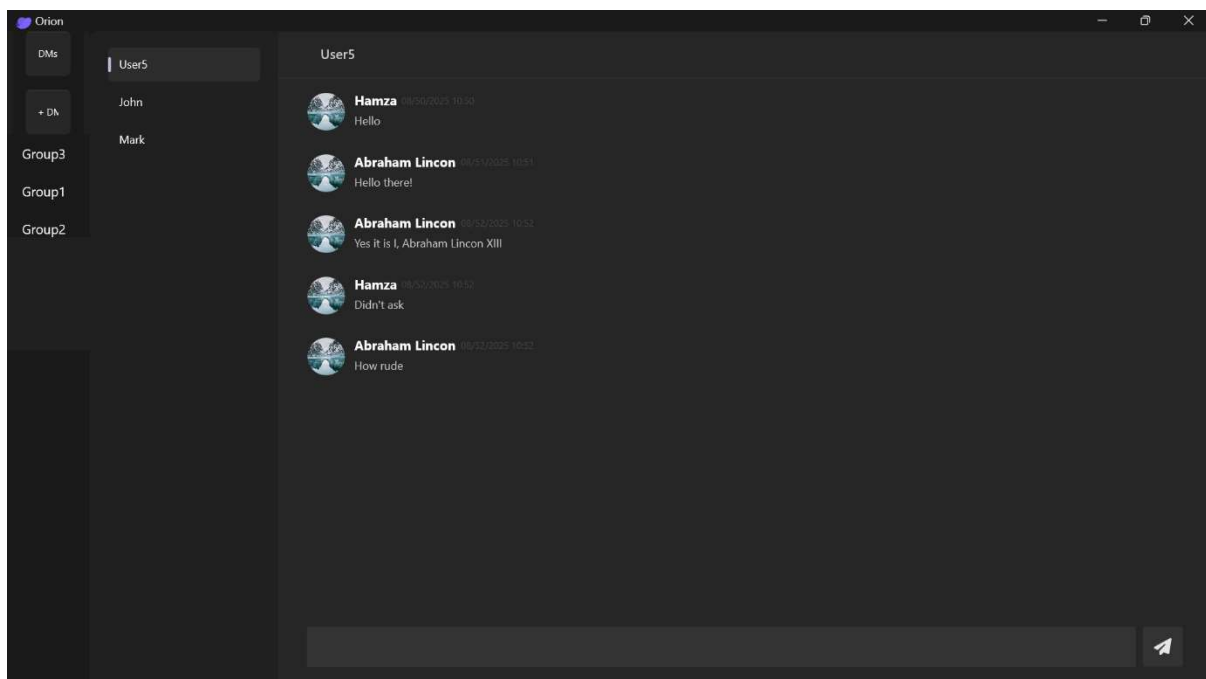
Sprint 7

GOALS

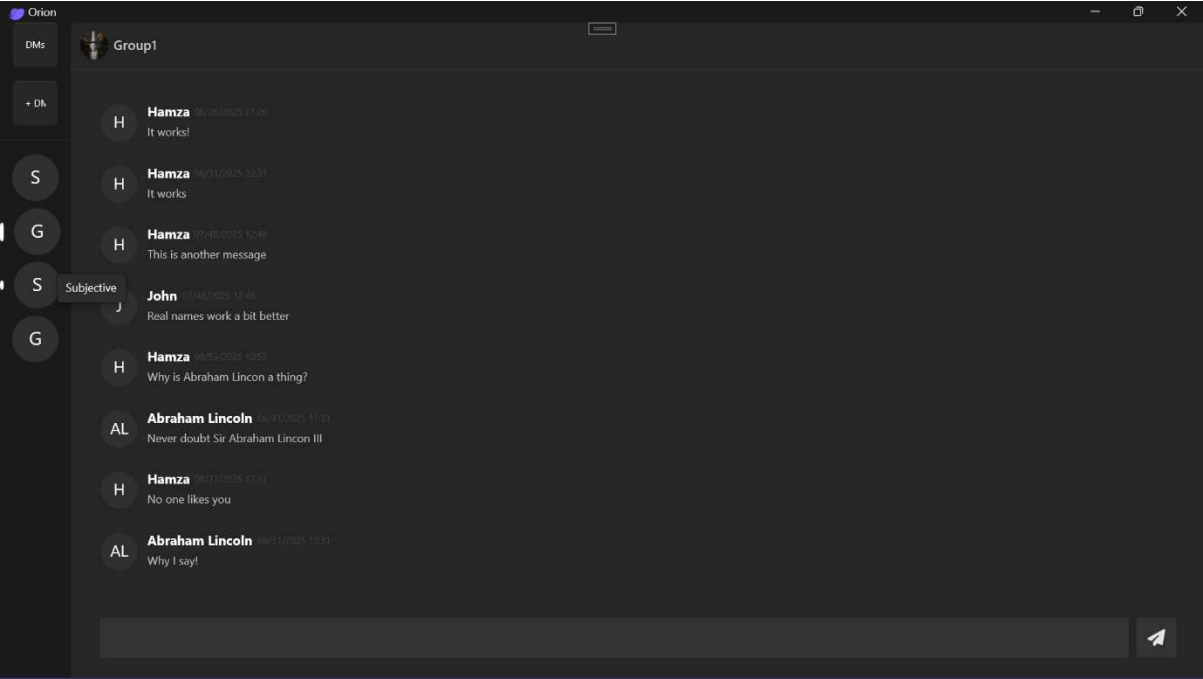
There is not a lot of time left, so I need to make the solution more usable, this will be done by design changes and error messages. The usability of the client app will be prioritised over the usability of the admin portal.

CODING

I started by adding a default user profile photo to the client app, there is not anything stored in the database but it is just shown on the client app.

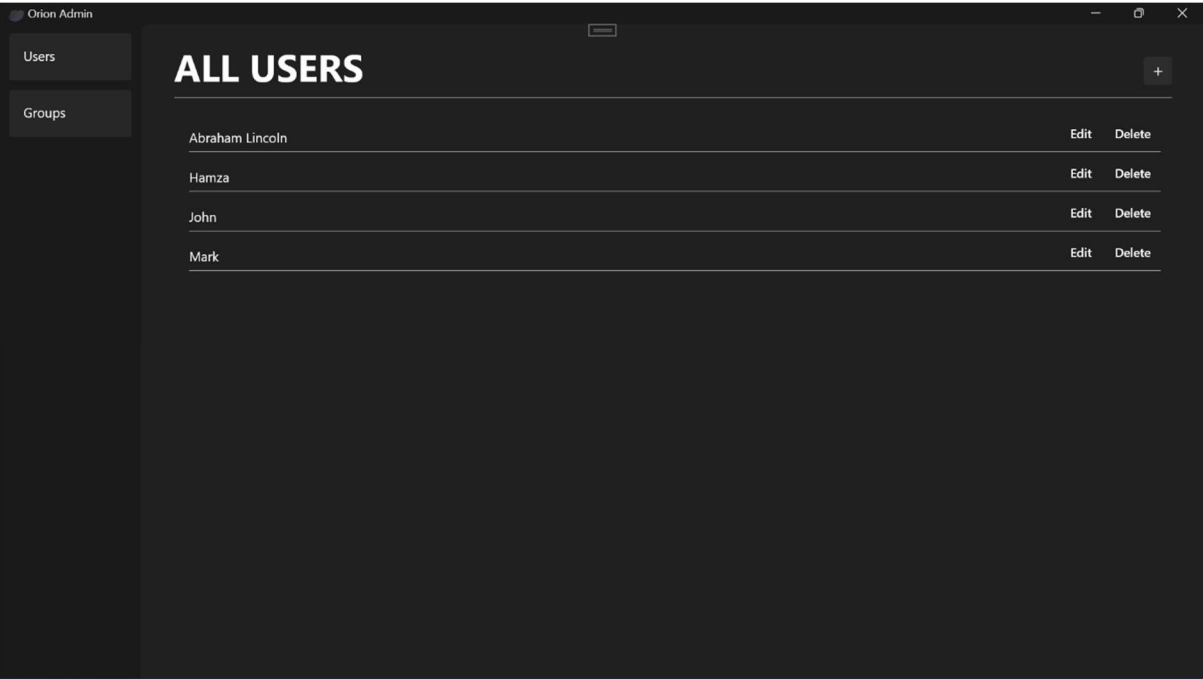


I changed the groups and removed the default profile photo and replaced it with the first few letters of a username or groupname. This is shown below.



I also added a tooltip to each group so that you could see the name of the group by hovering over it.

I changed the admin portal design as well.



I also did the same with groups.

CHAPTER 4 EVALUATION

Solution Overview & Post-Development Testing

A video of 2 client portals, the server & admin portal and the router running at the same time is shown below. In case it does not load, or it has very bad resolution, I have attached the video file along with my documentation.



As shown, there are 3 applications that make up the entire solution:

- Router – Routes messages between the server and user portals.
- Server & Admin Portal – The server with an interface that acts as an admin portal.
- User Portal – The user portal.

This differs greatly from the proposed 4 applications: Router, Server, User Portal and Admin Portal. Despite this, it is still usable and fulfils a good portion of the success criteria.

Final Solution Evaluation

SUCCESS CRITERIA COMPARISON

Success Criteria to Fulfilment Status Tables

Criteria to Fulfilment Status Table

Criteria	Fulfilment Status
1a: Messages update in real-time without needing a refresh.	Fulfilled
1b: Messages are reliably delivered to only the recipients requested (A message meant for only Alice should not be sent to Bob).	Fulfilled
2a: Users can add their own direct communications with other users.	Fulfilled
2b: Only users within a direct communication can see it and send messages within it.	Fulfilled
3a: All messages sent within a communication (direct messaging or grouped), that have not been deleted, are listed chronologically.	Fulfilled
3b: Last updated time or sending time is displayed with messages.	Fulfilled
3c: If messages are updated, their previous versions are not visible except in logs.	Fulfilled
3d: If messages are deleted, they cannot be viewed except in logs.	Fulfilled
4a: All transmissions should be symmetrically encrypted.	Fulfilled
4b: Sensitive information such as file attachments and user credentials should be asymmetrically encrypted when sending and receiving data.	Not fulfilled
5a: Normal users can send and receive messages.	Fulfilled
5b: Administrators can create and edit groups and users.	Partially Fulfilled – Super administrators can but administrators cannot since administrators are basically the same as normal users.
5c: Only super administrators can create other administrators.	Fulfilled
6a: Support for sending images and files (.txt, .pdf, .png, .jpeg – most common file types).	Not Fulfilled
7a: Messages sent to groups are received by every member of the group.	Fulfilled

Criteria to Fulfilment Status Table - Continued

Criteria	Fulfilment Status
7b: Everyone can see the members of any group they are part of.	Not Fulfilled
7c: Groups are displayed with the group's photo and name.	Partially Fulfilled – Groups do not have profile photos but their 'default' profile photo along with their name is displayed
8a: Admins can track app usage and activity via logs.	Not Fulfilled – The database needs to be manually queried to retrieve previous logs. Router logs are available for the lifetime of a Router instance.
8b: Every interaction is logged	Partially Fulfilled – Every interaction is logged but they are not accessible to the users, the logs are only stored in the database

USABILITY

User Portal

The User Portal has a lot of usability features:

- A user can see which group they are in using the group heading
- A selected group is highlighted in the navbar
- A user can see which direct communication they are in using the heading showing the recipient profile
- A user cannot send a message above 1000 characters, once they get above 200 characters a counter is displayed then, once they get above 1000 characters a useful validation message is shown, and the send button is disabled
- If a user enters above 1000 characters, their extra characters are not deleted so they do not lose any 'work' they might have done
- The user can also see what account they are signed in as, on the top left
- Hovering over groups or direct communication profiles also displays the full name of the group or the other member of the direct communication

When showing the application to the User, the stakeholder for the User Portal, they said the following:

"Orion has a very nice interface and was quite easy for me to use. It was very clear where I was and what I was doing, I could also clearly see who sent which message. I like how writing a big message gives me a counter that indicates how many characters I have left, and the textbox expands so I get a better preview of my message.

However, I was annoyed at the fact that pressing enter would not send the message but insert a new line and I would have preferred if I could simply press Shift+Enter to insert a new line and press enter

to send a message. I am also upset that there is no way for me to share files and attachments as that would have been very useful.”

Server & Admin Portal

The fact that the server and admin portal both run in the same application makes it very easy for administrators to accidentally close the server. When showing the application to the stakeholders, they, due to reflex, had closed the admin portal by accident numerous times causing the entire solution to have to be reloaded. In addition, since only a single server could be connected to the router at a single time and the server must be run on a specific node in the network, there is no flexibility for administrators to add/edit users/groups on any node in the network. However, the fact that the admin portal can only be accessed on a single node in the network makes it more secure since it is harder to get physical access to the admin portal.

Due to a lack of time, there were not many usability features built into the admin portal. The issues include:

- It is very easy for the administrators to delete users and groups by accident.
- When adding a user, the label for the password is quite confusing (the label states that it should be left blank to not change the password)
- When changing or adding a password for a user, there is no validation to ensure the administrator creates a secure password
- The administrator can remove all administrators including themselves in which case the only way to be able to have another administrator is by manually accessing and changing the database
- It is very easy to accidentally make another user an administrator

If administrators are trained on how to use the admin portal effectively and are very careful when they use it, it could be used. However, due to human nature, it is very easy to make a mistake, and the application provides no functionality for handling mistakes.

After showing the Administrator how to use the Admin Portal and explaining how it is joined to the server, they tried to use the Admin Portal. They said the following:

“The user interface looks quite nice, and it is easy for me to navigate the application and manage users and groups.

It is annoying that it is very easy for me to accidentally stop the server, it would have been useful to have a popup that shows when closing the app so that I do not accidentally close it. I will have to be extra careful when I am deleting users and groups, and it would be useful if I could have a popup of some sort that confirms whether I want to delete that user or group.”

Router

The router was always supposed to be a console app, there were never meant to be any usability features.

PERFORMANCE

Performance Testing

I tested the solution on a computer with the following specifications (all the instances of all the applications were run on this computer at the same time during testing):

- 8GB RAM
- Intel Core i5 8th Gen 8250U CPU

With this, I was able to run up-to 5 instances of the client app smoothly (these 5 are not including the server and the router, these were still run on the same computer). However, I was not able to take any screen recordings or screen shots. The client applications did not crash and neither did the router or the server.

Since I was able to achieve this with the limited specs detailed above, with a dedicated server with much higher specifications for running the Server and Router, the solution should be able to sustain much more connections, possibly up to 20. Since the company is quite small, this should suffice.